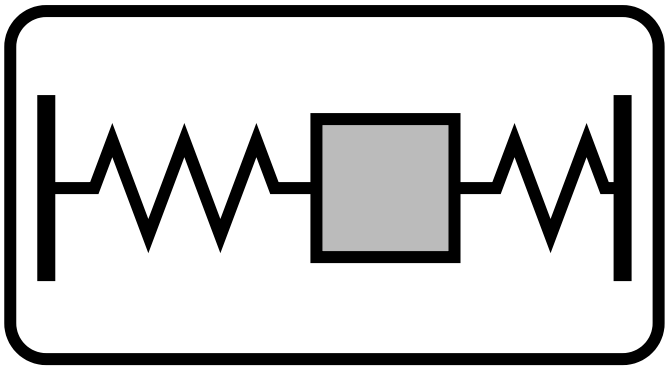
AN ENGINEERING GUIDE



INSIDE THE

VIBRATION LAB

How a Lagrangian physics engine, a click-and-draw design interface and a direct-to-video recorder were built — every equation derived, every function explained, every line accounted for.

BY TEAM NOVA

First edition · 2026

LAGRANGIAN MECHANICS · RK4 · C++17 · WIN32 · GDI+ · MEDIA FOUNDATION · H.264

Inside the Vibration Lab
by Team NOVA

First edition, 2026.

This guide describes the Vibration Lab program that lives in the folder *examples/vibration-lab* of the NOVA IDE project. It is a companion to *How to Build the NOVA C-IDE*.

Every source listing in this guide is copied automatically from the real files at the moment the PDF is built, so the listings always match the program. Every number printed as a test result was produced by running the program's own self-test. All screenshots are real, taken from the program itself and converted to black and white for printing.

Names such as Windows, Media Foundation, H.264, GDI+ and MPEG-4 belong to their respective owners.

Set in Times New Roman, 12 point, with 1.5 line spacing.

Contents

Preface: Read This First 1
What this guide is 1
Who this guide is for 2
How the guide is organised 2
The special boxes 2
About the code listings 3
An honest word 3

PART I — THE IDEA 4

Chapter 1. What We Built 5
1.1 A tour of the finished program 5
1.2 The requirements we started from 6
1.3 The five source files 7
1.4 Building and running 8
1.5 Chapter checks 8

Chapter 2. From Newton to Lagrange 9
2.1 Newton's way, and where it hurts 9
2.2 The big idea: use energy, not forces 9
2.3 Generalized coordinates: how many numbers describe the system? 10
2.4 The Lagrangian and Hamilton's principle 10
2.5 Deriving the Euler–Lagrange equation 11
2.6 What Hamilton's principle leaves out: damping and pushes 12
2.7 The first worked example: one mass, one spring, one damper 12
2.8 Rails: constraints made easy 13
2.9 Why Lagrange suits a program that users extend by clicking 13
2.10 Chapter checks 14

Chapter 3. Deriving Our Equations 15
3.1 Step 1: the coordinates 15
3.2 Step 2: the geometry of one link 15
3.3 Step 3: the spring's potential energy and its force 16
3.4 Step 4: the damper's dissipation and its force 16
3.5 Step 5: gravity 17
3.6 Step 6: the outside forces 17
3.7 Step 7: assemble the equation of motion 18
3.8 Step 8: the law of energy, derived 18
3.9 Step 9: the exact answer for one mass, to test against 19
3.10 Step 10: a worked numerical example 20
3.11 Step 11: another physical effect, the static sag 21
3.12 Chapter checks 22

Chapter 4. How a Computer Solves It: RK4 23
4.1 The problem with exact answers 23
4.2 Turning second-order into first-order 23
4.3 The simplest method, and why it fails 23
4.4 RK4: four peeks, one weighted guess 24
4.5 How small should a step be? Stability 25

4.6 The accumulator: real time versus physics time 25
4.7 Keeping the energy books: the trapezoid rule 26
4.8 Finding the dominant frequency 26
4.9 Checking the solver 27
4.10 Chapter checks 27

PART II — THE ENGINE, LINE BY LINE 29

Chapter 5. The Model: engine.hpp, Part 1 30
5.1 The file header and the namespace 30
5.2 Constants and enumerations 31
5.3 Node: one thing on the canvas 31
5.4 Link: a spring or a damper 32
5.5 Model, part 1: the container and two helper functions 33
5.6 Model, part 2: adding things 34
5.7 Model, part 3: removing things 35
5.8 Model, part 4: automatic rest length 36
5.9 State: the numbers that change while it runs 36
5.10 Chapter checks 37

Chapter 6. The Physics: engine.hpp, Part 2 39
6.1 linkInfo: the geometry of a link 39
6.2 kinetic: T 40
6.3 potential: V 41
6.4 rayleigh: D 42
6.5 driveForce: the harmonic push 42
6.6 inputPower: P = Q · v 42
6.7 accel: the Euler–Lagrange equations 43
6.8 Chapter checks 45

Chapter 7. The Integrator and the Meter: engine.hpp, Part 3 46
7.1 rk4: one step of the fourth-order Runge–Kutta method 46
7.2 The Meter: checking the law of energy 47
7.3 advance: step and keep the books 48
7.4 dominantFrequency: the frequency finder 49
7.5 Chapter checks 50

Chapter 8. Saving and Examples: model_io.hpp 51
8.1 The file format 51
8.2 saveModel 51
8.3 loadModel 52
8.4 The four examples 54
8.5 Chapter checks 56

PART III — THE INTERFACE, LINE BY LINE 57

Chapter 9. How a Windows Program Works 58
9.1 A program that waits for events 58
9.2 Two kinds of loops: waiting and polling 59
9.3 Drawing: GDI+, brushes and pens 59
9.4 Double buffering: never draw straight onto the screen 60
9.5 The coordinate systems in this program 60
9.6 How the files fit together 61
9.7 Chapter checks 61

Chapter 10. The Brain: app.hpp 63
10.1 The header and the vocabulary 63
10.2 The Frame: one row of telemetry 64
10.3 The App struct: all the memory of the program 64
10.4 Small helpers: time, clamping, toast 66
10.5 Screen layout 66
10.6 Converting between world and screen 67
10.7 logFrame: recording the telemetry 68
10.8 Controlling the simulation 69
10.9 Hit testing: what did the user click? 72
10.10 Editing 73
10.11 The property sliders 74
10.12 Buttons 76
10.13 The mouse 78
10.14 Chapter checks 80
Chapter 11. The Artist: render.hpp 82
11.1 The header 82
11.2 Fonts, colours and the first helpers 83
11.3 Springs and dashpots, drawn flat 84
11.4 drawNodeBody: masses and anchors 85
11.5 drawLink: a spring or damper at any angle 87
11.6 drawScene: assembling the canvas 89
11.7 drawHud: the telemetry box 90
11.8 Plots 92
11.9 The right-hand panel 96
11.10 render: putting the layers together 97
11.11 Chapter checks 98
Chapter 12. The Receptionist: live.cpp 99
12.1 The header and includes 99
12.2 The Canvas: our hidden picture 100
12.3 Files and folders 101
12.4 Exporting the telemetry to CSV 101
12.5 Reading the dominant frequency 102
12.6 The keyboard 103
12.7 The window procedure 104
12.8 Reading the command line 105
12.9 main: the front door 106
12.10 Chapter checks 109
PART IV — THE RECORDER AND THE TESTS 110
Chapter 13. The Direct-to-Video Recorder 111
13.1 What a video really is 111
13.2 Why we did not write our own encoder, and did not use ffmpeg 112
13.3 A very short COM primer 112
13.4 The Sink Writer pipeline 113
13.5 encoder.hpp: the header comment and includes 113
13.6 The class and its small accessors 114
13.7 open: set everything up 114
13.8 addFrame: send one picture 116
13.9 close: finish the file 118

13.10 The private part: helpers and cleanup 118
13.11 Connecting the recorder to the program (live.cpp) 119
13.12 The recording clock: why the video has the right speed 121
13.13 How we verified the recorder 121
13.14 Chapter checks 122

Chapter 14. Testing: How We Know It Is Right 124
14.1 Running the tests 124
14.2 The test machinery 124
14.3 Check 1 and 2: one mass against the textbook 125
14.4 Check 3: a driven pair with a wiggling mouse 126
14.5 Check 4: a frictionless spring pendulum 126
14.6 Check 5: rails 127
14.7 Check 6: the frequency finder 127
14.8 Check 7: save and load 127
14.9 Check 8: the video encoder 128
14.10 Check 9: designing by clicking, then running 128
14.11 What each test would catch 129
14.12 The test output 130
14.13 What the tests cannot tell you 130
14.14 Chapter checks 130

PART V — USING IT AND GROWING IT 132

Chapter 15. The User Manual, with Experiments 133
15.1 Running the program inside NOVA 133
15.2 The screen at a glance 134
15.3 Experiment 1: build a mass on a spring from scratch 134
15.4 Experiment 2: resonance 135
15.5 Experiment 3: a car suspension 136
15.6 Experiment 4: a hardening spring 137
15.7 Experiment 5: a spring pendulum 137
15.8 Experiment 6: record a video 138
15.9 What every number in the telemetry means 139
15.10 Keys and mouse 139
15.11 When something goes wrong 140
15.12 Chapter checks 140

Chapter 16. Where to Go Next 141
16.1 Level 1: small and safe 141
16.2 Level 2: new physics 141
16.3 Level 3: better numerics 142
16.4 Level 4: the interface and the recorder 142
16.5 Level 5: larger projects 142
16.6 A habit for every project 143
16.7 Chapter checks 143

Appendix A. Glossary 144

Appendix B. Maths to Code 146

Appendix C. Files and Folders 147

Preface: Read This First

What this guide is

This guide explains one program from the inside: the **Vibration Lab**. The Vibration Lab lets you draw a system made of **masses**, **springs**, **dampers** and **ground anchors** on a canvas, press **Run**, and watch it move. It shows you the numbers while it moves (the *telemetry*), and it can record everything to an **.mp4 video** file.

Three things sit underneath the pictures, and this guide explains each one fully:

1. **The physics engine.** A small piece of C++ that knows the laws of motion. It is built with a method called the **Lagrangian approach**. You will see where every equation comes from, and exactly which line of code each equation became.
2. **The interface.** The windows, buttons, sliders, mouse handling and drawing. It is plain C++ talking to Windows. No game engine and no big library.
3. **The video recorder.** A "direct to video" encoder that takes the picture on screen, 60 times a second, and writes it into an H.264 video file, using the video encoder that is already inside Windows.

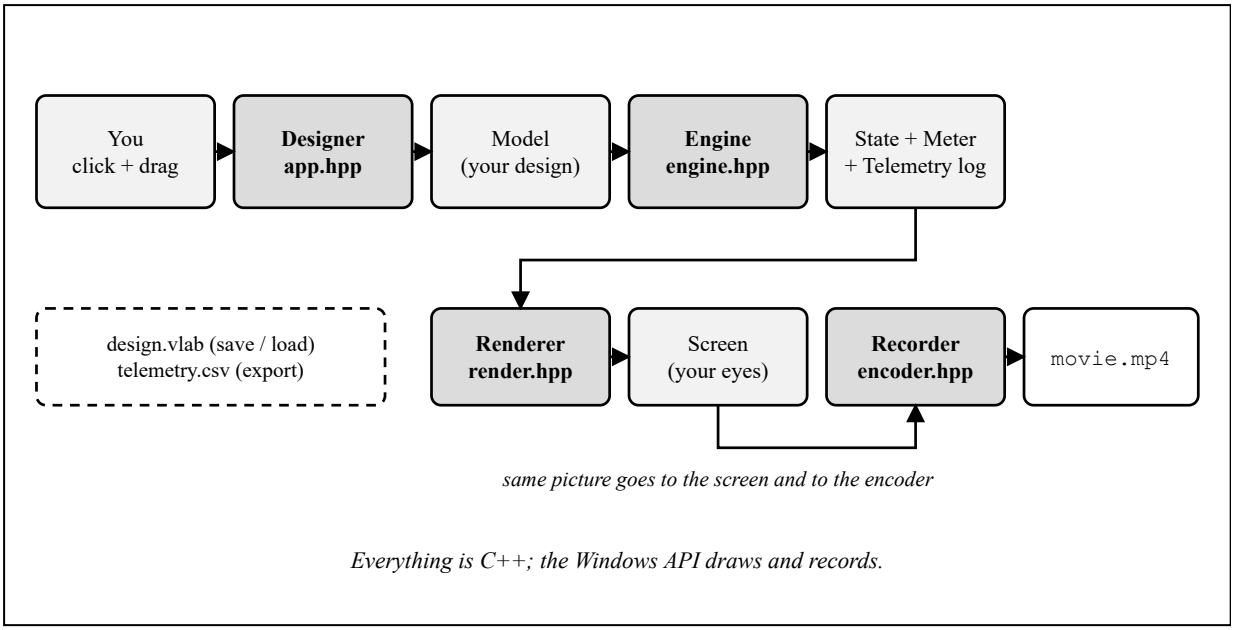


Figure 0.1. *The whole program in one picture: what you do, what the computer does with it, and where the results go.*

Who this guide is for

You do not need to know advanced mathematics. If you know what a *function* is, and you have seen a little C or C++, you can follow every step. Every strange word is explained the first time it appears, and all of them are collected again in the glossary at the back (Appendix A).

If you want the long, slow introduction to C++, the other book in this series, *How to Build the NOVA C-IDE*, has full chapters on C and C++. This guide is the deep dive into one project.

How the guide is organised

Part	What is in it
I. The Idea	What we built, and the physics: Newton's way versus Lagrange's way, the derivation of our equations, and how a computer solves them (RK4).
II. The Engine, Line by Line	The files `engine.hpp` and `model_io.hpp`, every function, every line, each tied back to an equation.
III. The Interface, Line by Line	How a Windows program works, then `app.hpp`, `render.hpp` and `live.cpp`.
IV. The Recorder and the Tests	How video works, the Media Foundation encoder in `encoder.hpp`, and the nine self-tests that prove the program is right.
V. Using It and Growing It	A step-by-step user manual with worked experiments, and a list of projects to build next.
Appendices	Glossary, a table that links every symbol in the maths to a name in the code, and a quick-reference of keys.

The special boxes

> **WHY?**
> A **WHY?** box explains the reason behind a choice. Knowing *why* a line is written the way it is lets you change it safely.

> **WHAT IF IT WERE DIFFERENT?**
> A **WHAT IF** box shows what would break if the code were different. Seeing the failure makes the correct version easy to remember.

> **MATHS TO CODE**
> A **MATHS TO CODE** box places an equation next to the line of code that carries it out. These are the most important boxes in Part II.

> **KEY WORD**
> A **KEY WORD** box defines a piece of jargon in plain English.

> **TRY IT**
> A **TRY IT** box asks you to do something in the program and watch the result.

> **CHECKPOINT**
> A **CHECK** box says what you should see. If you do not see it, stop and find out why.

About the code listings

Every listing in this guide starts with its file name and the range of line numbers it covers, and each line carries its own number on the left. The listings are copied **automatically from the real files** when the PDF is built. That means they can never be out of date, and when the text says "line 142" it means line 142 of the real file.

> **NOTE**
> The program is for Windows 10 or 11. It needs only a C++17 compiler (g++ from MSYS2 was used) and CMake. The libraries it uses (`user32`, `gdi32`, `gdiplus`, `mfplat`, `mfreadwrite`) are part of Windows.

An honest word

Reading explains; **doing** teaches. The best way to use this guide is to keep the program open beside it, change one thing at a time, and watch what happens. Every chapter ends with checks and exercises for exactly that reason.

Welcome to the Vibration Lab.

Part I — The Idea

CHAPTER 1

What We Built

1.1 A tour of the finished program

The Vibration Lab is one window. The big area at the top-left is the **canvas**, where you draw. The area below it holds live **plots**. The strip on the right is the **panel**, with tools, buttons and sliders. The box in the corner of the canvas is the **telemetry** readout.

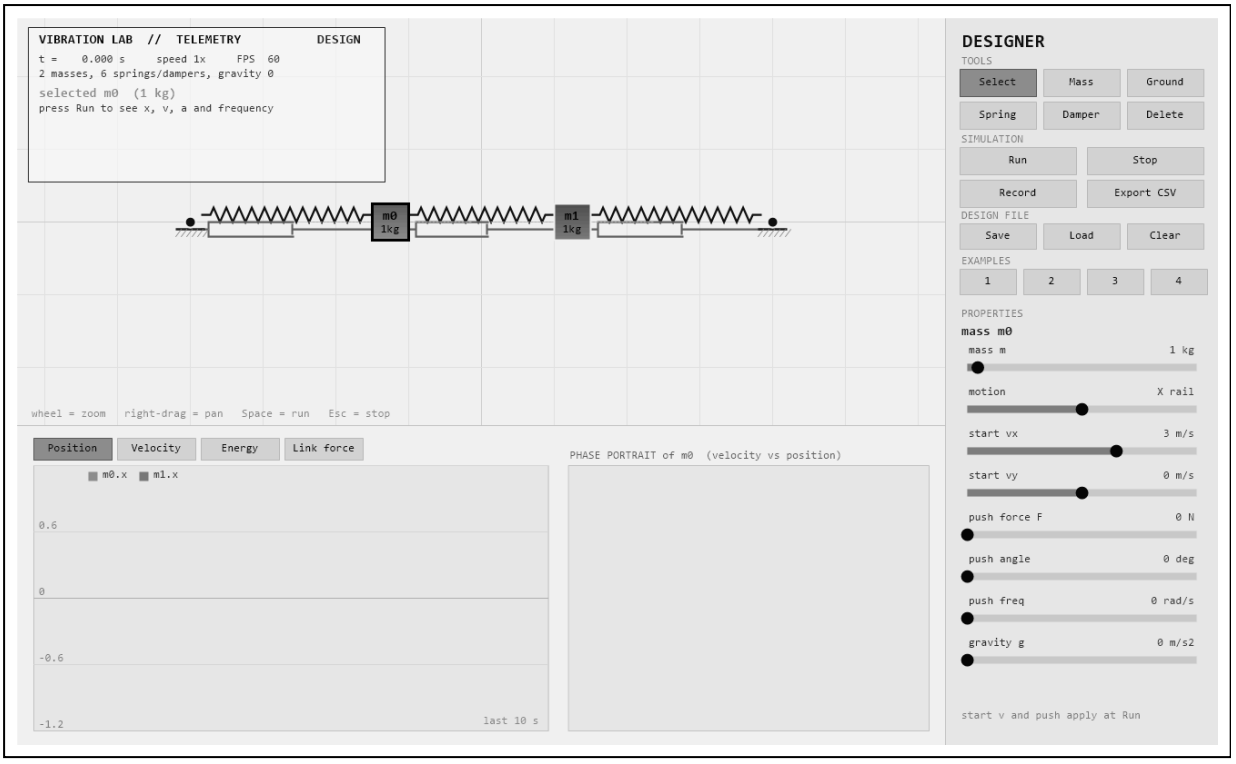


Figure 1.1. *The Vibration Lab in DESIGN mode: two masses on a rail between two walls (example 2). The right panel holds the tools and the sliders for whatever is selected.*

You always do things in the same order:

1. **Design.** Pick the **Ground** tool and click to place a wall or anchor. Pick the **Mass** tool and click to place a block. Pick **Spring** (or **Damper**) and click two things to join them.
2. **Tune.** Pick the **Select** tool, click a mass or a link, and use the sliders to set its mass, stiffness, damping, starting speed and so on.
3. **Run.** Press **Run** (or the space bar). The physics starts. You can still drag any mass with the mouse, as if you were holding it with a rubber band.
4. **Read the telemetry.** Numbers, plots and a phase portrait show what the system is doing. The *energy-law error* shows whether the simulation obeys the laws of physics.

5. **Record.** Press **Record** (or F9). The picture is written to an .mp4 file in the folder `recordings`. Press **Export CSV** to save every number to a spreadsheet file.

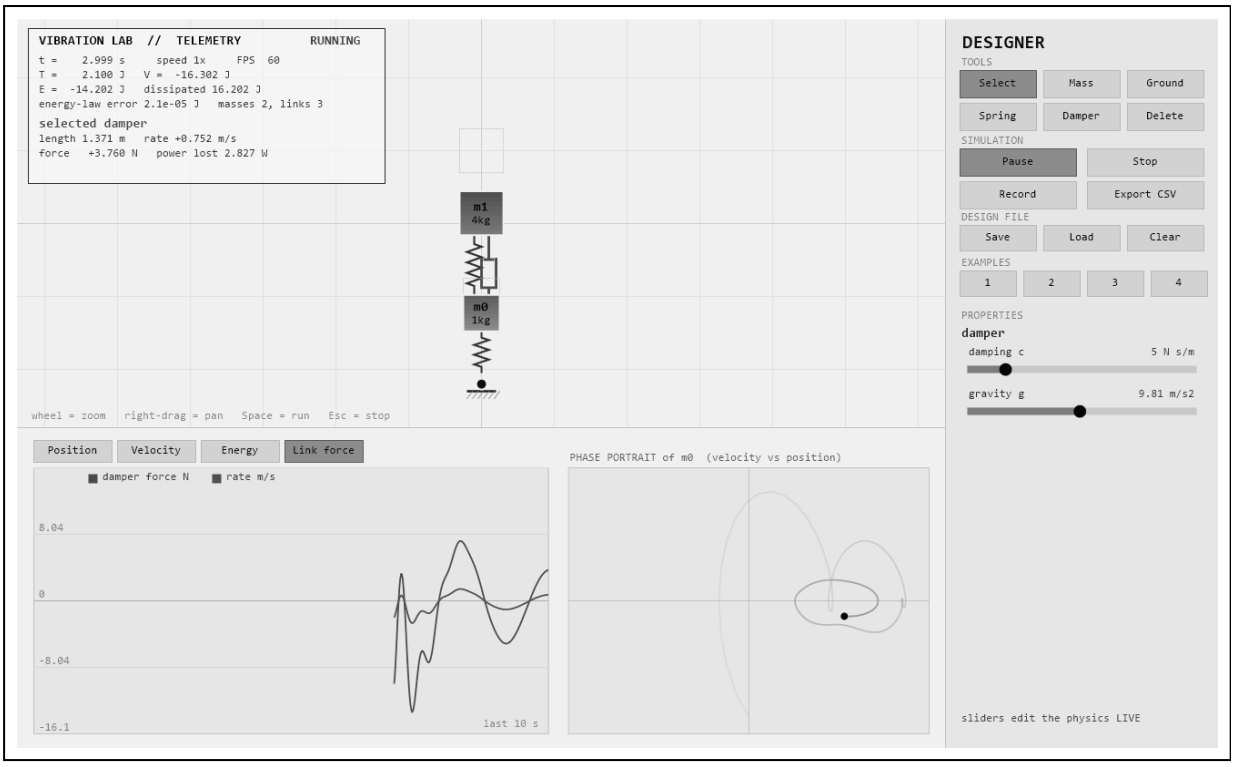


Figure 1.2. *A running simulation (example 4, a quarter-car suspension). The damper is selected, so the telemetry box shows its force and the bottom plot shows that force over time.*

1.2 The requirements we started from

Before writing code, we wrote down what the program had to do. A list like this is called the **requirements**. Each one later became a feature, and each feature became code you will meet in this guide.

#	Requirement	Where it lives
1	Simulate masses, springs, dampers and fixed ground, in two dimensions	`engine.hpp`
2	Derive the equations of motion with the Lagrangian method, not with force diagrams	Chapters 2 and 3, `accel()`
3	Never "blow up" for sensible inputs, and prove it is right	RK4 in Chapter 4, tests in Chapter 14
4	Let the user design a system by clicking, with no code to edit	`app.hpp`
5	Show telemetry (numbers and plots) while it runs	`render.hpp`, `Frame` log
6	Record the screen to a standard video file	`encoder.hpp`
7	Keep designs between sessions, and export data	`model_io.hpp`, CSV export

#	Requirement	Where it lives
8	Run well on a plain Windows PC with nothing extra installed	Win32 + GDI+ + Media Foundation

1.3 The five source files

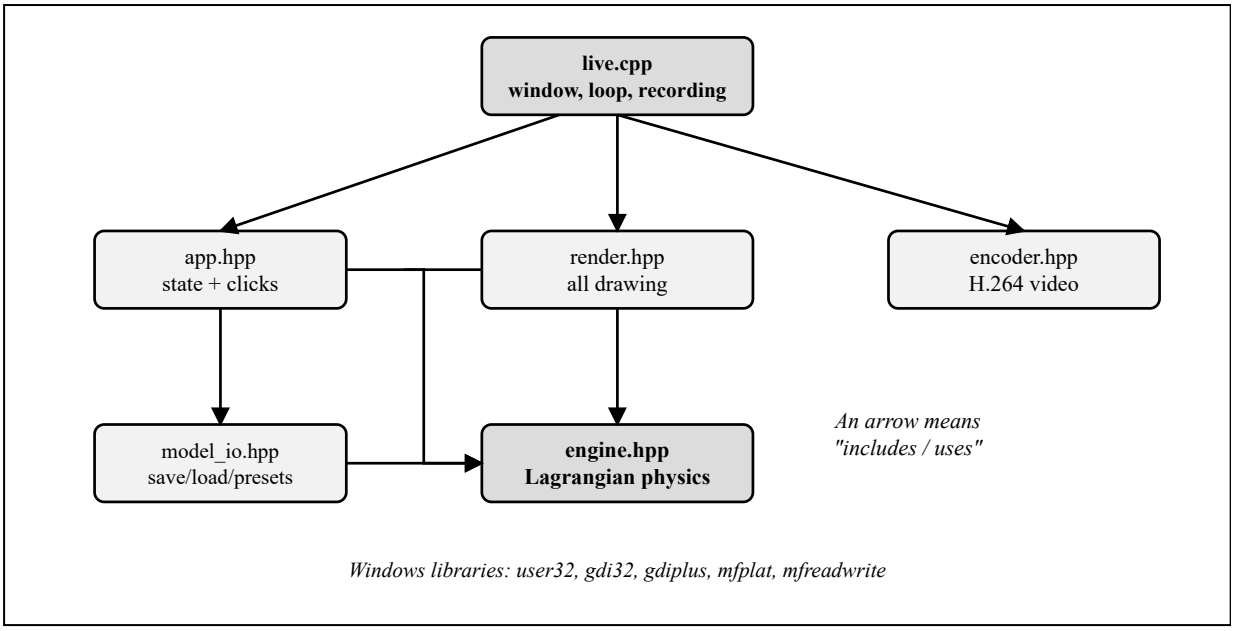


Figure 1.3. *The five source files and who includes whom. The physics (engine.hpp) knows nothing about windows or pictures, which is why it can be tested without opening a window.*

WHY?

Why is the physics kept in its own file that knows nothing about drawing? Because then the physics can be tested on its own, quickly and exactly (Chapter 14), and later reused in a different program: a web page, a game, a command-line tool. This idea is called **separation of concerns**: each file has one job.

File	Lines	Its one job
`engine.hpp`	about 310	The model of the system and the laws of motion
`model_io.hpp`	about 130	Save, load and ready-made example designs
`app.hpp`	about 430	What the program remembers, and what every click and key does
`render.hpp`	about 490	Everything painted on screen
`encoder.hpp`	about 150	Writing frames into an H.264 video file
`live.cpp`	about 460	The window, the main loop, recording, CSV export, the self-test

The header files end in `.hpp` because they contain code that other files include. The program has exactly **one** file that gets compiled on its own: `live.cpp`. It includes the others. That keeps the build simple: `CMakeLists.txt` has a single `add_executable` line.

1.4 Building and running

TRY IT

1. Open the folder `examples/vibration-lab` in NOVA IDE. The run-target box at the top should say `vibration_live`.
2. Press the green Run triangle. CMake configures, g++ compiles, and a window opens.
3. Press **Run** inside the window and drag a block with the mouse.

From a terminal the same thing is:

SHELL

```
cmake -S . -B build -G Ninja -DCMAKE_CXX_COMPILER=g++
cmake --build build
build/vibration_live.exe              # opens the window
build/vibration_live.exe --selftest   # runs the nine checks, no window
```

CHECKPOINT

`--selftest` should end with the words **all checks passed**. Chapter 14 explains each of the nine checks.

1.5 Chapter checks

QUIZ

1. Name the three big parts of the program, in one word each.
2. Why is `engine.hpp` separate from `render.hpp`?
3. What do the buttons Run, Stop and Record each do?
4. Which file is compiled on its own?

ANSWERS

1. Physics (engine), interface (designer), recorder (video).
2. So the physics can be tested without a window, and reused elsewhere.
3. Run starts or pauses the simulation, Stop returns to design mode, Record starts or finishes the video file.
4. `live.cpp` (it includes the other files).

CHAPTER 2

From Newton to Lagrange

2.1 Newton's way, and where it hurts

You probably learned physics the Newton way: draw the object, draw every force on it, add the forces, and say **F = m a**. For one block on one spring this is easy. For a real machine it becomes painful:

- A machine with ten parts needs ten force diagrams.
- Some forces are **constraint forces** (the force a rail pushes back with so the block cannot leave it). They are unknown at the start, and you must solve for them.
- Every time you add a part, you must redraw diagrams and redo the algebra.

A program that lets the *user* add parts by clicking cannot ask the user to redo algebra. We needed a method that works for **any** collection of parts with the same recipe, every time. That method is the **Lagrangian approach**, invented by Joseph-Louis Lagrange in 1788.

2.2 The big idea: use energy, not forces

Forces point in directions, so adding them needs care. **Energy is just a number.** Energy has no direction. Lagrange's idea: describe the system by its **energies**, and let a short, fixed piece of calculus turn those energies into the equations of motion.

You need only three energy-like quantities:

> **KEY WORD**
>
> **Kinetic energy (T).** Energy of motion. For a mass: $T = \frac{1}{2} m v^2$.
>
> **Potential energy (V).** Energy stored by position. A stretched spring stores $V = \frac{1}{2} k s^2$. A raised mass stores $V = m g h$.
>
> **Dissipation (D).** A measure of how fast energy is being turned into heat by friction-like parts such as dampers.

2.3 Generalized coordinates: how many numbers describe the system?

To describe where everything is, you need a list of numbers. The list is called the **generalized coordinates**, written **q**. You choose the smallest list that fully fixes the position of every part.

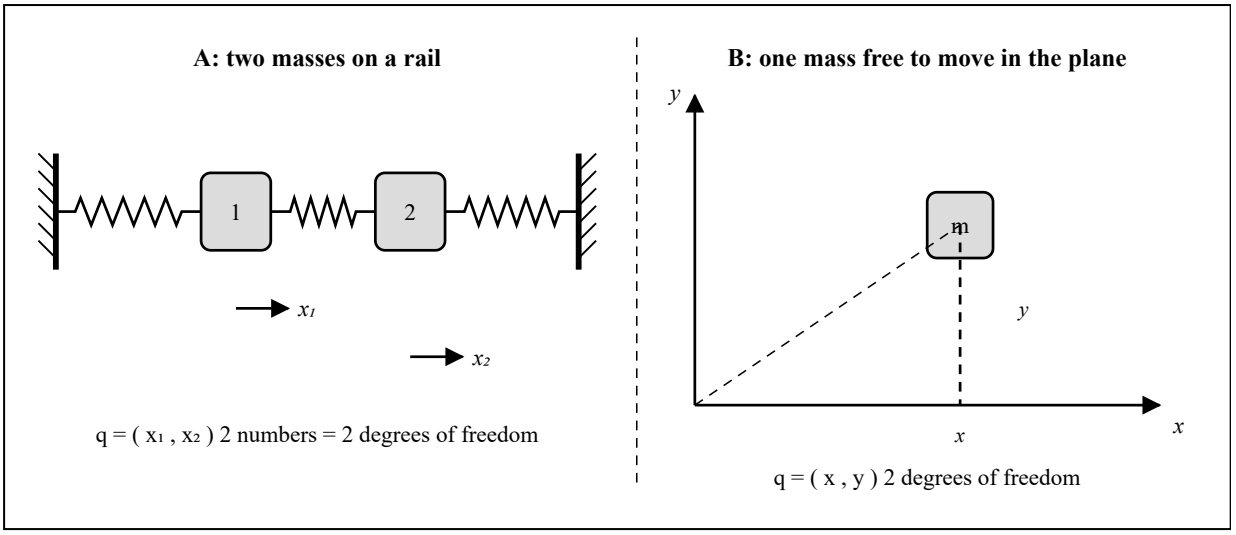


Figure 2.1. *Left: two masses on a rail need two numbers. Right: one mass free in the plane also needs two numbers.*

> **KEY WORD**
>
> **Degrees of freedom (DOF).** The number of independent numbers needed to describe the position of the system. Two masses on a rail: 2 DOF. One free mass in a plane: 2 DOF. A free mass in a plane that is forced onto a rail: only 1 DOF.

Our engine stores *two* numbers for every node (`x` and `y`), and then **removes** a degree of freedom for any mass that is on a rail. That is simpler to code than choosing a different coordinate list for every design.

2.4 The Lagrangian and Hamilton's principle

Write the **Lagrangian** as the kinetic energy minus the potential energy:

$$L = T - V \tag{2.1}$$

Why subtract? Nobody can give a one-line reason; it is simply what works. A deeper justification is **Hamilton's principle**: out of all the paths a system could follow between two moments in time, the real path is the one that makes the **action** *stationary*, where

$$S = \int L \, dt \ (\text{from } t_1 \text{ to } t_2) \tag{2.2}$$

"Stationary" means that if you nudge the path a tiny bit, the action does not change to first order, in the same way that the bottom of a bowl is flat for a tiny nudge.

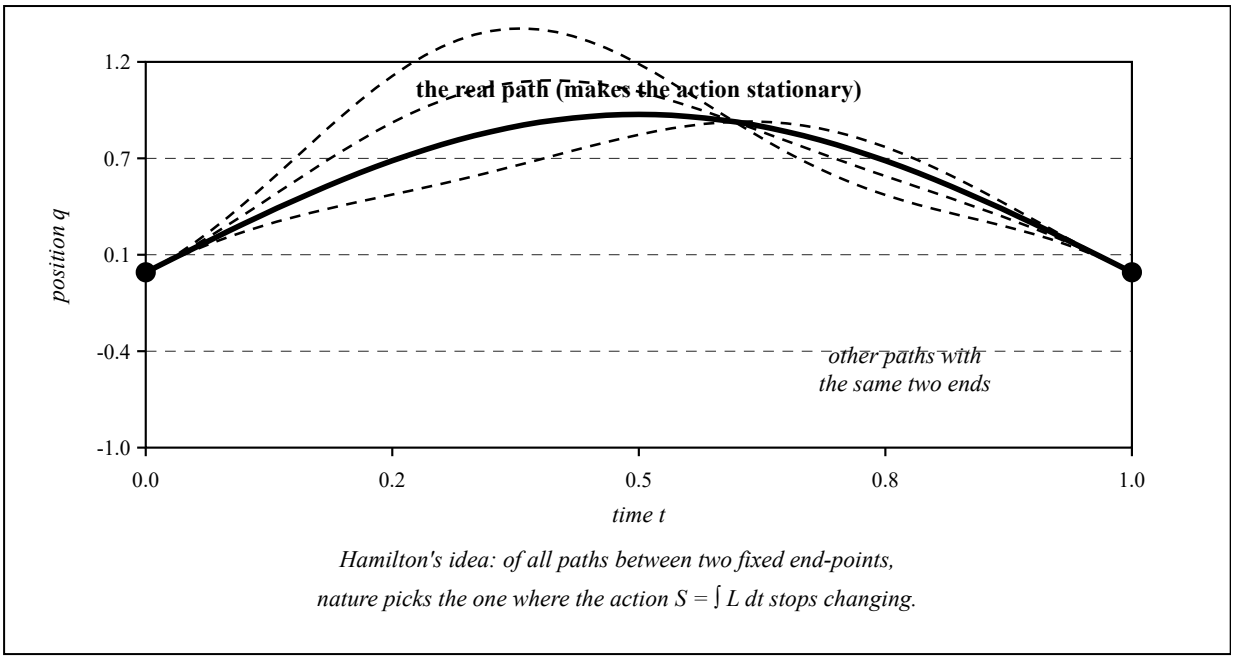


Figure 2.2. *The real path (thick) and some nearby nudged paths (dashed). All start and end at the same two points. The real path is the one where the action stops changing.*

2.5 Deriving the Euler–Lagrange equation

This short derivation is the engine of the whole method. Take it slowly; each line is simple.

Step 1. Suppose `q(t)` is the real path. Make a nudged path `q(t) + ε η(t)`, where ε is a small number and η(t) is any smooth shape that is **zero at both ends** (the nudged path must start and finish at the same points).

Step 2. The action of the nudged path is a function of ε. If the real path is stationary, its derivative with respect to ε is zero at ε = 0:

$$dS/d\varepsilon = \int \left(\partial L/\partial q \cdot \eta + \partial L/\partial \dot{q} \cdot \dot{\eta} \right) dt = 0 \tag{2.3}$$

(We used the chain rule: L depends on q and on the speed $\dot{q}$.)

Step 3. Integrate the second term by parts. That moves the time-derivative from η onto the other factor and produces a minus sign:

$$\int \partial L/\partial \dot{q} \cdot \dot{\eta}\, dt = [\, \partial L/\partial \dot{q} \cdot \eta \,] - \int d/dt(\partial L/\partial \dot{q}) \cdot \eta\, dt \tag{2.4}$$

The bracket term is zero, because η is zero at both ends.

Step 4. Put it together:

$$\int \left(\partial L/\partial q - d/dt\left(\partial L/\partial \dot{q} \right) \right) \cdot \eta \, dt = 0 \tag{2.5}$$

This must be true for **every** shape η. The only way that can happen is for the bracket to be zero at every moment:

$$\boldsymbol{d/dt\left(\partial L/\partial \dot{q} \right) - \partial L/\partial q = 0} \tag{2.6}$$

That is the **Euler–Lagrange equation**, one equation for each coordinate q. Notice what happened: we never mentioned a force, a rail, or a free-body diagram.

2.6 What Hamilton's principle leaves out: damping and pushes

Hamilton's principle describes systems that keep their energy. Real systems have dampers (which turn energy into heat) and outside pushes (a motor, or your mouse). Both fit into the same equation, with two extra terms:

$$d/dt\left(\partial L/\partial \dot{q} \right) - \partial L/\partial q + \partial D/\partial \dot{q} = Q \tag{2.7}$$

> **KEY WORD**
>
> **Rayleigh dissipation function (D).** For a viscous damper (a dashpot whose force is proportional to speed), D = ½ c ṡ², where c is the damping coefficient and ṡ is how fast its two ends are moving apart. It is named after Lord Rayleigh. The term ∂D/∂q̇ gives the damper's force.
>
> **Generalized force (Q).** The outside forces, written in the coordinates we chose. For a push F on a mass that moves along coordinate x, Q = F.

> **WHY?**
>
> **Why a damper's force is c·ṡ.** Differentiating D = ½ c ṡ² with respect to the speed gives c ṡ: a force that grows in step with the speed, and which opposes it. This is exactly how a shock absorber behaves, which is why D is the right tool.

2.7 The first worked example: one mass, one spring, one damper

Let the coordinate be x (the displacement from rest). Then:

$$T = \tfrac{1}{2} m \dot{x}^2, \; V = \tfrac{1}{2} k x^2, \; D = \tfrac{1}{2} c \dot{x}^2, \; Q = F(t) \tag{2.8}$$

Compute each piece of (2.7):

- ∂L/∂ẋ = ∂T/∂ẋ = m ẋ, so d/dt of it is **m ẍ**.

- $\partial L/\partial x = -\partial V/\partial x = \mathbf{-k\,x}$.
- $\partial D/\partial \dot{x} = \mathbf{c\,\dot{x}}$.

Put them in (2.7):

$$\boldsymbol{m\,\ddot{x} + c\,\dot{x} + k\,x = F(t)} \tag{2.9}$$

This is the most famous equation in vibration theory. We obtained it **without drawing a force diagram**. The same three-step recipe (write T, V, D; differentiate; assemble) will give us the equations for a whole network of parts in Chapter 3.

2.8 Rails: constraints made easy

A **rail** forces a mass to move along a line only. A *horizontal* rail (X rail) allows only x; a *vertical* rail (Y rail) allows only y. In the Lagrangian method, a rail simply **deletes** one coordinate: the blocked coordinate is no longer a variable, so the rail's pushing force never appears in the equations.

In the code, we keep both x and y for every node, and we **zero the acceleration and the starting speed in the blocked direction**. Chapter 6 shows the two lines that do it, and test 4 in Chapter 14 proves the blocked direction never drifts.

> **WHAT IF IT WERE DIFFERENT?**
> **What if we used Newton's way for rails?** We would have to compute, at every instant, how hard the rail pushes sideways so that the sideways acceleration is exactly zero. With Lagrange, that push never has to be computed at all, because the rail is built into the coordinates.

2.9 Why Lagrange suits a program that users extend by clicking

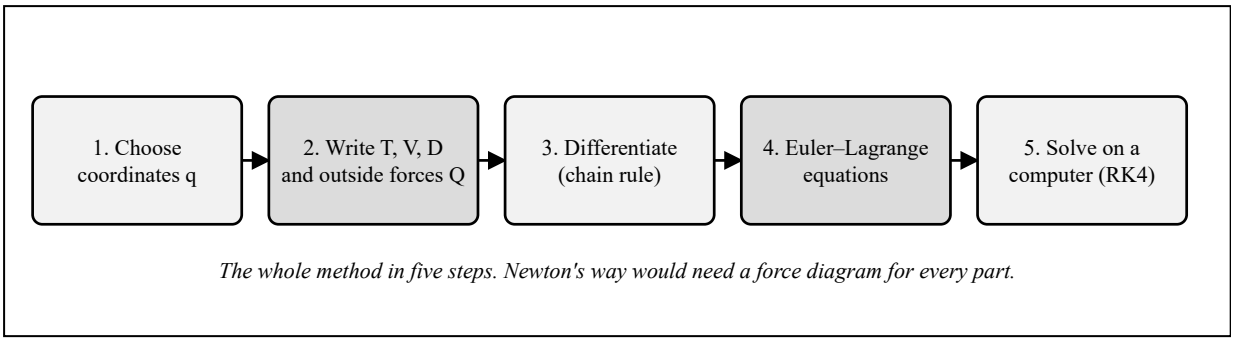


Figure 2.3. *The Lagrangian recipe. Steps 1 to 4 are algebra, done once on paper (Chapter 3). Step 5 is done by the computer, thousands of times per second.*

Adding a new kind of part (say, a *friction* element) means adding **one new term** to T, V, D or Q, then differentiating that one term. Nothing else changes. That is why the engine in Chapter 6 is one short function (`accel`) with one loop over links and one loop over masses.

2.10 Chapter checks

QUIZ

1. What is the Lagrangian L?
2. What does it mean for an action to be "stationary"?
3. Why must the nudge η be zero at both ends?
4. For T = ½ m ẋ², what is ∂T/∂ẋ? And its time derivative?
5. In the Euler–Lagrange equation with dissipation, which term gives the damper's force?
6. How many degrees of freedom do three masses have if one is on an X rail and two are free in the plane?
7. Why does the rail's pushing force never appear in our equations?

ANSWERS

1. L = T − V, kinetic minus potential energy.
2. A tiny nudge of the path does not change the action to first order (like the flat bottom of a bowl).
3. The nudged path must start and end at the same two points as the real path; also it makes the boundary term in the by-parts step vanish.
4. ∂T/∂ẋ = m ẋ; its time derivative is m ẍ.
5. ∂D/∂q̇ (it equals c ṡ for a damper).
6. 1 + 2 + 2 = 5.
7. Because the rail removes a coordinate, so the force that holds the mass on the rail does no work and never enters the equations.

CHAPTER 3

Deriving Our Equations

In Chapter 2 we met the method. Now we use it on the real model: masses and ground anchors in a plane, joined by springs and dampers, with gravity, a harmonic push, and a mouse force. We will derive every formula that appears in `engine.hpp`.

3.1 Step 1: the coordinates

Every node *i* (a mass or a ground anchor) has a position (x_i, y_i) and a velocity $(v_{x,i}, v_{y,i})$. The program stores them in two flat arrays:

$$q = (x_0, y_0, x_1, y_1, \ldots)\ v = (v_{x0}, v_{y0}, v_{x1}, v_{y1}, \ldots) \tag{3.1}$$

So node *i* lives at `q[2i]` *and* `q[2`i+1`]`. A ground anchor is just a node that never moves.

3.2 Step 2: the geometry of one link

A spring or damper joins two nodes, *a* and *b*. Three quantities describe it:

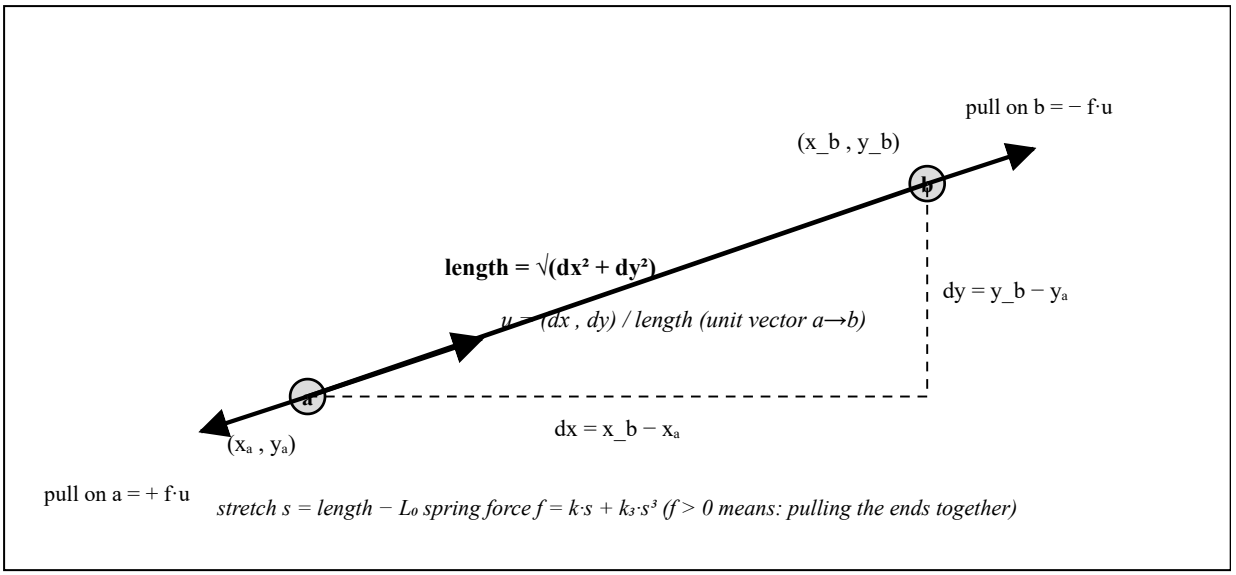


Figure 3.1. *The geometry of one link. Everything the engine needs comes from the vector d from a to b.*

$$d = r_b - r_a,\ \ell = |d| = \sqrt{(d_x^2 + d_y^2)},\ u = d / \ell \tag{3.2}$$

- **d** is the arrow from *a* to *b*.

- **ℓ** (the length) is how long that arrow is.
- **u** is the arrow shrunk to length 1: the **unit vector** that points along the link.

We need one more fact: how the length changes when a node moves. Differentiate $\ell = \sqrt{(d_x^2 + d_y^2)}$ with respect to the x-position of *b*:

$$\partial \ell / \partial x_b = d_x / \ell = u_x, \text{ and similarly } \partial \ell / \partial y_b = u_y \tag{3.3}$$

For node *a*, d has a minus sign, so $\partial \ell / \partial r_a = -\mathbf{u}$. In words: **stretching the link by moving b along u makes it longer; moving a along u makes it shorter.**

3.3 Step 3: the spring's potential energy and its force

Let L_0 be the spring's rest length. The **stretch** is $s = \ell - L_0$. A spring stores

$$V_{spring} = \tfrac{1}{2} k s^2 + \tfrac{1}{4} k_3 s^4 \tag{3.4}$$

The first term is the ideal (linear) spring. The second term, with the **cubic stiffness** k_3, makes the spring harder to stretch the further it goes: a "hardening" spring. When $k_3 = 0$ it vanishes.

We need the *gradient* of V with respect to the coordinates. The chain rule links V to the coordinates through s:

$$\partial V / \partial r_b = (dV/ds) \cdot (\partial s / \partial r_b) = f \cdot u, \; \partial V / \partial r_a = -f \cdot u, \text{ where } f = k s + k_3 s^3 \tag{3.5}$$

The force on a node is **minus** the gradient of the potential (a ball rolls *down* the hill). So:

$$\text{force of spring on } b = -f u, \text{ force of spring on } a = +f u \tag{3.6}$$

Check the signs with a feeling: if the spring is stretched, $s > 0$ and $f > 0$. The force on *b* is $-f\mathbf{u}$, which points back toward *a*. The force on *a* is $+f\mathbf{u}$, pointing toward *b*. A stretched spring pulls its two ends together, exactly as it should.

> **MATHS TO CODE**
> In the code, **f** is the field `force` that `linkInfo()` computes for a spring, and the two signs in (3.6) are the lines `F[2L.a] += in.force * in.ux` *and* `F[2`L.b] -= in.force * in.ux` inside `accel()`.

3.4 Step 4: the damper's dissipation and its force

How fast does the link's length change? Differentiate ℓ with respect to time, using (3.3) and the chain rule:

$$\dot{s} = d\ell/dt = \partial\ell/\partial r_b \cdot v_b + \partial\ell/\partial r_a \cdot v_a = u \cdot (v_b - v_a) \tag{3.7}$$

So ṡ is the part of the *relative velocity* that lies along the link. A velocity sideways to the link does not change its length and a damper does not resist it.

A damper with coefficient c has

$$D_{damper} = \tfrac{1}{2}\, c\, \dot{s}^2,\ \partial D/\partial v_b = c\, \dot{s}\, u,\ \partial D/\partial v_a = -\, c\, \dot{s}\, u \tag{3.8}$$

and the damper force on *b* is −(c ṡ) **u** and on *a* is +(c ṡ) **u**. The very same signs as the spring. That is why, in the code, a spring and a damper differ only in how they compute the number called `force`; the code that *applies* it is shared.

> **WHY?**
> **Why this sharing matters.** The Lagrangian method told us that a damper's equations have the same shape as a spring's. We wrote the "apply a force along the link" code **once** and used it for both. If we had derived each by force diagrams, we might never have noticed.

3.5 Step 5: gravity

Gravity pulls toward +y (screen coordinates have y pointing down). A mass at height y has potential energy that **falls** as y grows, measured from its starting y_0:

$$V_g = -\, m\, g\, (y - y_0) \tag{3.9}$$

Differentiating, $\partial V_g/\partial y = -m\, g$, so the force is +m g, pointing down. We subtract y_0 so that the potential energy starts at zero and the readout is easy to read; subtracting a constant changes no forces.

3.6 Step 6: the outside forces

Two kinds of outside force are included:

- **The drive:** a harmonic push on any mass, $F(t) = F_0 \cos(\omega t)$ in a direction at angle θ: components (F cos θ, F sin θ). With θ = 0 it pushes right; with θ = 90° it pushes down (y is down).
- **The mouse spring:** when you drag a mass, the program adds a force that pulls it toward the cursor, F = m (700 (target − q) − 45 v). It is a critically-damped spring tied to the cursor. The gains 700 and 45 were chosen so the mass follows quickly without ringing. Multiplying by m makes a heavy mass and a light mass feel equally "grabbable".

Both are **generalized forces Q**. Together with the springs and dampers, they enter the right-hand side of (2.7).

3.7 Step 7: assemble the equation of motion

Because T = Σ ½ m |v|² depends only on the velocities, and V and D depend only on positions and velocities respectively, equation (2.7) simplifies for every mass *i*:

$$\boldsymbol{m_i\, a_i = -\,\partial V/\partial r_i - \partial D/\partial v_i + Q_i} \tag{3.10}$$

Writing out what each term is for our model, summing over all the links that touch mass *i*:

$$m_i\, a_i = \Sigma_{links}\,(\,\pm\,(f_{spring} + f_{damper})\,u\,) + (\,0\,,\, m_i\, g\,) + F_{drive}(t) + F_{mouse} \tag{3.11}$$

The sign is "+" if mass *i* is the *a* end of the link and "−" if it is the *b* end. This is the equation `accel()` solves, once per mass, every one-thousandth of a second.

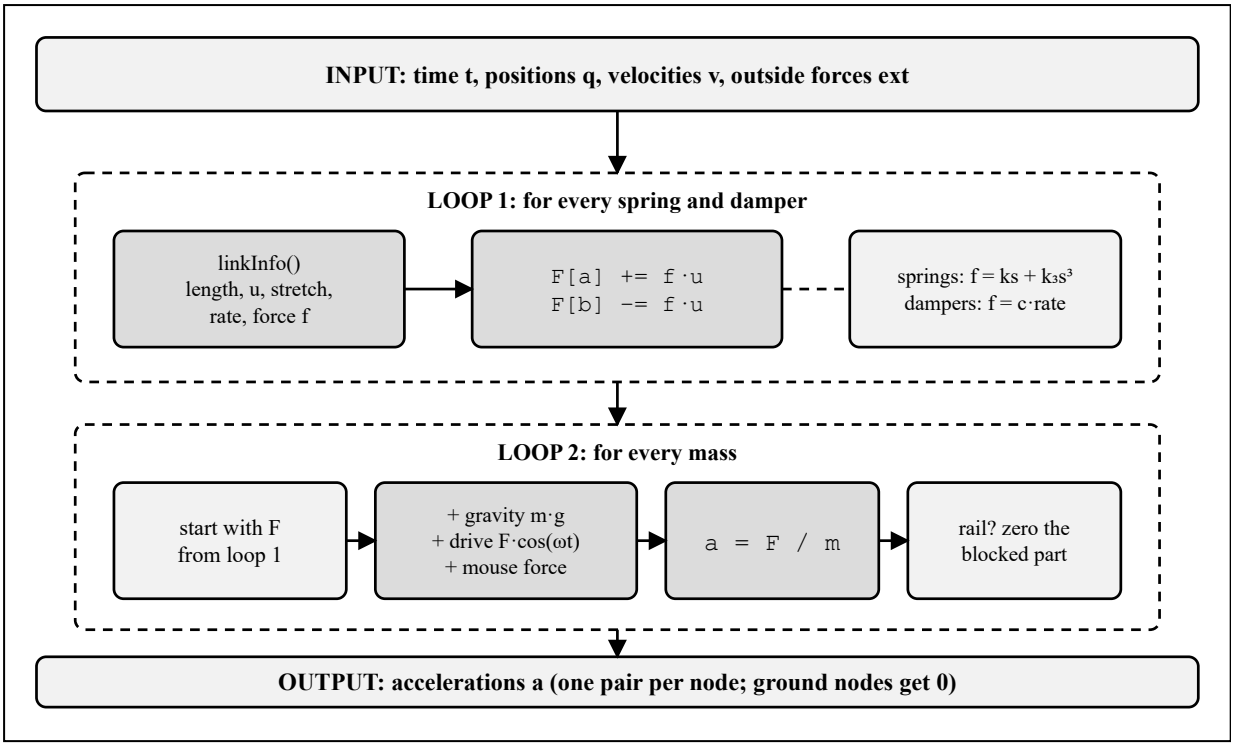


Figure 3.2. *The data flow inside accel(): loop 1 gathers link forces onto the nodes, loop 2 turns forces into accelerations.*

Finally, a mass on a rail has its blocked acceleration component set to zero (Chapter 2).

3.8 Step 8: the law of energy, derived

The program checks itself using a law of energy. Here is the proof.

Let E = T + V. Differentiate it with respect to time. The kinetic part gives Σ m **v** · **a**; the potential part gives Σ (∂V/∂**r**) · **v**. Substitute **m a** from (3.10):

$$dE/dt = \Sigma\, v \cdot (-\partial V/\partial r - \partial D/\partial v + Q) + \Sigma\, (\partial V/\partial r) \cdot v = -\Sigma\, v \cdot \partial D/\partial v + \Sigma\, Q \cdot v \quad (3.12)$$

The potential terms cancel. Now use a fact about functions like D that are **quadratic in the velocities**: *Euler's theorem on homogeneous functions* says Σ v · ∂D/∂v = 2 D. Therefore:

$$\boldsymbol{dE/dt = -\,2\,D + P_{in}},\ \textit{where}\ P_{in} = \Sigma\, Q \cdot v \quad (3.13)$$

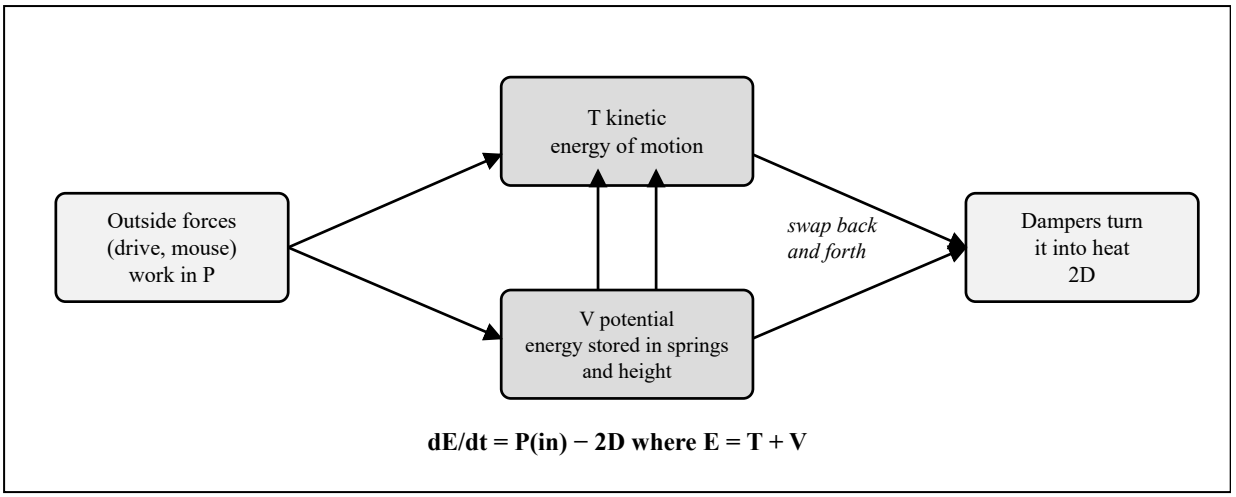


Figure 3.3. *Energy flows in from drives and the mouse, swaps between kinetic and potential, and leaves as heat through the dampers. Equation (3.13) says exactly this.*

In words: **the total energy changes only because dampers burn it (−2D) and outside forces feed it (P_{in}).** Integrating (3.13) over time gives the law the `Meter` checks:

$$E(t) = E(0) + \int P_{in}\, dt - \int 2D\, dt \quad (3.14)$$

WHY?

Why this is a powerful test. (3.14) is *not* something the code was written to satisfy. It is a consequence of the physics. If the code has a wrong sign, a missing factor, or a bug in a force, the energy will not balance and the *energy-law error* on screen will grow. The error is a free, built-in bug detector.

3.9 Step 9: the exact answer for one mass, to test against

For one mass with a linear spring and a damper and no push, equation (2.9) is m ẍ + c ẋ + k x = 0. Try a solution of the form $x = e^{rt}$. Substituting gives m r² + c r + k = 0, whose roots are

$$r = -\zeta\, \omega_n \pm i\, \omega_d \quad (3.15)$$

with the three famous numbers:

$$\omega_n = \sqrt{(\,k / m\,)},\ \zeta = c / (\,2\,\sqrt{(\,k\,m\,)}\,),\ \omega_d = \omega_n\,\sqrt{(\,1 - \zeta^2\,)} \tag{3.16}$$

KEY WORD

Natural frequency (ω_n). How fast the system would oscillate with no damping, in radians per second. Divide by 2π for hertz (cycles per second).

Damping ratio (ζ). How strong the damping is, compared with the amount that exactly prevents any oscillation. ζ < 1 rings; ζ = 1 is *critical*; ζ > 1 is *overdamped*.

Damped frequency (ω_d). The actual ringing speed when ζ < 1. It is a little lower than ω_n.

For ζ < 1, with starting position x_0 and starting speed v_0, the exact answer is

$$x(t) = e^{-\zeta\,\omega_n\,t}\,(\,x_0\,\cos\,\omega_d\,t + (\,v_0 + \zeta\,\omega_n\,x_0\,)\,/\,\omega_d\,\cdot\,\sin\,\omega_d\,t\,) \tag{3.17}$$

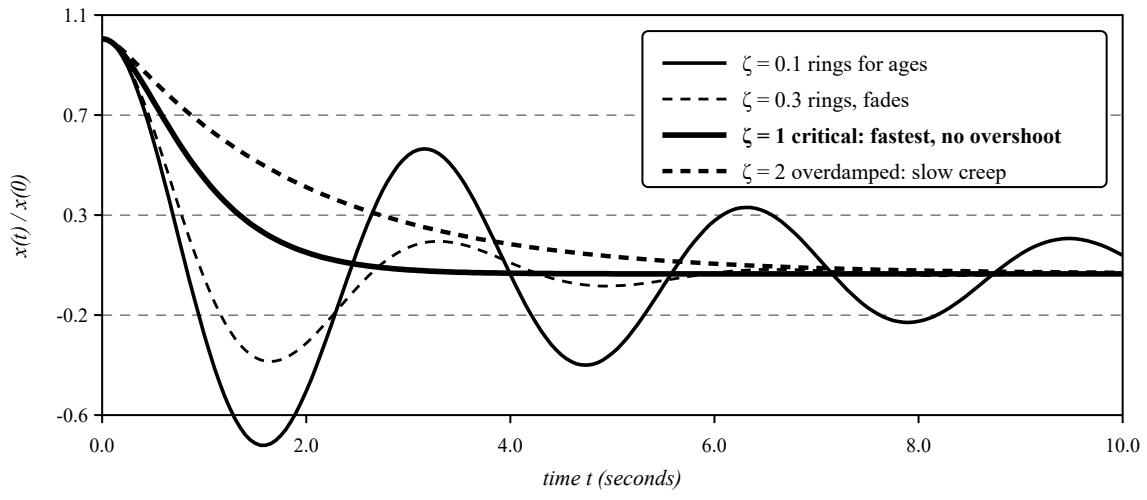


Figure 3.4. *The same system with four damping ratios. Critical damping (thick line) returns to rest fastest without overshoot; very light damping rings for a long time; heavy damping creeps back slowly.*

3.10 Step 10: a worked numerical example

Example 1 of the program is: m = 2 kg, k = 30 N/m, c = 1.5 N·s/m, with the spring stretched by 1 m and released from rest. Then:

- $\omega_n = \sqrt{(30 / 2)} = \sqrt{15} =$ **3.873 rad/s**, so a frequency of 3.873 / 2π = 0.616 Hz.
- ζ = 1.5 / (2 √(30 × 2)) = 1.5 / 15.49 = **0.0968**. Very lightly damped: it will ring many times.
- $\omega_d = 3.873 \times \sqrt{(1 - 0.0968^2)} =$ **3.855 rad/s**, so the real ringing frequency is 0.6136 Hz and the period is 1.63 s.

- The envelope of the ringing falls as $e^{-\zeta\,\omega_n\,t} = e^{-0.375\,t}$. After 10 seconds it has dropped to $e^{-3.75} = 2.3$ % of its start.

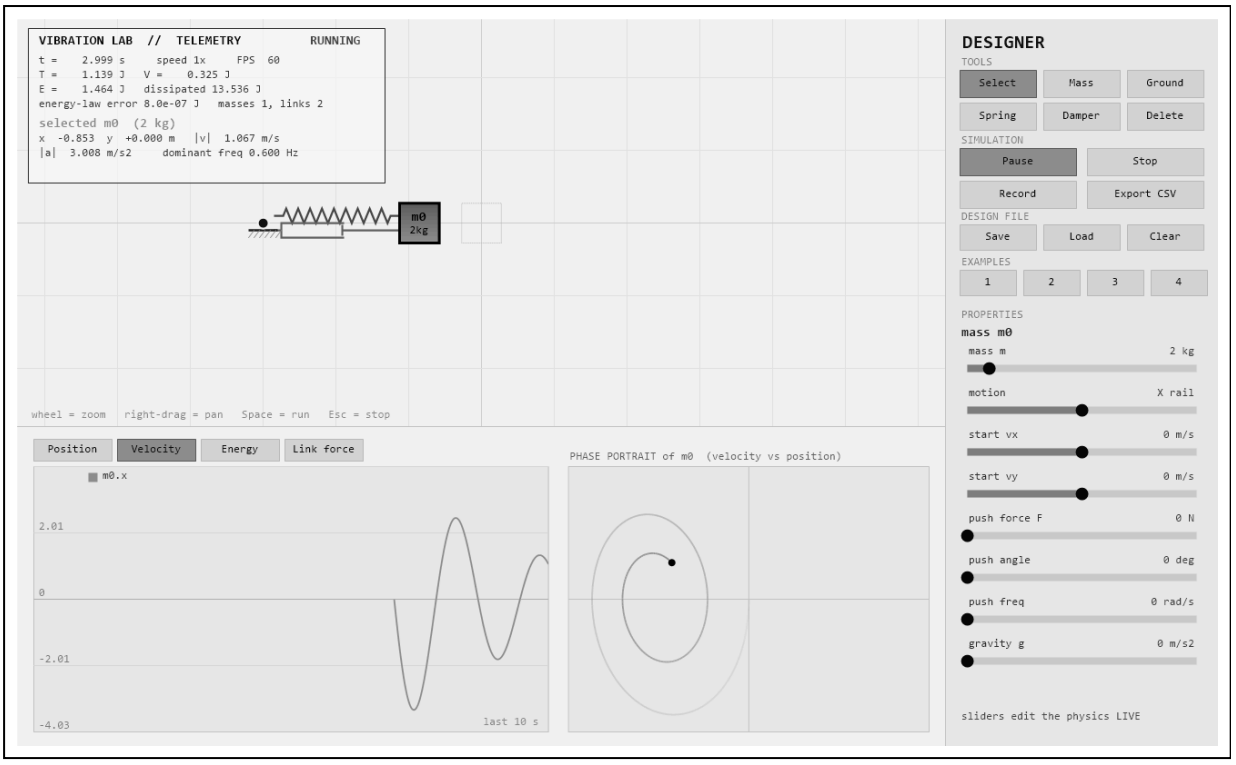


Figure 3.5. *Example 1 running. This is the system of the worked example: the Velocity tab shows a decaying oscillation, and the dominant frequency in the telemetry box matches the damped frequency we calculated.*

When you run example 1 in the program and select the mass, the telemetry reports a *dominant frequency* near 0.61 Hz, which agrees with ω_d.

> **TRY IT**
> Load example 1 (the **1** button). Select the mass, press Run, and look at the "dominant freq" line. Then select the spring and raise its stiffness k from 30 to 120. How should the frequency change? (Hint: $\omega_n \propto \sqrt{k}$, so four times the stiffness gives about twice the frequency.)

3.11 Step 11: another physical effect, the static sag

In example 4 (the quarter-car), gravity pulls the masses down and the springs hold them up. The system settles where the spring force equals the weight. For a mass m hanging on one spring k, the sag is

$$s_{static} = m\,g\,/\,k \tag{3.18}$$

You can check this against the program: heavier cars sag further, and the number in the telemetry box when the plot flattens matches (3.18) for a single spring.

3.12 Chapter checks

QUIZ

1. In (3.2), what is the unit vector u, and what is its length?
2. A spring is compressed ($s < 0$). Which way is the force on node b? Show it from (3.6).
3. Why does a damper ignore sideways motion?
4. Why do we subtract y_0 in the gravity energy?
5. What does (3.13) tell you if a system has no dampers and no push?
6. Compute ω_n and ζ for $m = 1$, $k = 100$, $c = 4$.
7. By what factor does the sag $m\,g\,/\,k$ change if the spring is made twice as stiff?

ANSWERS

1. $u = d\,/\,\ell$, the arrow from a to b shrunk to length 1. Its length is exactly 1.
2. $s < 0$ gives $f < 0$, so the force on b is $-f\,u = +|f|\,u$: it points *away* from a. A compressed spring pushes its ends apart.
3. Because $\dot{s} = u \cdot (v_b - v_a)$ only counts the part of the velocity along the link; sideways motion does not change the length.
4. So that V starts at zero and is easy to read; subtracting a constant does not change any force.
5. $dE/dt = 0$: energy is conserved. (The program tests this with the spring pendulum, Chapter 14.)
6. $\omega_n = \sqrt{(100/1)} = 10$ rad/s; $\zeta = 4\,/\,(2\,\sqrt{100}) = 0.2$.
7. It halves.

CHAPTER 4

How a Computer Solves It: RK4

4.1 The problem with exact answers

Formula (3.17) works for **one** mass with a **linear** spring. As soon as you add a second mass, a cubic spring, gravity with a swinging mass, or your mouse, nobody can write the answer as a formula. The computer must find it **step by step**, taking tiny jumps in time. This is called **numerical integration**.

4.2 Turning second-order into first-order

Equation (3.10) has accelerations (second derivatives). Numerical methods like first derivatives, so we write the position and the velocity as two unknowns that each have a first derivative:

$$dq/dt = v,\ dv/dt = a(\ t,\ q,\ v\ ) \tag{4.1}$$

Here `a( t, q, v )` is exactly what the function `accel()` computes. The pair (q, v) is called the **state**, which is why the code has a struct named `State`.

4.3 The simplest method, and why it fails

Euler's method says: look at the slope now, and walk a step h in that direction.

$$q_{new} = q + h\,v,\ v_{new} = v + h\,a \tag{4.2}$$

For a frictionless oscillator, each Euler step multiplies the "size" of the motion by $\sqrt{(1 + h^2\omega^2)}$, which is always **more than 1**. The oscillation grows with every step: energy appears from nowhere.

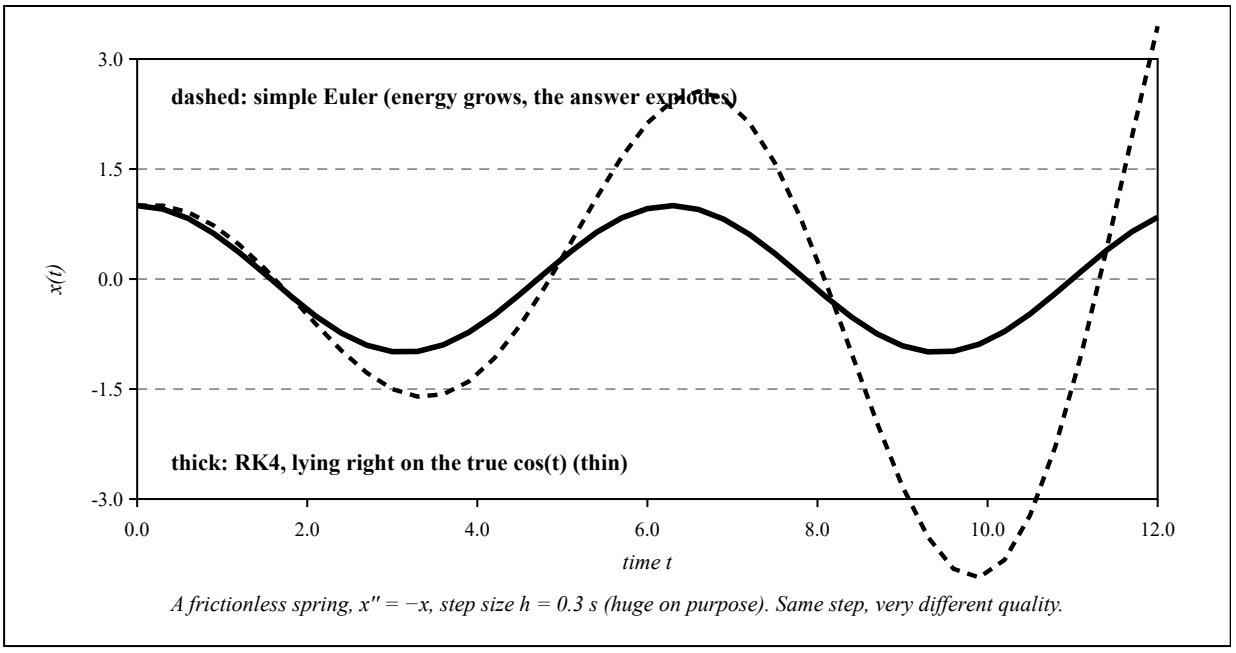


Figure 4.1. *A frictionless spring. Simple Euler's answer (dashed) swells until it explodes; RK4 (thick) lies on top of the true answer, with the same step size.*

WHAT IF IT WERE DIFFERENT?

What if the engine used Euler's method? A frictionless spring would gain energy and explode. A damped system would look fine for small steps and wrong for large ones. Because users can set any stiffness, we cannot allow a method that blows up so easily.

4.4 RK4: four peeks, one weighted guess

The **fourth-order Runge–Kutta method** (RK4) looks at the slope four times inside one step, then combines them with weights 1, 2, 2, 1:

- $\mathbf{k_1}$: the slope at the start of the step.
- $\mathbf{k_2}$: the slope in the middle, using k_1 to guess where the middle is.
- $\mathbf{k_3}$: the slope in the middle again, using k_2 for a better guess.
- $\mathbf{k_4}$: the slope at the end, using k_3.

$$y_{new} = y + h \cdot (k_1 + 2\,k_2 + 2\,k_3 + k_4) / 6 \tag{4.3}$$

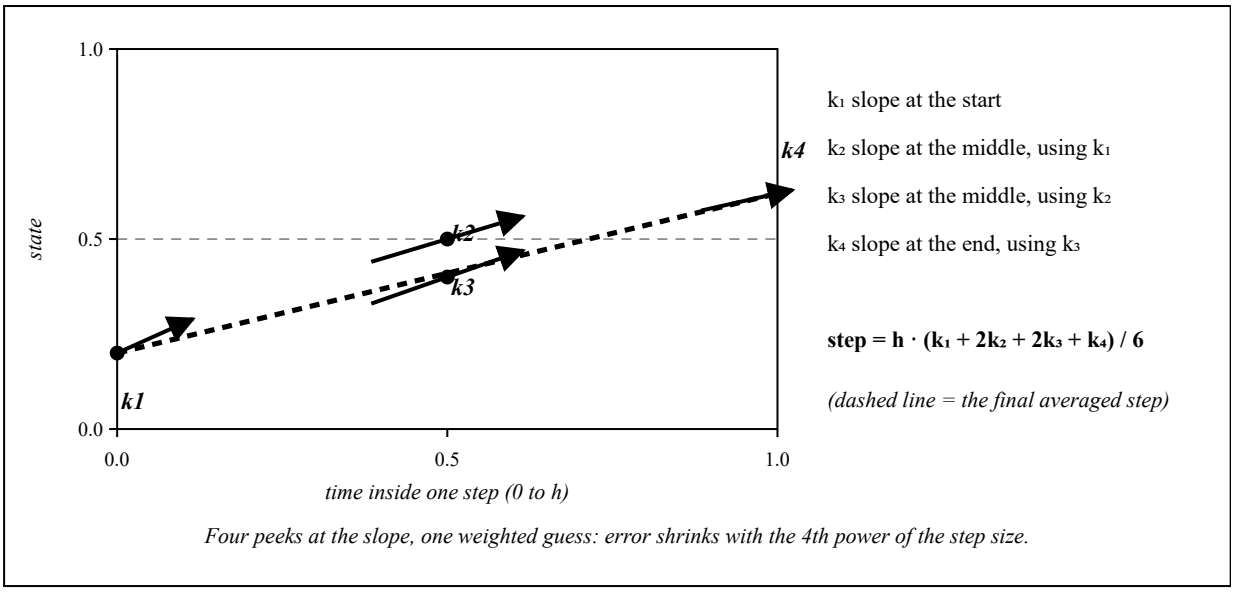


Figure 4.2. *The four slopes inside one step, and the weighted average that RK4 actually uses (dashed).*

The weights 1, 2, 2, 1 come from matching the Taylor series of the true solution up to the h^4 term. The result: the error in one step shrinks like h^5, and the error accumulated over a whole run shrinks like $\mathbf{h^4}$. Halve the step and the error drops sixteen times.

In our engine, `rk4()` applies this to both q and v at once. The four "slopes" are the velocities and accelerations at the four sample points; Chapter 7 goes through it line by line.

4.5 How small should a step be? Stability

RK4 stays stable as long as $h \times \omega_{max}$ is below about 2.8, where ω_{max} is the *fastest* oscillation in the system. Our step is **h = 1 millisecond**. The stiffest, lightest thing the sliders allow is k = 500 N/m with m = 0.1 kg, so $\omega_{max} = \sqrt{(500 / 0.1)} \approx 71$ rad/s, and h ω = 0.071. That is forty times below the limit, so the program is very safe, yet it is cheap: 1000 steps per second is trivial for a modern computer.

> **WHY?**
> **Why a *fixed* step of exactly 1 ms?** A fixed step gives repeatable results: the same design always produces the same motion, which matters for recording and for tests. A step that changed with the frame rate would make the simulation behave differently on a slow computer.

4.6 The accumulator: real time versus physics time

The screen is redrawn roughly 60 times per second, but the gaps between redraws are uneven. The physics must still advance in steps of exactly 1 ms. The solution is a number called the **accumulator** (`simAcc` in the code).

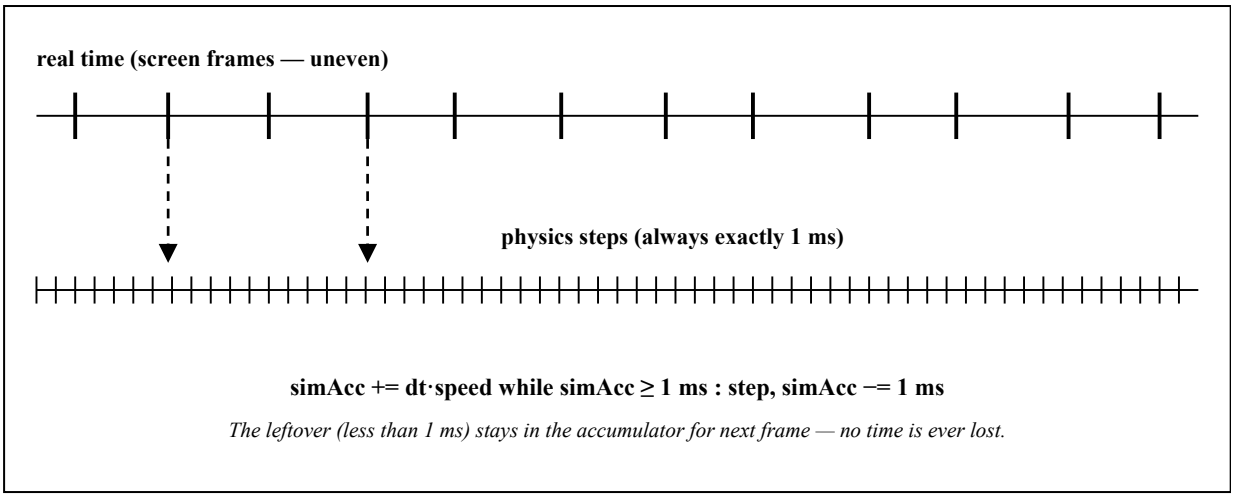


Figure 4.3. *Each frame we add the real elapsed time (times the speed knob) to the accumulator, then take as many 1 ms physics steps as fit. The leftover waits for the next frame.*

If a frame took 17.3 ms, we take 17 steps and keep 0.3 ms in the accumulator. Nothing is lost, and nothing is invented. A safety limit of 800 steps per frame stops a "spiral of death" if the computer stalls: a long pause would otherwise demand thousands of steps, which would make the next frame even later.

4.7 Keeping the energy books: the trapezoid rule

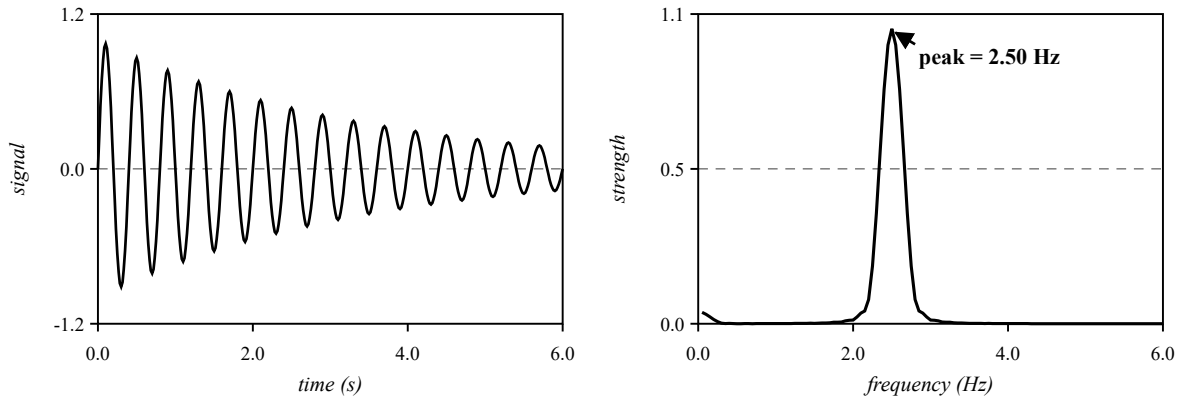
The `Meter` integrates the two power terms of (3.14). It uses the **trapezoid rule**: for each step it averages the power at the start and at the end of the step, then multiplies by h. That is accurate to the second order, and it costs only two extra evaluations per step.

4.8 Finding the dominant frequency

The telemetry shows a *dominant frequency* of the selected mass. To find it, the program tries many test frequencies f and, for each, measures how well a sine wave of that frequency matches the recorded motion. This is a **discrete Fourier transform**:

$$X(f) = \Sigma_n \, (x_n - \bar{x}) \cdot w_n \cdot e^{- i \, 2\pi f \, t_n} \tag{4.4}$$

The test frequency that gives the largest |X| wins.



The frequency finder tries many frequencies and keeps the one that matches the wiggle best. True answer: 2.5 Hz.

Figure 4.4. *A decaying 2.5 Hz wiggle (left) and its spectrum (right). The tallest peak sits at 2.5 Hz.*

Two details improve it. First, we subtract the average $\bar{x}$ so a constant offset (like sag) does not look like a "0 Hz tone". Second, we multiply by a **Hann window** $w_n = ½ − ½ \cos(2\pi\ n\ /\ (N−1))$, which fades the signal in and out so that cutting the recording at arbitrary points does not smear the peak.

4.9 Checking the solver

We trust the numerical method only after testing it against two things nobody can argue with:

1. **The exact answer** (3.17), for one mass. The result: the largest gap over 10 seconds is **6.9×10^{-12} metres**. That is a small fraction of the width of a single atom, far below anything visible.
2. **The law of energy** (3.14). For the driven two-mass system with a wiggling mouse force, the worst error over 20 seconds was **1.0×10^{-5} joules**.

Chapter 14 shows how these are automated.

4.10 Chapter checks

QUIZ

1. Why do numerical methods turn the acceleration equation into two first-order equations?
2. In Euler's method, does the energy of a frictionless oscillator go up, down, or stay the same?
3. What are the four weights in RK4?
4. If you halve the step size, how much smaller is RK4's total error?
5. Why do we keep the leftover time in the accumulator?

6. Why is the average $\bar{x}$ subtracted before the Fourier test?

ANSWERS

1. Because the standard algorithms work on first derivatives: $dq/dt = v$ and $dv/dt = a$.
2. It goes up (it blows up).
3. 1, 2, 2, 1 (divided by 6).
4. Sixteen times (error $\propto h^4$).
5. So no time is lost or invented, however uneven the frame gaps are.
6. So that a steady offset is not mistaken for a very slow wiggle.

Part II — The Engine, Line by Line

CHAPTER 5

The Model: engine.hpp, Part 1

This chapter reads the first third of `engine.hpp`: the data structures that describe *what the user drew*. The physics comes in Chapter 6. Every listing below is real source; read the listing first, then the explanation under it.

5.1 The file header and the namespace

ENGINE.HPP (LINES 1 TO 24)

```
   1  // engine.hpp -- a Lagrangian physics engine for planar mass / spring / damper
networks.
   2  //
   3  // WHAT IT SIMULATES
   4  //   Point masses and fixed ground anchors scattered on a 2-D plane, joined by springs
and
   5  //   dampers.  y points DOWN (like the screen).  A mass can move freely, or be forced
onto a
   6  //   horizontal rail (X only) or a vertical rail (Y only).
   7  //
   8  // HOW (the Lagrangian recipe, derived fully in the book)
   9  //   generalized coordinates   q = (x0, y0, x1, y1, ...)           positions of every
node
  10  //   kinetic energy            T = sum  1/2 m_i |v_i|^2
  11  //   potential energy          V = sum  [ 1/2 k s^2 + 1/4 k3 s^4 ]   (springs, s =
length - rest length)
  12  //                                 + sum  ( - m g (y - y_design) )   (gravity, y is
down)
  13  //   Rayleigh dissipation      D = sum  1/2 c (sdot)^2               (dampers, sdot =
rate of length change)
  14  //   outside forces            Q_i = drive force + mouse force
  15  //   Euler-Lagrange equations  d/dt( dL/dqdot ) - dL/dq + dD/dqdot = Q       with L = T
- V
  16  //   which, because T depends only on velocities and V only on positions, collapse to
  17  //                             m_i a_i = - dV/dq_i - dD/dqdot_i + Q_i
  18  //   accel() evaluates exactly that, step() integrates it with 4th-order Runge-Kutta.
  19  #pragma once
  20  #include <algorithm>
  21  #include <cmath>
  22  #include <vector>
  23
  24  namespace lag {
```

- **The first twenty or so lines** are a comment. In C++ everything after `//` on a line is ignored by the compiler and written for humans. This one summarises the physics so that anyone opening the file sees the plan at once: the model, the energies, and the Euler–Lagrange equation.
- `#pragma once` tells the compiler: "if another file includes me twice, only read me the first time." Without it, `live.cpp` and `model_io.hpp` would both pull in `engine.hpp`, and the compiler would complain that `struct Node` was defined twice.

- `#include <algorithm>` gives `std::fmax`-style helpers and `std::remove_if`; `<cmath>` gives `std::sqrt`, `std::hypot`, `std::cos`; `<vector>` gives `std::vector`, a resizable array.
- `namespace lag { ... }` puts every name inside a labelled box called `lag` (for *Lagrange*). Outside the box we write `lag::Model`. This stops our names (`Node`, `State`, `step`) from clashing with names in Windows' enormous header files.

> **WHY?**
> **Why a namespace?** `windows.h` defines thousands of names, including macros with very common words such as `min`, `max` and `small`. Wrapping our code in `namespace lag` avoids silent collisions.

5.2 Constants and enumerations

ENGINE.HPP (LINES 26 TO 31)

```
26  constexpr int    MAX_NODES = 24;
27  constexpr double PI        = 3.14159265358979323846;
28
29  enum class NodeKind { Mass, Ground };
30  enum class LinkKind { Spring, Damper };
31  enum Rail { FREE = 0, RAIL_X = 1, RAIL_Y = 2 };
```

- `constexpr int MAX_NODES = 24;` A *constant known at compile time*. We refuse to place more than 24 nodes. Why 24? The telemetry log stores several numbers per node for every frame, so a limit keeps memory use predictable. The limit is also enforced in `addMass` and `addGround`.
- `constexpr double PI = 3.14159265358979323846;` The ratio of a circle's circumference to its diameter, with 21 digits (more than a `double` can hold, which is fine). We define it ourselves because `M_PI` is not part of the C++ standard and is missing on some compilers.
- `enum class NodeKind { Mass, Ground };` A type that can only be one of two names. `enum class` (rather than plain `enum`) keeps the names inside the type, so we must write `NodeKind::Mass`. That prevents accidents like comparing a `NodeKind` to a `LinkKind`.
- `enum class LinkKind { Spring, Damper };` The same idea for links.
- `enum Rail { FREE = 0, RAIL_X = 1, RAIL_Y = 2 };` A plain `enum`, on purpose: its names are plain integers `0, 1, 2`. We want that, because the user edits the rail with a *slider*, and a slider produces a number. (See the field `rail` in `Node`.)

5.3 Node: one thing on the canvas

ENGINE.HPP (LINES 34 TO 43)

```
34  struct Node {
35      NodeKind kind = NodeKind::Mass;
36      double x = 0, y = 0;          // design position in metres
```

```
  37      double m = 1.0;               // mass in kg (ignored for ground)
  38      double vx0 = 0, vy0 = 0;      // starting velocity (m/s)
  39      double rail = FREE;           // 0 free, 1 X-rail, 2 Y-rail  (a double so UI
sliders can edit it)
  40      double driveF = 0;            // harmonic push amplitude (N)
  41      double driveAngle = 0;        // push direction in degrees: 0 = right, 90 = down
  42      double driveW = 0;            // push frequency (rad/s)
  43  };
```

A **struct** is a bundle of named fields. A `Node` is either a mass or a ground anchor.

- **`NodeKind kind = NodeKind::Mass;`** Which of the two it is. The `= ...` is a *default value*, used when we write `Node n;`.
- **`double x = 0, y = 0;`** The **design position**: where you placed it with the mouse, in metres. Note the comment: *y points down*, like the screen. A larger y is lower on screen. Choosing the screen's convention avoids a sign flip every time the program converts between maths and pixels.
- **`double m = 1.0;`** The mass in kilograms. Ignored for ground anchors (an anchor is conceptually infinitely heavy).
- **`double vx0 = 0, vy0 = 0;`** The **starting velocity**. When you press Run, the masses begin with these speeds. This is how you "hit" a mass at time zero.
- **`double rail = FREE;`** The rail setting, as a `double`. It is a `double` and not an `enum` so the *same* slider code that edits masses and stiffnesses can edit it (with a "snap to whole numbers" option). It is converted back with `(int)std::lround(nd.rail)` wherever it is used.
- **`double driveF`, `driveAngle`, `driveW`.** A harmonic push `F cos(ωt)`: its amplitude (newtons), direction (degrees) and angular frequency (radians per second). Zero amplitude means no push.

WHAT IF IT WERE DIFFERENT?

What if `rail` were an enum class? We would need a separate, special-purpose control in the panel for choosing it, and a conversion to and from numbers for the save file. Storing it as a number made the slider code, the file code and the engine all simpler. The price is a cast, `(int)std::lround(...)`, in a few places.

5.4 Link: a spring or a damper

ENGINE.HPP (LINES 45 TO 53)

```
  45  struct Link {
  46      LinkKind kind = LinkKind::Spring;
  47      int a = -1, b = -1;           // indices of the two nodes it joins
  48      double k = 20.0;              // spring stiffness (N/m)
  49      double k3 = 0.0;              // cubic stiffness (N/m^3): 0 = ideal linear spring
  50      double L0 = 1.0;              // spring rest length (m)
  51      bool   autoRest = true;       // while true, L0 follows the drawn length
  52      double c = 0.8;               // damper coefficient (N s/m)
  53  };
```

- `LinkKind kind`: spring or damper. One struct serves both, because they share the same geometry; only the *force law* differs.
- `int a = -1, b = -1;` The **indices** of the two nodes in `Model::nodes`. The value −1 means "not set". Indices (rather than pointers) stay valid when a vector is copied, which matters for saving and for undo-style features.
- `double k`, `k3`, `L0`. Spring data: stiffness k, cubic stiffness k_3, and rest length L_0 (the length at which the spring pulls with zero force).
- `bool autoRest = true;` While true, L_0 follows the *drawn* length, so a spring you draw is born relaxed. Once you move the "rest length" slider by hand, this becomes false and L_0 stays where you put it. Chapter 10 shows where it is cleared.
- `double c`. Damper data: the damping coefficient. A damper ignores the spring fields, and a spring ignores `c`. The unused fields cost a few bytes and buy a very simple design.

5.5 Model, part 1: the container and two helper functions

ENGINE.HPP (LINES 55 TO 58)

```
55  struct Model {
56      std::vector<Node> nodes;
57      std::vector<Link> links;
58      double gravity = 0.0;         // m/s^2, pulls toward +y (down)
```

- `std::vector<Node> nodes;` A resizable list of every mass and anchor. A node's *position in this list* is its identity: node 3 is `nodes[3]`.
- `std::vector<Link> links;` The same for springs and dampers.
- `double gravity = 0.0;` One number for the whole world, in m/s². Zero means weightless. The sliders allow up to 20.

ENGINE.HPP (LINES 60 TO 64)

```
60      int massCount() const {
61          int n = 0;
62          for (const Node& nd : nodes) n += (nd.kind == NodeKind::Mass);
63          return n;
64      }
```

- `int massCount() const` Counts the nodes that are masses. The word `const` after the parentheses promises that this function does not change the model, so it can be called on a read-only model.
- `n += (nd.kind == NodeKind::Mass);` A comparison produces `true` or `false`, which C++ treats as 1 or 0, so adding it counts the matches. It is a compact way to count.
- `for (const Node& nd : nodes)` A *range-for*: "for every element in `nodes`". The `const Node&` means "look at it, do not copy it, do not change it".

ENGINE.HPP (LINES 65 TO 67)

```
65      double distance(int a, int b) const {
66          return std::hypot(nodes[b].x - nodes[a].x, nodes[b].y - nodes[a].y);
```

```
67      }
```

- `std::hypot(dx, dy)` computes $\sqrt{(dx^2 + dy^2)}$ without overflow or loss of accuracy. It is the length of the arrow between nodes a and b: equation (3.2).

5.6 Model, part 2: adding things

ENGINE.HPP (LINES 68 TO 73)

```
68      int addMass(double x, double y, double m = 1.0) {
69          if ((int)nodes.size() >= MAX_NODES) return -1;
70          Node n; n.kind = NodeKind::Mass; n.x = x; n.y = y; n.m = m;
71          nodes.push_back(n);
72          return (int)nodes.size() - 1;
73      }
```

- `if ((int)nodes.size() >= MAX_NODES) return -1;` The limit check. We return −1, which means *failure*; the caller (the click handler) shows a message. The cast `(int)` is needed because `size()` returns an unsigned type.
- `Node n; n.kind = ...; n.x = x; ...` Build a node on the stack, fill in the fields.
- `nodes.push_back(n);` Copy it onto the end of the list.
- `return (int)nodes.size() - 1;` Return its new index. The caller uses it to select the new node.

ENGINE.HPP (LINES 74 TO 79)

```
74      int addGround(double x, double y) {
75          if ((int)nodes.size() >= MAX_NODES) return -1;
76          Node n; n.kind = NodeKind::Ground; n.x = x; n.y = y;
77          nodes.push_back(n);
78          return (int)nodes.size() - 1;
79      }
```

- The same pattern as `addMass`, with `kind = Ground` and no mass value.

ENGINE.HPP (LINES 80 TO 90)

```
80      // Returns the new link's index, or -1 if it makes no sense (same node twice, or a
duplicate).
81      int addLink(LinkKind kind, int a, int b) {
82          if (a == b || a < 0 || b < 0 || a >= (int)nodes.size() || b >=
(int)nodes.size()) return -1;
83          if (nodes[a].kind == NodeKind::Ground && nodes[b].kind == NodeKind::Ground)
return -1;  // nothing can move
84          for (const Link& l : links)
85              if (l.kind == kind && ((l.a == a && l.b == b) || (l.a == b && l.b == a)))
return -1;
86          Link l; l.kind = kind; l.a = a; l.b = b;
87          l.L0 = std::fmax(0.05, distance(a, b));
88          links.push_back(l);
89          return (int)links.size() - 1;
90      }
```

This function is the "gatekeeper" for links. It refuses nonsense and returns −1:

- `if (a == b || a < 0 || b < 0 || a >= nodes.size() ...)` You cannot join a node to itself, and both indices must exist. Without this check, a stray click could create a link to

node −1 and the program would read memory that is not ours (a crash).

- `if (nodes[a].kind == Ground && nodes[b].kind == Ground) return -1;` A link between two fixed anchors can never do anything, because neither end can move, so we refuse it. (If we allowed it, `accel()` would still compute a force and then ignore it.)
- **The duplicate loop.** Two *springs* between the same pair would behave as one stiffer spring, which is confusing, so we refuse that. But a spring and a damper between the same pair is allowed: it is the classic **parallel spring-damper** (a car's shock absorber). The test `l.kind == kind && ((l.a == a && l.b == b) || (l.a == b && l.b == a))` says "same kind, same two nodes, in either order".
- `l.L0 = std::fmax(0.05, distance(a, b));` A new spring starts at the distance between its ends (so it is relaxed), but never less than 5 cm. A zero-length rest length would make the geometry unstable.

5.7 Model, part 3: removing things

ENGINE.HPP (LINES 91 TO 91)

```
  91      void removeLink(int i) { if (i >= 0 && i < (int)links.size())
links.erase(links.begin() + i); }
```

- The one-line `removeLink` checks the index is valid, then `links.erase(links.begin() + i)` deletes element *i* and shifts later ones down.

ENGINE.HPP (LINES 92 TO 98)

```
  92      void removeNode(int i) {
  93          if (i < 0 || i >= (int)nodes.size()) return;
  94          for (int j = (int)links.size() - 1; j >= 0; --j)             // links touching
it die with it
  95              if (links[j].a == i || links[j].b == i) links.erase(links.begin() + j);
  96          for (Link& l : links) { if (l.a > i) --l.a; if (l.b > i) --l.b; }   // later
nodes shift down by one
  97          nodes.erase(nodes.begin() + i);
  98      }
```

Removing a node is trickier than removing a link, because links refer to nodes **by index**. Deleting node *i* changes the index of every later node. The function therefore does three things, in this order:

1. **Delete every link that touches the node.** The loop runs from the last link down to the first: `for (int j = (int)links.size() - 1; j >= 0; --j)`. Going *backwards* matters, because erasing an element shifts the later ones down, which would make a forward loop skip an element.
2. **Renumber.** `for (Link& l : links) { if (l.a > i) --l.a; if (l.b > i) --l.b; }`. Any link that pointed at a node *after* the deleted one now points one place earlier. Links that touched the deleted node are already gone.
3. **Delete the node itself.**

WHAT IF IT WERE DIFFERENT?

What if we deleted the node first and renumbered later? The remaining links would briefly point at the wrong nodes, and, worse, links attached to the deleted node would point at whatever node slid into its place. Always remove the *references* before removing the *thing*.

5.8 Model, part 4: automatic rest length

ENGINE.HPP (LINES 99 TO 103)

```
  99      // Springs that have never been given a rest length by hand follow their drawn
length.
 100      void refreshAutoRest() {
 101          for (Link& l : links)
 102              if (l.kind == LinkKind::Spring && l.autoRest) l.L0 = std::fmax(0.05,
distance(l.a, l.b));
 103      }
```

- Loops over all links; for every **spring** whose `autoRest` is still true, it sets L_0 to the current drawn distance. The interface calls this after every drag, so a spring you stretch while designing is still relaxed. Chapter 10 shows when.

5.9 State: the numbers that change while it runs

ENGINE.HPP (LINES 106 TO 111)

```
 106  // ---------------------------------------------------------------------- the moving state
 107  // q and v are flat arrays:  q[2i] = x_i,  q[2i+1] = y_i   (same for v).
 108  struct State {
 109      std::vector<double> q, v;
 110      double t = 0;
 111  };
```

- The **design** (the `Model`) says what exists. The **state** says where everything is *right now*. We keep them separate on purpose: pressing Stop must put every mass back at its design position, and that is trivial if the design was never touched while running.
- `std::vector<double> q, v;` Two flat arrays with **two numbers per node**: `q[2i]` = x_i, `q[2i+1]` = y_i, and the same for v. This layout is exactly equation (3.1) of Chapter 3.
- `double t = 0;` The simulated time in seconds.

WHY?

Why flat arrays rather than a vector of little point structs? The integrator treats the whole state as one long list of numbers and does the same arithmetic to every entry (`q[i] += h * v[i]`). A flat array makes that a simple loop, and makes RK4 independent of how many nodes there are.

ENGINE.HPP (LINES 113 TO 129)

```
 113  inline State initialState(const Model& M) {
 114      State s;
 115      const int n = (int)M.nodes.size();
```

```
116      s.q.assign(2 * n, 0.0);
117      s.v.assign(2 * n, 0.0);
118      for (int i = 0; i < n; ++i) {
119          const Node& nd = M.nodes[i];
120          s.q[2 * i] = nd.x;
121          s.q[2 * i + 1] = nd.y;
122          if (nd.kind == NodeKind::Mass) {
123              int rail = (int)std::lround(nd.rail);
124              s.v[2 * i]     = (rail == RAIL_Y) ? 0.0 : nd.vx0;
125              s.v[2 * i + 1] = (rail == RAIL_X) ? 0.0 : nd.vy0;
126          }
127      }
128      return s;
129  }
```

- `const int n = (int)M.nodes.size();` The number of nodes.
- `s.q.assign(2 * n, 0.0); s.v.assign(2 * n, 0.0);` Make both arrays of length 2n, filled with zero. Ground anchors keep zero velocity forever.
- The loop copies each node's design position into `q`. Only for **masses** it also sets the velocity.
- `int rail = (int)std::lround(nd.rail);` Rounds the slider's number to the nearest whole number (0, 1 or 2).
- `s.v[2i] = (rail == RAIL_Y) ? 0.0 : nd.vx0;` *The* conditional operator*: "if a vertical rail, the horizontal speed must be zero, otherwise use the chosen vx0". Likewise for y. This is the starting-velocity half of the rail rule: a mass cannot begin with speed in a direction it is not allowed to move.

CHECKPOINT

In the program, load example 4 (quarter-car), select the wheel and look at the "start vx" slider: moving it changes nothing, because the wheel is on a Y rail. That is `initialState` at work.

5.10 Chapter checks

QUIZ

1. Why is `y` positive downward in this program?
2. Why does `removeNode` loop through the links *backwards*?
3. Why can a spring and a damper join the same two nodes, but two springs cannot?
4. What does `#pragma once` prevent?
5. Why is the rail stored as a `double`?
6. Where does the program keep the *design*, and where the *current positions*?

ANSWERS

1. It matches the screen, so converting between metres and pixels needs no sign flip.
2. Erasing an element shifts later ones down; going backwards means the elements we still have to visit are not moved.

3. A spring + damper in parallel is a standard, meaningful part (a shock absorber); two parallel springs are just one stiffer spring and would only confuse.
4. The same file being read twice (which would define the same struct twice).
5. So the same slider code can edit it; it is cast back with `std::lround`.
6. Design in `Model`; current positions in `State`.

CHAPTER 6

The Physics: engine.hpp, Part 2

This is the heart of the program. Every function in this chapter is one of the equations from Chapter 3, written in C++. Keep Chapter 3 open beside this one.

6.1 linkInfo: the geometry of a link

ENGINE.HPP (LINES 132 TO 137)

```
 132  struct LinkInfo {
 133      double len = 0, ux = 1, uy = 0;   // length and unit vector pointing a -> b
 134      double stretch = 0;               // len - L0              (springs)
 135      double rate = 0;                  // d(len)/dt             (dampers; also reported
for springs)
 136      double force = 0;                 // tension along the link: > 0 means it pulls its
ends together
 137  };
```

A `LinkInfo` is a **report card** for one link at one instant. Instead of recomputing the length and direction in five different places, we compute them once and return a bundle:

- `len`: the length ℓ. `ux, uy`: the unit vector $\mathbf{u}$. `stretch`: $s = \ell - L_0$. `rate`: $\dot{s}$. `force`: the tension along the link, positive when it pulls its ends together.

ENGINE.HPP (LINES 139 TO 154)

```
 139  inline LinkInfo linkInfo(const Model& M, const std::vector<double>& q, const
std::vector<double>& v, int li) {
 140      const Link& L = M.links[li];
 141      LinkInfo r;
 142      double dx = q[2 * L.b] - q[2 * L.a], dy = q[2 * L.b + 1] - q[2 * L.a + 1];
 143      r.len = std::hypot(dx, dy);
 144      if (r.len > 1e-9) { r.ux = dx / r.len; r.uy = dy / r.len; }          // guard: two
nodes on top of each other
 145      double dvx = v[2 * L.b] - v[2 * L.a], dvy = v[2 * L.b + 1] - v[2 * L.a + 1];
 146      r.rate = r.ux * dvx + r.uy * dvy;                                     // component
of relative velocity along the link
 147      if (L.kind == LinkKind::Spring) {
 148          r.stretch = r.len - L.L0;
 149          r.force = L.k * r.stretch + L.k3 * r.stretch * r.stretch * r.stretch;   //
dV/ds
 150      } else {
 151          r.force = L.c * r.rate;                                                //
dD/d(sdot)
 152      }
 153      return r;
 154  }
```

Go through it line by line, with equations (3.2) to (3.7) beside you:

- **`const Link& L = M.links[li];`** A shorthand name for "the li-th link", without copying it.

- `double dx = q[2L.b] - q[2L.a], dy = ...;` The vector $\mathbf{d} = \mathbf{r}_b - \mathbf{r}_a$ from (3.2). Node b's x is at `q[2*L.b]`.
- `r.len = std::hypot(dx, dy);` $\ell = |\mathbf{d}|$.
- `if (r.len > 1e-9) { r.ux = dx / r.len; ... }` $\mathbf{u} = \mathbf{d}/\ell$. The guard matters: if two nodes sit exactly on top of each other, $\ell = 0$ and dividing by it would give not-a-number (NaN), which would then spread through every later calculation and destroy the simulation. When ℓ is tiny we keep the default $\mathbf{u} = (1, 0)$. 1e-9 means 10^{-9}.
- `double dvx = v[2L.b] - v[2L.a], dvy = ...;` The relative velocity $\mathbf{v}_b - \mathbf{v}_a$.
- `r.rate = r.ux * dvx + r.uy * dvy;` The dot product $\mathbf{u} \cdot (\mathbf{v}_b - \mathbf{v}_a)$: equation (3.7), $\dot{s}$.
- `if (L.kind == LinkKind::Spring) { r.stretch = r.len - L.L0; r.force = L.k * r.stretch + L.k3 * sss; }` The spring's stretch and the force $f = k\,s + k_3\,s^3$ from (3.5).
- `else { r.force = L.c * r.rate; }` The damper's force $c\,\dot{s}$ from (3.8).

MATHS TO CODE

Maths	Code
$d = r_b - r_a$	`dx, dy`
$\ell = \lvert d \rvert$	`r.len`
$u = d / \ell$	`r.ux, r.uy`
$s = \ell - L_0$	`r.stretch`
$\dot{s} = u \cdot (v_b - v_a)$	`r.rate`
$f = k\,s + k_3\,s^3$ (spring)	`r.force`
$f = c\,\dot{s}$ (damper)	`r.force`

WHAT IF IT WERE DIFFERENT?

What if we forgot the `len > 1e-9` guard? Drag a mass exactly onto its own anchor and the program would divide by zero. In floating-point arithmetic that gives infinity or NaN, the NaN poisons every number, and the screen would fill with garbage. The guard turns a catastrophe into a harmless no-op.

6.2 kinetic: T

ENGINE.HPP (LINES 157 TO 163)

```
157  inline double kinetic(const Model& M, const std::vector<double>& v) {
158      double T = 0;
159      for (size_t i = 0; i < M.nodes.size(); ++i)
160          if (M.nodes[i].kind == NodeKind::Mass)
161              T += 0.5 * M.nodes[i].m * (v[2 * i] * v[2 * i] + v[2 * i + 1] * v[2 * i +
1]);
```

```
 162      return T;
 163  }
```

- Loops over every node and **skips ground anchors** (`if (... == Mass)`), which have no motion.
- `0.5 * m * (vx*vx + vy*vy)` is $T = \frac{1}{2} m |\mathbf{v}|^2$, since $|\mathbf{v}|^2 = v_x^2 + v_y^2$. Summing over masses gives the total kinetic energy.
- Why `vx*vx` and not `pow(vx, 2)`? Multiplication is exact and fast; `pow` is a slower general-purpose function.

6.3 potential: V

ENGINE.HPP (LINES 165 TO 178)

```
 165  inline double potential(const Model& M, const std::vector<double>& q) {
 166      double V = 0;
 167      std::vector<double> zero(q.size(), 0.0);
 168      for (size_t li = 0; li < M.links.size(); ++li) {
 169          if (M.links[li].kind != LinkKind::Spring) continue;
 170          LinkInfo in = linkInfo(M, q, zero, (int)li);
 171          const Link& L = M.links[li];
 172          V += 0.5 * L.k * in.stretch * in.stretch + 0.25 * L.k3 * in.stretch *
in.stretch * in.stretch * in.stretch;
 173      }
 174      for (size_t i = 0; i < M.nodes.size(); ++i)
 175          if (M.nodes[i].kind == NodeKind::Mass)
 176              V -= M.nodes[i].m * M.gravity * (q[2 * i + 1] - M.nodes[i].y);   // y is
down, so going down lowers V
 177      return V;
 178  }
```

- `std::vector<double> zero(q.size(), 0.0);` A list of zeros the same size as `q`. We call `linkInfo` only to get the *geometry*, but that function needs a velocity list, so we hand it a list of zeros.
- The first loop adds, for every **spring** (dampers are skipped, they store no energy), $V = \frac{1}{2} k s^2 + \frac{1}{4} k_3 s^4$ from (3.4).
- The second loop adds each mass's gravity energy: `V -= m * g * (y - y_design)`. This is (3.9): `-=` because $V_g = -m g (y - y_0)$. Moving *down* (y grows) lowers the potential energy, which is why a falling ball speeds up.

WHY?

Why reference the design y? If we used the raw y, a system drawn near the bottom of the canvas would begin with a large positive V and one drawn near the top with a large negative V. Using `(y - y_design)` makes every system start with $V_{gravity} = 0$, which makes the telemetry easy to read. Forces are unaffected, since forces come from *differences* in V.

6.4 rayleigh: D

ENGINE.HPP (LINES 180 TO 188)

```
 180  inline double rayleigh(const Model& M, const std::vector<double>& q, const
std::vector<double>& v) {
 181      double D = 0;
 182      for (size_t li = 0; li < M.links.size(); ++li) {
 183          if (M.links[li].kind != LinkKind::Damper) continue;
 184          LinkInfo in = linkInfo(M, q, v, (int)li);
 185          D += 0.5 * M.links[li].c * in.rate * in.rate;
 186      }
 187      return D;
 188  }
```

- The mirror image of `potential` for dampers: skip springs, and add `0.5 * c * rate * rate`, which is D = ½ c ṡ² from (3.8). It is used to compute the heat power 2D in the energy law (3.13).

6.5 driveForce: the harmonic push

ENGINE.HPP (LINES 191 TO 196)

```
 191  inline void driveForce(const Node& n, double t, double& fx, double& fy) {
 192      double f = n.driveF * std::cos(n.driveW * t);
 193      double a = n.driveAngle * PI / 180.0;
 194      fx = f * std::cos(a);
 195      fy = f * std::sin(a);
 196  }
```

- `double f = n.driveF * std::cos(n.driveW * t);` The size of the push at time t: $F_0 \cos(\omega t)$. It swings between $+F_0$ and $-F_0$.
- `double a = n.driveAngle * PI / 180.0;` Degrees to radians: multiply by π/180. C++ trigonometric functions use radians.
- `fx = f * std::cos(a); fy = f * std::sin(a);` Split the push into x and y parts.
- **`double& fx, double& fy`** are *reference parameters*: the function writes its two answers into the caller's variables. That is how a C++ function returns two values without making a struct.

6.6 inputPower: P = Q · v

ENGINE.HPP (LINES 199 TO 209)

```
 199  inline double inputPower(const Model& M, double t, const std::vector<double>& v, const
std::vector<double>& ext) {
 200      double P = 0;
 201      for (size_t i = 0; i < M.nodes.size(); ++i) {
 202          if (M.nodes[i].kind != NodeKind::Mass) continue;
 203          double fx, fy;
 204          driveForce(M.nodes[i], t, fx, fy);
 205          if (2 * i + 1 < ext.size()) { fx += ext[2 * i]; fy += ext[2 * i + 1]; }
 206          P += fx * v[2 * i] + fy * v[2 * i + 1];
 207      }
 208      return P;
 209  }
```

- This is $P_{in} = \Sigma\, \mathbf{Q} \cdot \mathbf{v}$ from (3.13): force times velocity is power. For every mass it adds the drive force and the mouse force (`ext`) and multiplies by the velocity.
- `if (2*i + 1 < ext.size())` guards against a shorter `ext` vector. The mouse-force list is created when the simulation starts; this check stops a crash if a caller passes an empty list (as the unit tests sometimes do).

6.7 accel: the Euler–Lagrange equations

ENGINE.HPP (LINES 211 TO 242)

```
 211  // ------------------------------------------------------------------ Euler-Lagrange
equations -> accelerations
 212  inline void accel(const Model& M, double t, const std::vector<double>& q, const
std::vector<double>& v,
 213                    const std::vector<double>& ext, std::vector<double>& a) {
 214      const size_t n = M.nodes.size();
 215      std::vector<double> F(2 * n, 0.0);                        // total force on every
node
 216
 217      for (size_t li = 0; li < M.links.size(); ++li) {          // springs and dampers
 218          const Link& L = M.links[li];
 219          LinkInfo in = linkInfo(M, q, v, (int)li);             // in.force = dV/ds
(spring) or dD/dsdot (damper)
 220          // s depends on q_b with slope +u and on q_a with slope -u, so:  -dV/dq_b = -f
u,  -dV/dq_a = +f u
 221          F[2 * L.a]     += in.force * in.ux;
 222          F[2 * L.a + 1] += in.force * in.uy;
 223          F[2 * L.b]     -= in.force * in.ux;
 224          F[2 * L.b + 1] -= in.force * in.uy;
 225      }
 226      a.assign(2 * n, 0.0);
 227      for (size_t i = 0; i < n; ++i) {
 228          const Node& nd = M.nodes[i];
 229          if (nd.kind != NodeKind::Mass) continue;              // ground never
accelerates
 230          double fx = F[2 * i], fy = F[2 * i + 1];
 231          fy += nd.m * M.gravity;                               // -dV/dy of the gravity
term
 232          double dx, dy;
 233          driveForce(nd, t, dx, dy);
 234          fx += dx; fy += dy;
 235          if (2 * i + 1 < ext.size()) { fx += ext[2 * i]; fy += ext[2 * i + 1]; }
 236          double ax = fx / nd.m, ay = fy / nd.m;
 237          int rail = (int)std::lround(nd.rail);
 238          if (rail == RAIL_X) ay = 0;                           // a rail removes one
degree of freedom
 239          if (rail == RAIL_Y) ax = 0;
 240          a[2 * i] = ax; a[2 * i + 1] = ay;
 241      }
 242  }
```

This one function is the whole physics of the program. Read it in two halves.

The setup (first lines).

- `const size_t n = M.nodes.size();` The node count.
- `std::vector<double> F(2 * n, 0.0);` One force per coordinate, all starting at zero. After loop 1 it holds the total *link* force on every node.

Loop 1: every spring and damper.

- `LinkInfo in = linkInfo(M, q, v, (int)li);` The geometry report for this link, including its `force`.
- `F[2L.a] += in.force * in.ux;` *and* `F[2L.a+1] += in.force * in.uy;` The force on node **a** is **+f u** (3.6). The loop adds, so a node touched by several links receives all of their forces.
- `F[2L.b] -= in.force * in.ux;` *and the y-version. The force on node b is −f u*.*

Notice that **springs and dampers go through exactly the same four lines**. The only difference is how `in.force` was computed inside `linkInfo`. This sharing is the practical payoff of the derivation in Chapter 3.

Loop 2: every mass.

- `a.assign(2 * n, 0.0);` The answer array, starting at zero, so ground anchors stay at 0.
- `if (nd.kind != NodeKind::Mass) continue;` Skip anchors: they never accelerate.
- `double fx = F[2i], fy = F[2i+1];` Start from the link forces.
- `fy += nd.m * M.gravity;` Add gravity: $-\partial V_g/\partial y = +m\,g$ (3.9).
- `driveForce(nd, t, dx, dy); fx += dx; fy += dy;` Add the drive.
- `if (2i + 1 < ext.size()) { fx += ext[2i]; ... }` Add the mouse force.
- `double ax = fx / nd.m, ay = fy / nd.m;` Newton's law, rearranged: a = F/m. This is (3.11).
- `if (rail == RAIL_X) ay = 0; if (rail == RAIL_Y) ax = 0;` The rail rule: the blocked acceleration is zeroed.
- `a[2i] = ax; a[2i+1] = ay;` Store the answer.

MATHS TO CODE

Equation	Line of code
(3.6) spring/damper force on a: +f u	`F[2*L.a] += in.force * in.ux`
(3.6) spring/damper force on b: −f u	`F[2*L.b] -= in.force * in.ux`
(3.9) gravity force +m g	`fy += nd.m * M.gravity`
drive F cos(ωt)	`driveForce(...)` then `fx += dx`
mouse force	`fx += ext[2*i]`
(3.10) m a = ΣF	`ax = fx / nd.m`
rails	`if (rail == RAIL_X) ay = 0`

WHAT IF IT WERE DIFFERENT?

What if we computed `fx / nd.m` for a ground anchor? Ground has the default mass 1, so it would move when pulled! The `continue` for non-masses is what makes a wall a wall.

TRY IT

Select a spring in example 2, raise its stiffness to the maximum, and watch the masses vibrate faster. Then lower the *mass* slider of a block to its minimum (0.1 kg) and raise the stiffness again. If the motion became violent or the numbers blew up, you are seeing what the stability limit of Chapter 4 protects against. The program notices non-finite numbers and stops with a message.

6.8 Chapter checks

QUIZ

1. Why does `linkInfo` return a struct, not just one number?
2. What does the guard `len > 1e-9` prevent?
3. Why does `potential` pass a list of zeros to `linkInfo`?
4. In `accel`, which two lines apply a spring's force to its two ends, and why do their signs differ?
5. Which line makes a wall a wall?
6. Why is the mouse force added in `accel` and not in the integrator?

ANSWERS

1. Because the length, unit vector, stretch, rate and force are all needed by different callers; computing them once avoids repeating the work and the bugs.
2. Division by zero when two nodes coincide, which would create NaN.
3. It needs only geometry, but `linkInfo` requires velocities, so we give it zeros.
4. `F[2L.a] += ...` *and* `F[2L.b] -= ...`: the two ends are pulled in opposite directions (equal and opposite forces).
5. `if (nd.kind != NodeKind::Mass) continue;`, so ground nodes keep zero acceleration.
6. Because the force is part of the equation of motion at every stage of RK4 and must be included whenever the acceleration is evaluated.

CHAPTER 7

The Integrator and the Meter: engine.hpp, Part 3

7.1 rk4: one step of the fourth-order Runge–Kutta method

ENGINE.HPP (LINES 244 TO 263)

```
 244  // ------------------------------------------------------------------ one RK4 step
 245  inline void rk4(const Model& M, State& s, const std::vector<double>& ext, double h) {
 246      const size_t N = s.q.size();
 247      std::vector<double> a1, a2, a3, a4, qt(N), vt(N), v2, v3, v4;
 248      accel(M, s.t, s.q, s.v, ext, a1);
 249      for (size_t i = 0; i < N; ++i) { qt[i] = s.q[i] + 0.5 * h * s.v[i]; vt[i] = s.v[i]
+ 0.5 * h * a1[i]; }
 250      v2 = vt;
 251      accel(M, s.t + 0.5 * h, qt, vt, ext, a2);
 252      for (size_t i = 0; i < N; ++i) { qt[i] = s.q[i] + 0.5 * h * v2[i]; vt[i] = s.v[i] +
0.5 * h * a2[i]; }
 253      v3 = vt;
 254      accel(M, s.t + 0.5 * h, qt, vt, ext, a3);
 255      for (size_t i = 0; i < N; ++i) { qt[i] = s.q[i] + h * v3[i]; vt[i] = s.v[i] + h *
a3[i]; }
 256      v4 = vt;
 257      accel(M, s.t + h, qt, vt, ext, a4);
 258      for (size_t i = 0; i < N; ++i) {
 259          s.q[i] += h / 6.0 * (s.v[i] + 2 * v2[i] + 2 * v3[i] + v4[i]);
 260          s.v[i] += h / 6.0 * (a1[i] + 2 * a2[i] + 2 * a3[i] + a4[i]);
 261      }
 262      s.t += h;
 263  }
```

This function advances the state by one small time step *h*. Compare it with equation (4.3) and Figure 4.2.

- `const size_t N = s.q.size();` The number of coordinates (twice the node count).
- `std::vector<double> a1, a2, a3, a4, qt(N), vt(N), v2, v3, v4;` Scratch space. `a1..a4` hold the four acceleration samples (the "slopes" of velocity). `qt, vt` are the trial positions and velocities at which each new sample is taken. `v2, v3, v4` remember the trial velocities, because the *position* updates need them.
- **Stage 1.** `accel(M, s.t, s.q, s.v, ext, a1);` Acceleration at the start of the step: that is k_1 (for velocity). The slope of position at the start is just the velocity `s.v`.
- **Stage 2.** `qt[i] = s.q[i] + 0.5hs.v[i]; vt[i] = s.v[i] + 0.5ha1[i];` Walk half a step along the stage-1 slopes to a trial point. `v2 = vt;` remembers that trial velocity (it is the position-slope at stage 2). Then `accel(M, s.t + 0.5*h, qt, vt, ext, a2);` measures the acceleration there.

- **Stage 3.** Same again, but walk half a step along the *stage-2* slopes: `qt = q + 0.5hv2; vt = v + 0.5ha2`. Save `v3`, measure `a3`.
- **Stage 4.** Walk a *full* step along the stage-3 slopes: `qt = q + hv3; vt = v + ha3`. Save `v4`, measure `a4` at time `t + h`.
- **The final update.** `s.q[i] += h/6.0 * (s.v[i] + 2v2[i] + 2v3[i] + v4[i]);` and `s.v[i] += h/6.0 * (a1[i] + 2a2[i] + 2a3[i] + a4[i]);` The weighted average of equation (4.3), with weights 1, 2, 2, 1.
- `s.t += h;` Time moves forward.

> **WHY?**
>
> **Why do the position slopes (v, v2, v3, v4) differ?** Position changes at the rate of *velocity*, and velocity itself changes during the step. At the start the velocity is `s.v`; the trial velocities at the later stages are the best estimates of the velocity half-way and at the end. RK4 averages all four, which is the source of its accuracy.

> **WHAT IF IT WERE DIFFERENT?**
>
> **What if `accel` did not receive the time `t` at each stage?** The drive force F cos(ωt) depends on t. If every stage used the *start* time, the drive would be sampled only once per step and RK4 would lose its accuracy for driven systems. That is why `accel` takes `t + 0.5*h` and `t + h` at the later stages.

7.2 The Meter: checking the law of energy

ENGINE.HPP (LINES 265 TO 274)

```
 265  // ------------------------------------------------------------------ energy book-keeping (the "meter")
 266  // Law of energy:   E(t) = E(0) + work_in(t) - dissipated(t).   residual() should stay near 0.
 267  struct Meter {
 268      double T = 0, V = 0, E = 0, dissipated = 0, workIn = 0, E0 = 0;
 269      double residual() const { return E - (E0 + workIn - dissipated); }
 270      void measure(const Model& M, const State& s) { T = kinetic(M, s.v); V = potential(M, s.q); E = T + V; }
 271      void reset(const Model& M, const State& s) { measure(M, s); dissipated = workIn = 0; E0 = E; }
 272      // After a live edit (a slider moved) energy jumps for non-physical reasons: restart the books.
 273      void rebase(const Model& M, const State& s) { measure(M, s); E0 = E - (workIn - dissipated); }
 274  };
```

The Meter keeps the books of equation (3.14).

- `T, V, E`: the current kinetic, potential and total energy.
- `dissipated`: the running total of ∫ 2D dt, the heat produced so far.
- `workIn`: the running total of $\int P_{in}\,dt$, the work done by outside forces.
- `E0`: the energy at the start (or at the last "rebase").

- `double residual() const { return E - (E0 + workIn - dissipated); }` The *error*: the difference between the energy we measure and the energy the law (3.14) says we should have. In a perfect simulation it is zero. The telemetry box shows it as "energy-law error".
- `measure(M, s)` recomputes T, V and E from the state.
- `reset(M, s)` measures, zeroes the two totals, and records E_0. It is called when you press Run.
- `rebase(M, s)` is the subtle one. When you move a slider *during* a run (say, doubling a spring's stiffness), the potential energy jumps instantly. That is not a violation of physics, it is you reaching in and changing the system. To avoid a false alarm, `rebase` sets `E0 = E - (workIn - dissipated)`, so that the residual is exactly zero right after the edit, and the books continue from there.

> **WHY?**
>
> **Why rebase instead of ignoring the check while sliders move?** Because the check is only trustworthy when we keep it honest. After a rebase the residual is zero, and any later growth is a real warning.

7.3 advance: step and keep the books

ENGINE.HPP (LINES 276 TO 284)

```
 276  // Step the simulation AND keep the meter up to date (trapezoid rule for the two power
integrals).
 277  inline void advance(const Model& M, State& s, const std::vector<double>& ext, double h,
Meter& meter) {
 278      double p0 = inputPower(M, s.t, s.v, ext), d0 = 2.0 * rayleigh(M, s.q, s.v);
 279      rk4(M, s, ext, h);
 280      double p1 = inputPower(M, s.t, s.v, ext), d1 = 2.0 * rayleigh(M, s.q, s.v);
 281      meter.workIn     += 0.5 * (p0 + p1) * h;
 282      meter.dissipated += 0.5 * (d0 + d1) * h;
 283      meter.measure(M, s);
 284  }
```

- `double p0 = inputPower(...), d0 = 2.0 * rayleigh(...);` The input power and the heat power 2D at the **start** of the step.
- `rk4(M, s, ext, h);` Take the step.
- `double p1 = ..., d1 = ...;` The same two powers at the **end** of the step.
- `meter.workIn += 0.5 * (p0 + p1) * h;` and `meter.dissipated += 0.5 * (d0 + d1) * h;` The **trapezoid rule**: the area under the power curve over one step is the average of the two ends times the width. This is accurate to second order, so the books stay accurate.
- `meter.measure(M, s);` Update T, V and E from the new state.

This is the function the interface calls 1000 times per simulated second.

7.4 dominantFrequency: the frequency finder

ENGINE.HPP (LINES 286 TO 310)

```
 286  // ------------------------------------------------------------------ frequency finder
(telemetry)
 287  // Slow discrete Fourier transform, good enough for a few hundred samples. Returns Hz,
or 0 if no clear tone.
 288  inline double dominantFrequency(const std::vector<double>& x, double sampleRate) {
 289      const size_t N = x.size();
 290      if (N < 32) return 0;
 291      double mean = 0;
 292      for (double d : x) mean += d;
 293      mean /= (double)N;
 294      double best = 0, bestF = 0, total = 0;
 295      int count = 0;
 296      for (double f = 0.05; f <= std::fmin(30.0, sampleRate / 2.2); f += 0.025, ++count)
{
 297          double re = 0, im = 0;
 298          for (size_t i = 0; i < N; ++i) {
 299              double w = 0.5 - 0.5 * std::cos(2 * PI * (double)i / (double)(N - 1));   //
Hann window
 300              double ang = 2 * PI * f * (double)i / sampleRate;
 301              re += (x[i] - mean) * w * std::cos(ang);
 302              im -= (x[i] - mean) * w * std::sin(ang);
 303          }
 304          double mag = std::hypot(re, im);
 305          total += mag;
 306          if (mag > best) { best = mag; bestF = f; }
 307      }
 308      // a real tone sticks out: its peak must be well above the average of all
frequencies tried
 309      return (best > 1e-6 && count > 0 && best > 4.0 * total / count) ? bestF : 0;
 310  }
```

The function takes a list of samples and the sample rate, and returns the strongest frequency in hertz (or 0 if nothing stands out).

- `if (N < 32) return 0;` Too few samples to say anything.
- The **mean** is computed and later subtracted, so a steady offset (sag) does not count.
- `for (double f = 0.05; f <= std::fmin(30.0, sampleRate / 2.2); f += 0.025, ++count)` Try frequencies from 0.05 Hz up to 30 Hz in steps of 0.025 Hz. The upper limit is also capped at `sampleRate/2.2`: you cannot see a wiggle faster than half the sample rate (the **Nyquist limit**); 2.2 leaves a safety margin.
- The inner loop computes the real and imaginary parts of equation (4.4): `re += (x[i]-mean) * w * cos(ang)` and `im -= (x[i]-mean) * w * sin(ang)`, where `w` is the Hann window. Together `re` and `im` measure how much a cosine and a sine of that frequency match the data.
- `double mag = std::hypot(re, im);` The strength at this frequency.
- The best strength and its frequency are remembered; the sum `total` and `count` let us compute the **average** strength.
- **The last line:** `return (best > 1e-6 && count > 0 && best > 4.0 * total / count) ? bestF : 0;` A real tone must stand out: its strength must be at least **four times the**

average over all tested frequencies. A flat, noisy spectrum (no clear tone) returns 0 and the interface shows "--".

WHAT IF IT WERE DIFFERENT?

What if there were no threshold? The function would always return *some* frequency, even for a motionless system or pure noise, and the display would show a meaningless number. A measurement that can say "I don't know" is more honest than one that always answers.

7.5 Chapter checks

QUIZ

1. How many times does `rk4` call `accel`?
2. Why are `v2`, `v3`, `v4` saved?
3. What does `residual()` measure, and what value would a perfect run give?
4. Why is `rebase` needed?
5. What does the trapezoid rule do in `advance`?
6. What would `dominantFrequency` return for a motionless system?

ANSWERS

1. Four times per step.
2. Because the position updates in stages 3 and 4 and in the final average use those trial velocities.
3. The gap between the measured energy and E_0 + work in − heat. Zero.
4. After a live edit the energy jumps for non-physical reasons; rebasing zeroes the residual so later growth is a real warning.
5. It averages the power at the start and end of the step to integrate it accurately.
6. 0 (the signal has no peak above the threshold).

CHAPTER 8

Saving and Examples: model_io.hpp

The second file of the engine layer has two jobs: save and load a design to a text file, and provide four ready-made example designs.

8.1 The file format

A design file is plain text, one thing per line. Here is what example 1 looks like when saved:

```
vlab 1
# Vibration Lab design file
gravity 0
ground -3 0
mass 0 0 2 0 0 1 0 0 0
spring 0 1 30 0 2 0
damper 0 1 1.5
```

- The first word of a line says what it is. `mass` is followed by nine numbers (x, y, m, vx0, vy0, rail, driveF, driveAngle, driveW). `ground` has two. `spring` has k, k_3, L_0 and the autoRest flag; `damper` has c.
- Nodes appear in order, and links refer to them by **index** (0, 1, ...). So in this file the spring joins node 0 (the ground) and node 1 (the mass).
- A `#` starts a comment. The first line, `vlab 1`, is a **magic word with a version number**: the loader refuses files without it, so it cannot mistake some other text file for a design.

> **WHY?**
> **Why a text format and not a binary one?** You can open the file in NOVA and read or edit it, send it by email, and compare two versions line by line. A binary file would be a few bytes smaller and impossible to inspect. For a few dozen numbers, text is the better trade.

8.2 saveModel

MODEL_IO.HPP (LINES 19 TO 39)

```
19  inline bool saveModel(const std::string& path, const Model& M) {
20      std::ofstream f(path);
21      if (!f) return false;
22      f.precision(9);
23      f << "vlab 1\n# Vibration Lab design file\n";
24      f << "gravity " << M.gravity << "\n";
25      for (const Node& n : M.nodes) {
26          if (n.kind == NodeKind::Mass)
27              f << "mass " << n.x << " " << n.y << " " << n.m << " " << n.vx0 << " " <<
```

```
n.vy0 << " " << n.rail << " "
  28                << n.driveF << " " << n.driveAngle << " " << n.driveW << "\n";
  29          else
  30              f << "ground " << n.x << " " << n.y << "\n";
  31      }
  32      for (const Link& l : M.links) {
  33          if (l.kind == LinkKind::Spring)
  34              f << "spring " << l.a << " " << l.b << " " << l.k << " " << l.k3 << " " <<
l.L0 << " " << (l.autoRest ? 1 : 0) << "\n";
  35          else
  36              f << "damper " << l.a << " " << l.b << " " << l.c << "\n";
  37      }
  38      return true;
  39  }
```

- `std::ofstream f(path); if (!f) return false;` Open the file for writing; if it cannot be opened (a locked folder, say), return `false` so the interface can show an error.
- `f.precision(9);` Write numbers with 9 significant digits, enough that loading gives back the same design to within rounding.
- `f << "vlab 1\n# ..."` writes the header. `\n` is a newline.
- Then one loop over `M.nodes` writes a `mass` or `ground` line, and a second loop over `M.links` writes a `spring` or `damper` line. `(l.autoRest ? 1 : 0)` writes a boolean as 1 or 0.
- `return true;` when finished. The file closes automatically when `f` goes out of scope (a C++ feature called RAII: the object cleans up after itself).

8.3 loadModel

MODEL_IO.HPP (LINES 41 TO 78)

```
  41  inline bool loadModel(const std::string& path, Model& out) {
  42      std::ifstream f(path);
  43      if (!f) return false;
  44      Model M;
  45      std::string line;
  46      bool header = false;
  47      while (std::getline(f, line)) {
  48          size_t h = line.find('#');
  49          if (h != std::string::npos) line.erase(h);
  50          std::istringstream ss(line);
  51          std::string word;
  52          if (!(ss >> word)) continue;
  53          if (word == "vlab") { header = true; continue; }
  54          if (word == "gravity") { ss >> M.gravity; }
  55          else if (word == "mass") {
  56              Node n; n.kind = NodeKind::Mass;
  57              ss >> n.x >> n.y >> n.m >> n.vx0 >> n.vy0 >> n.rail >> n.driveF >>
n.driveAngle >> n.driveW;
  58              if (n.m < 0.01) n.m = 0.01;
  59              if ((int)M.nodes.size() < MAX_NODES) M.nodes.push_back(n);
  60          } else if (word == "ground") {
  61              Node n; n.kind = NodeKind::Ground;
  62              ss >> n.x >> n.y;
  63              if ((int)M.nodes.size() < MAX_NODES) M.nodes.push_back(n);
  64          } else if (word == "spring" || word == "damper") {
  65              Link l; int ar = 1;
  66              l.a = l.b = -1;
  67              l.kind = (word == "spring") ? LinkKind::Spring : LinkKind::Damper;
  68              ss >> l.a >> l.b;
  69              if (l.kind == LinkKind::Spring) { ss >> l.k >> l.k3 >> l.L0 >> ar;
```

```
l.autoRest = (ar != 0); }
  70              else ss >> l.c;
  71              int n = (int)M.nodes.size();
  72              if (l.a >= 0 && l.b >= 0 && l.a < n && l.b < n && l.a != l.b)
M.links.push_back(l);
  73          }
  74      }
  75      if (!header) return false;
  76      out = M;
  77      return true;
  78  }
```

Reading is harder than writing, because **files can be wrong**: the user may have edited them, or they may be damaged. So this function is defensive.

- `Model M;` Read into a **temporary** model, not the real one. The real model changes only on success, at the very end: `out = M;`. If anything goes wrong halfway, the user's current design is untouched.
- `while (std::getline(f, line))` Read one line at a time.
- `size_t h = line.find('#'); if (h != npos) line.erase(h);` Throw away comments.
- `std::istringstream ss(line); std::string word; if (!(ss >> word)) continue;` Make the line readable like a stream; read the first word; if the line is empty, skip it.
- The chain of `if (word == "gravity") ... else if (word == "mass") ...` chooses what to do. `ss >> n.x >> n.y ...` reads numbers in order. If a number is missing, that field keeps its default.
- `if (n.m < 0.01) n.m = 0.01;` A mass of zero or less in a hand-edited file would make `accel` divide by zero; clamp it to a small positive value.
- `if ((int)M.nodes.size() < MAX_NODES) M.nodes.push_back(n);` Respect the node limit.
- For links: `if (l.a >= 0 && l.b >= 0 && l.a < n && l.b < n && l.a != l.b) M.links.push_back(l);` **Validate the indices** against the number of nodes read so far. A link pointing at node 99 in a file with three nodes is silently dropped instead of crashing the program later.
- `if (!header) return false;` No `vlab` line means it is not our file.

> **WHAT IF IT WERE DIFFERENT?**
>
> **What if the loader trusted the file?** A file with `spring 0 99 ...` would create a link to a node that does not exist. The first time `accel` indexed `q[299]` *the program would read memory beyond the array, giving garbage or a crash. Never trust data that comes from outside your program.**

8.4 The four examples

MODEL_IO.HPP (LINES 81 TO 89)

```
  81  inline Model presetOneDof() {                // the classic: ground - spring+damper -
mass, stretched 1 m then released
  82      Model M;
  83      int g = M.addGround(-3, 0), m = M.addMass(0, 0, 2.0);
  84      M.nodes[m].rail = RAIL_X;
  85      int s = M.addLink(LinkKind::Spring, g, m), d = M.addLink(LinkKind::Damper, g, m);
  86      M.links[s].k = 30; M.links[s].L0 = 2.0; M.links[s].autoRest = false;    // rest
length 2 m, drawn 3 m -> starts stretched by 1 m
  87      M.links[d].c = 1.5;
  88      return M;
  89  }
```

- Builds the classic single-mass system: `addGround(-3, 0)` (a wall 3 m to the left), `addMass(0, 0, 2.0)` (a 2 kg block), forced onto an X rail so it moves along a line.
- `addLink(Spring, g, m)` and `addLink(Damper, g, m)` join them in parallel.
- `M.links[s].k = 30; M.links[s].L0 = 2.0; M.links[s].autoRest = false;` The rest length is set to **2 m** while the drawn distance is **3 m**, so the spring starts **stretched by 1 m**. That is how this example "pulls the block aside and lets go" without needing a starting-velocity or a mouse. `autoRest = false` stops it from being reset to 3 m.
- `M.links[d].c = 1.5;` The damper.

This is exactly the system analysed in Chapter 3 Step 10: $\omega_n = 3.873$ rad/s, $\zeta = 0.0968$.

MODEL_IO.HPP (LINES 90 TO 102)

```
  90  inline Model presetTwoDof() {                // two masses on a rail between two walls
  91      Model M;
  92      int g1 = M.addGround(-4, 0), a = M.addMass(-1.25, 0, 1.0), b = M.addMass(1.25, 0,
1.0), g2 = M.addGround(4, 0);
  93      M.nodes[a].rail = M.nodes[b].rail = RAIL_X;
  94      M.nodes[a].vx0 = 3.0;
  95      int ids[3][2] = {{g1, a}, {a, b}, {b, g2}};
  96      double k[3] = {40, 25, 40}, c[3] = {0.6, 0.2, 0.6};
  97      for (int i = 0; i < 3; ++i) {
  98          int s = M.addLink(LinkKind::Spring, ids[i][0], ids[i][1]); M.links[s].k = k[i];
  99          int d = M.addLink(LinkKind::Damper, ids[i][0], ids[i][1]); M.links[d].c = c[i];
 100      }
 101      return M;
 102  }
```

- Two masses, two walls, three spring-damper pairs, built with a small loop over a table: `int ids[3][2] = {{g1, a}, {a, b}, {b, g2}};` lists which two nodes each pair joins, and the arrays `k[3]` and `c[3]` give their stiffness and damping. A table plus a loop is shorter, and easier to change, than six hand-written blocks.
- `M.nodes[a].vx0 = 3.0;` gives the first mass a push at the start.

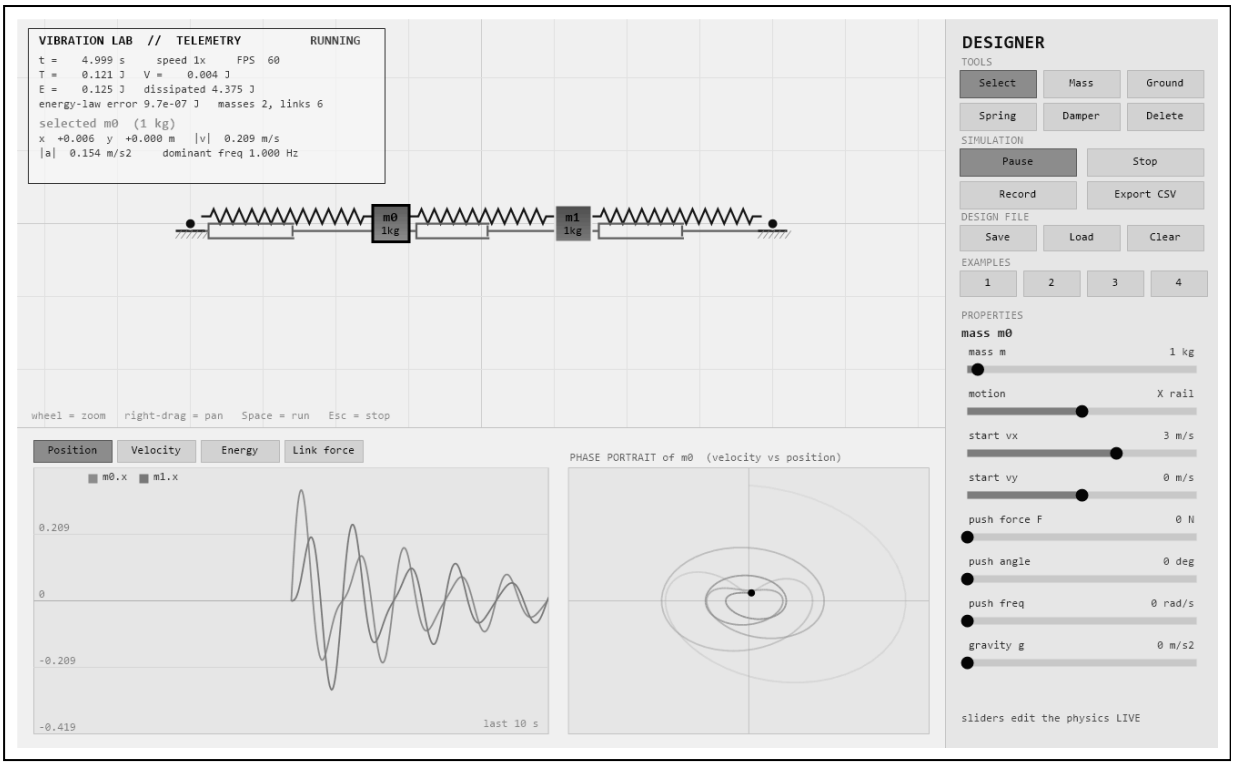


Figure 8.1. *Example 2 after five seconds: two coupled masses. Each curve is a mixture of two vibration patterns, so the energy slowly passes back and forth between the masses.*

- Two coupled masses have **two** natural frequencies (they can move together or against each other). In the telemetry you will see the motion is a mixture of both: a "beat" pattern.

MODEL_IO.HPP (LINES 103 TO 111)

```
 103  inline Model presetSpringPendulum() {          // a mass on a spring hanging from the
ceiling, free to swing and bounce
 104      Model M;
 105      M.gravity = 9.81;
 106      int g = M.addGround(0, -1.8), m = M.addMass(1.6, 0.6, 1.0);
 107      int s = M.addLink(LinkKind::Spring, g, m), d = M.addLink(LinkKind::Damper, g, m);
 108      M.links[s].k = 60; M.links[s].L0 = 2.0; M.links[s].autoRest = false;
 109      M.links[d].c = 0.25;
 110      return M;
 111  }
```

- `M.gravity = 9.81;` Earth's gravity. One anchor on the "ceiling" at (0, −1.8) and one **free** mass (no rail) at (1.6, 0.6), joined by a spring with rest length 2 m. A mass on a spring that can both bounce and swing is a **spring pendulum**; its motion is rich and, for some starting positions, chaotic. It is the best example of why we used a general 2-D engine rather than a 1-D one.

MODEL_IO.HPP (LINES 112 TO 122)

```
 112  inline Model presetQuarterCar() {              // road - tyre spring - wheel -
suspension spring+damper - car body
 113      Model M;
 114      M.gravity = 9.81;
 115      int road = M.addGround(0, 2.2), wheel = M.addMass(0, 1.0, 1.0), body = M.addMass(0,
-1.0, 4.0);
 116      M.nodes[wheel].rail = M.nodes[body].rail = RAIL_Y;
 117      M.nodes[wheel].vy0 = -2.0;                 // the wheel hits a bump
 118      int t = M.addLink(LinkKind::Spring, road, wheel); M.links[t].k = 220;
```

```
119      int s = M.addLink(LinkKind::Spring, wheel, body); M.links[s].k = 70;
120      int d = M.addLink(LinkKind::Damper, wheel, body); M.links[d].c = 5.0;
121      return M;
122  }
```

- The **quarter-car** model used in vehicle engineering: one wheel and one quarter of the car's body. The road is a ground anchor; the tyre is a spring (k = 220); the suspension is a spring (k = 70) plus a damper (c = 5) between the wheel (1 kg) and the body (4 kg). Both masses are on Y rails: they can only move up and down.
- `M.nodes[wheel].vy0 = -2.0;` Gives the wheel an upward speed of 2 m/s (negative y is up): the wheel has just hit a bump. Watch the body ride it out.
- With gravity switched on, the system first **sags** to its static equilibrium (Chapter 3, formula 3.18) before the bump response is visible.

MODEL_IO.HPP (LINES 123 TO 130)

```
123  inline Model presetByNumber(int n) {
124      switch (n) {
125          case 1: return presetOneDof();
126          case 2: return presetTwoDof();
127          case 3: return presetSpringPendulum();
128          default: return presetQuarterCar();
129      }
130  }
```

- A small **switch**: number 1 gives the first example, 2 the second, 3 the third, anything else the fourth. The buttons 1 to 4 and the keys 1 to 4 call this.

8.5 Chapter checks

QUIZ

1. Why does `loadModel` read into a temporary `Model`?
2. What does the first word `vlab` protect against?
3. Why is `autoRest` set to `false` in example 1?
4. Which example needs a free (railless) mass, and why?
5. What bad thing could happen if `loadModel` did not check link indices?

ANSWERS

1. So a failed or partial load leaves the user's current design untouched; the real model is replaced only on success.
2. Loading some other text file by mistake: without the header the loader returns false.
3. Otherwise the rest length would be reset to the drawn length (3 m) and the spring would start relaxed instead of stretched by 1 m.
4. Example 3 (the spring pendulum): it must swing in two dimensions.
5. The engine would later read outside its arrays: garbage numbers or a crash.

Part III — The Interface, Line by Line

CHAPTER 9

How a Windows Program Works

Before reading the interface code you need a mental model of what a Windows program *is*. This chapter gives it, in plain words, so that Chapters 10 to 13 read easily.

9.1 A program that waits for events

A command-line program runs from the first line to the last and stops. A **window** program is different: it opens a window and then *waits*. Whenever something happens (you move the mouse, press a key, resize the window, ask it to close) Windows puts a **message** into the program's **message queue**. The program's job is a loop: take a message, deal with it, repeat, forever, until the window is closed.

> **KEY WORD**
>
> **Message.** A small note from Windows to your program: "the mouse moved to (310, 225)", "the key W was pressed", "the window is now 1320 by 800".
>
> **Window procedure (WndProc).** The function *you* write that receives every message for your window. It has a big `switch` that says what to do for each kind of message.
>
> **Event loop.** The never-ending loop that fetches messages and passes them to the window procedure.

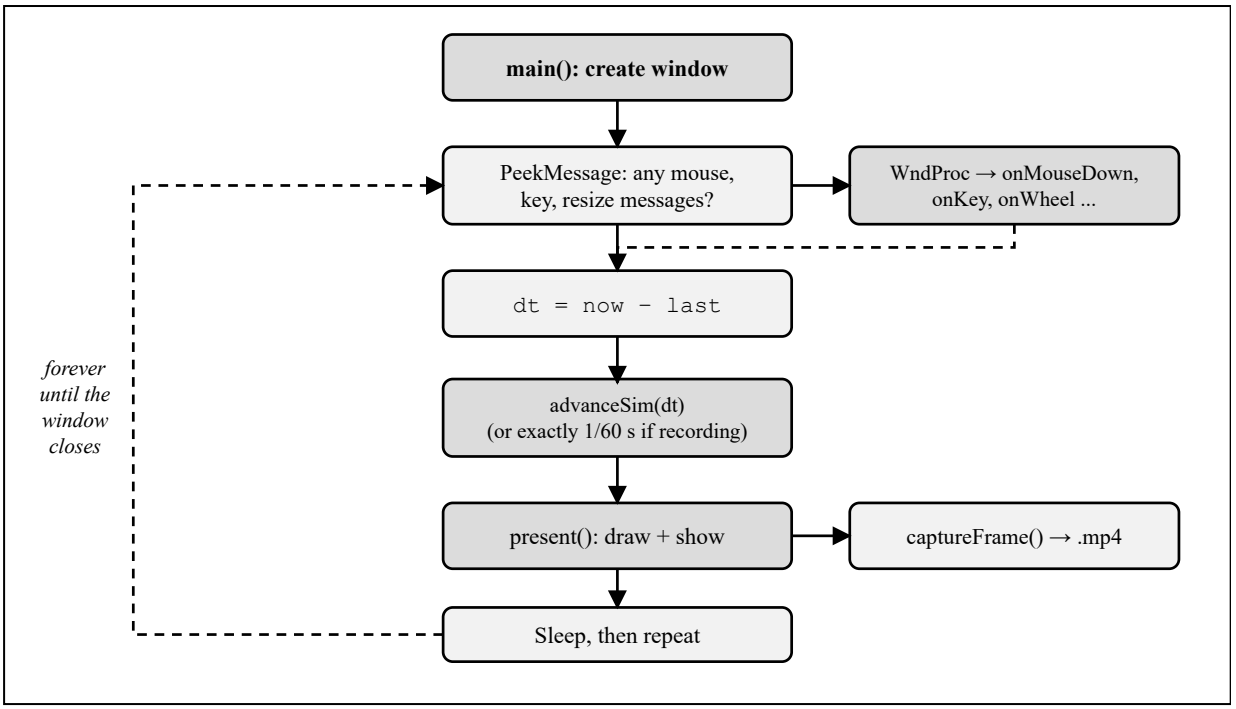


Figure 9.1. *The heart of the program (the main loop at the bottom of live.cpp). Each pass handles pending messages, moves the physics forward, draws, and optionally captures a video frame.*

9.2 Two kinds of loops: waiting and polling

The classic loop uses `GetMessage`, which **sleeps** until a message arrives. That is perfect for a word processor that does nothing between keystrokes. Our program is more like a game: the physics and the picture must keep changing **even when you touch nothing**. So we use `PeekMessage`, which says "is there a message? If not, carry on" and returns at once. The loop therefore alternates *handle any messages* and *do one frame of work*.

> **WHAT IF IT WERE DIFFERENT?**
>
> **What if we used `GetMessage`?** The window would freeze whenever you stopped moving the mouse, because the loop would be asleep waiting for the next message. The simulation would only advance when you wiggled the mouse.

9.3 Drawing: GDI+, brushes and pens

Windows can draw in several ways. We use **GDI+**, a drawing library built into Windows. It gives us:

- a `Graphics` object: a canvas you draw on;
- `Pen`s for lines (a colour and a width) and `SolidBrush`es for filled shapes;
- `DrawLine`, `DrawLines`, `FillRectangle`, `DrawString`, and so on;
- **anti-aliasing**: the smooth, un-jagged edges you see on the springs and circles;

- **transformations**: `TranslateTransform` and `RotateTransform` shift and turn the whole canvas, which is how a spring is drawn at any angle (Chapter 11).

9.4 Double buffering: never draw straight onto the screen

If you draw the background, then the springs, then the masses straight onto the visible window, the user sees each step happen: a flicker. The cure is **double buffering**: draw everything onto a hidden picture in memory, and only when it is complete, copy the whole thing to the window in one quick move (`BitBlt`, "bit-block transfer").

Our hidden picture is a **DIB section**: a bitmap whose pixels we can reach directly, as a plain block of memory. In `live.cpp` it is the struct `Canvas`.

> **WHY?**
> **This one choice made recording almost free.** Because we always draw into a block of memory we own, the video recorder can take the *same pixels* and hand them to the encoder. If we had drawn straight to the window, we would need a slow and fragile screen-capture step. One drawing, two uses: show it on the screen, and write it to the file.

9.5 The coordinate systems in this program

Three different "worlds" meet in this program, and mixing them up is the commonest source of bugs.

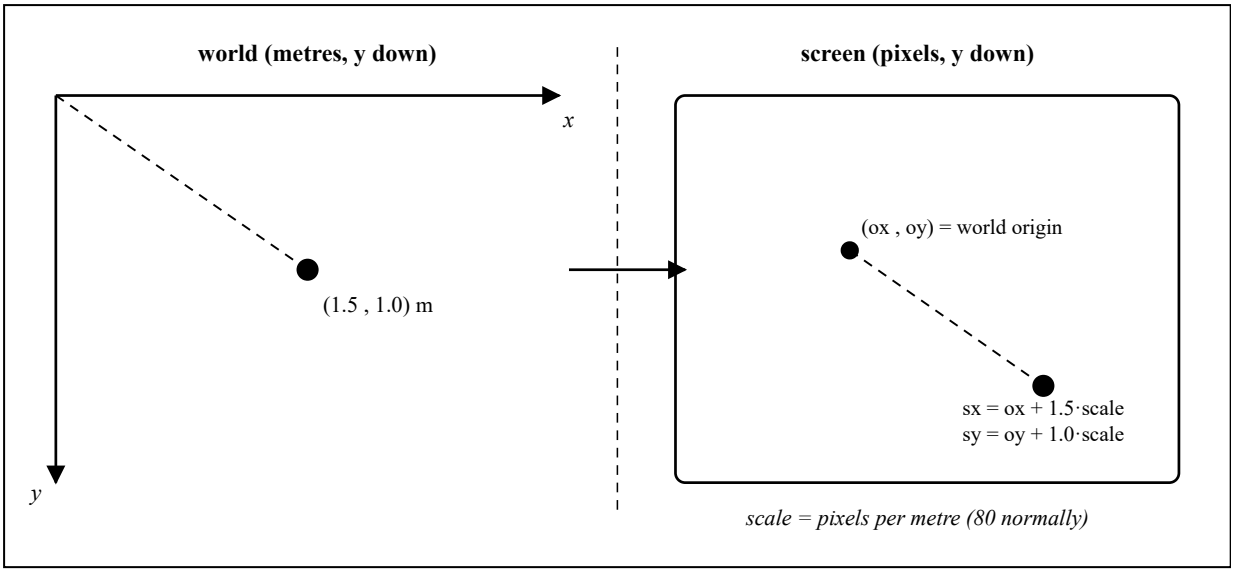


Figure 9.2. *The world is in metres with y pointing down; the screen is in pixels with y pointing down. Two numbers (the origin and the scale) convert between them.*

System	Unit	Origin	Used by
World	metres	wherever the user's view puts (0, 0)	the physics, the saved file
Screen / canvas	pixels	top-left of the window	all drawing, the mouse
Local (while drawing a link)	pixels	at one end of the link, x axis along the link	the spring and damper drawing routines

The conversion is a single line each way: `sx = ox + x * scale` and `x = (sx - ox) / scale`. The *view* is the three numbers `ox`, `oy`, `scale`. Panning changes `ox, oy`; zooming changes `scale`. The physics never sees them, so zooming never changes the simulation.

9.6 How the files fit together

The three interface files divide the work like a small company:

File	Role	Analogy
`app.hpp`	Decides what *happens*: what is selected, what a click does, which mode it is in	the manager
`render.hpp`	Decides what it *looks like*: it only reads state, never changes it	the artist
`live.cpp`	Talks to Windows: window, messages, loop, files, recording	the receptionist

WHY?

Why can `render.hpp` never change the state? If drawing code could change state, then drawing twice (once for the screen, once for the video) could give different results, and a bug in a drawing routine could corrupt the simulation. Keeping drawing *read-only* makes the picture a pure function of the state.

9.7 Chapter checks

QUIZ

1. What is a message, and who sends it?
2. Why does the program use `PeekMessage` and not `GetMessage`?
3. What is double buffering and what problem does it cure?
4. In which coordinate system does the physics work?
5. Why can zooming not change the physics?

ANSWERS

1. A note from Windows about an event (mouse, key, resize, close).

2. `PeekMessage` returns immediately so the simulation and drawing keep going when no messages arrive.
3. Drawing to a hidden bitmap and copying the finished picture to the window; it cures flicker (and lets the recorder reuse the picture).
4. In the world system: metres.
5. Zoom changes only the view (`scale`), which is used only for drawing and mouse conversion.

CHAPTER 10

The Brain: app.hpp

`app.hpp` is the file that decides *what happens* when you do something. It holds all the program's memory (the **application state**) and a family of small functions, each triggered by a click or a key.

10.1 The header and the vocabulary

APP.HPP (LINES 1 TO 18)

```
   1  // app.hpp -- the brain of the interface: what state the program keeps, and what every
click / key does.
   2  // (No drawing happens here -- that is render.hpp -- and no Windows plumbing -- that is
live.cpp.)
   3  //
   4  // Included by live.cpp AFTER <windows.h> and <gdiplus.h>.
   5  #pragma once
   6  #include <cmath>
   7  #include <cstdio>
   8  #include <cstring>
   9  #include <deque>
  10  #include <string>
  11  #include <vector>
  12
  13  #include "engine.hpp"
  14  #include "model_io.hpp"
  15
  16  #ifndef LAB_DIR
  17  #define LAB_DIR "."
  18  #endif
```

- The comment states the file's job and, importantly, what it does **not** do: no drawing and no Windows plumbing.
- `#pragma once` and the standard includes you already know.
- `#include "engine.hpp"` **and** `"model_io.hpp"` bring in the physics and the file code. (A name in quotes means "look next to this file first"; angle brackets mean "look in the compiler's own folders".)
- `#ifndef LAB_DIR / #define LAB_DIR "."` gives `LAB_DIR` a fallback. The real value is supplied by CMake (`target_compile_definitions(... LAB_DIR="...")`) and is the folder containing the source, so the program finds `design.vlab` and writes `recordings/` in the same place no matter which folder the IDE starts it from.

APP.HPP (LINES 21 TO 29)

```
  21  static const int    PANEL_W      = 300;     // width of the right-hand panel (pixels)
  22  static const double SCENE_FRAC   = 0.56;    // the drawing area takes this fraction of
the window height
  23  static const double PLOT_SECONDS = 10.0;    // the time plot shows the most recent 10 s
  24  static const double SIM_STEP     = 0.001;   // physics step: 1 millisecond
```

```
  25  static const size_t LOG_LIMIT    = 36000;   // keep at most 10 minutes of telemetry at
60 samples/s
  26
  27  enum Tool    { TOOL_SELECT, TOOL_MASS, TOOL_GROUND, TOOL_SPRING, TOOL_DAMPER };
  28  enum SelKind { SEL_NONE, SEL_NODE, SEL_LINK };
  29  enum Mode    { MODE_EDIT, MODE_RUN, MODE_PAUSE };
```

- `PANEL_W = 300` The right-hand panel is 300 pixels wide. Almost every layout number in the program derives from this and the window size.
- `SCENE_FRAC = 0.56` The drawing area takes 56 % of the window height; the plots take the rest.
- `PLOT_SECONDS = 10.0` The time plot always shows the last 10 seconds.
- `SIM_STEP = 0.001` The physics step, 1 ms (Chapter 4).
- `LOG_LIMIT = 36000` At most 36 000 telemetry rows are kept: ten minutes at 60 per second. Old rows are dropped. Without a limit, a simulation left running overnight would eat all the memory.
- **The three enums** (`Tool`, `SelKind`, `Mode`) name the choices the program juggles: which tool is picked, whether a node or a link (or nothing) is selected, and whether we are designing, running or paused. Enums make code read like English (`if (A.mode == MODE_RUN)`) and let the compiler catch typos.

10.2 The Frame: one row of telemetry

APP.HPP (LINES 31 TO 36)

```
  31  // One row of telemetry: everything we measured at one instant.
  32  struct Frame {
  33      double t = 0, T = 0, V = 0, dissipated = 0, workIn = 0;
  34      std::vector<float> nd;   // per node:  x, y, vx, vy
  35      std::vector<float> lk;   // per link:  stretch (spring) or rate (damper), force,
length
  36  };
```

A `Frame` is everything measured at one instant: time, the energies, and two lists. `nd` has **four numbers per node** (x, y, vx, vy) and `lk` has **three per link** (stretch or rate, force, length). They are `float`s, not `double`s, to halve the memory: 36 000 frames × (many numbers) adds up, and plots do not need 15 digits.

10.3 The App struct: all the memory of the program

APP.HPP (LINES 38 TO 70)

```
  38  struct App {
  39      lag::Model model;
  40      lag::State st;
  41      lag::Meter meter;
  42      Mode mode = MODE_EDIT;
  43      double simAcc = 0, speed = 1.0;
  44
  45      Tool tool = TOOL_SELECT;
  46      SelKind selKind = SEL_NONE;
```

```
  47      int sel = -1;                // index of the selected node or link
  48      int linkStart = -1;          // first node clicked while drawing a spring / damper
  49
  50      double ox = 0, oy = 0, scale = 80.0;      // view: screen = (ox, oy) + world *
scale
  51      double mx = 0, my = 0;                    // mouse position in pixels
  52      bool panning = false; double panX = 0, panY = 0;
  53      int dragNode = -1;                        // node being dragged with the mouse
  54      int dragSlider = -1;                      // slider being dragged
  55      std::vector<double> ext;                  // live forces on every node (the mouse
spring)
  56
  57      std::deque<Frame> log;
  58      int plotMode = 0;                         // 0 position, 1 velocity, 2 energy, 3
link force
  59      double plotScale[4] = {1, 1, 1, 1};
  60      double domFreq = 0, domFreqAt = 0;
  61
  62      double fps = 60;
  63      int stepsLastFrame = 0;
  64      std::string toastText; double toastUntil = 0;
  65
  66      // recording (filled in by live.cpp)
  67      bool recording = false;
  68      int recFrames = 0;
  69      std::string recPath;
  70  } A;
```

All of the program's changing information lives in one struct, and exactly one instance of it, named `A`, exists. Read the fields in groups:

- **The model and the physics.** `model` (the design), `st` (the current state), `meter` (energy books), `mode`, `simAcc` (the accumulator from Chapter 4), `speed` (the speed knob: 0.125× to 4×).
- **The selection.** `tool`; `selKind` and `sel` (what is selected, and its index); `linkStart` (the first node clicked while drawing a link, or −1).
- **The view.** `ox, oy, scale`: the three numbers from Chapter 9. `mx, my` is the mouse position. `panning, panX, panY` track a right-button drag.
- **Dragging.** `dragNode` (the node under the mouse, or −1), `dragSlider` (the slider being dragged, or −1), `ext` (a force for every coordinate: the mouse spring writes into it).
- **Telemetry.** `log` (a `std::deque`, a list good at adding to one end and removing from the other), `plotMode`, `plotScale` (smoothed plot ranges), `domFreq`.
- **Housekeeping.** `fps`, `stepsLastFrame`, the `toast` message.
- **Recording.** `recording`, `recFrames`, `recPath`.

> **WHY?**
> **Why one global struct instead of many separate variables?** Because every part of the interface needs to see the same state. A single struct named `A` makes it obvious where state lives (`A.mode`, `A.sel`), makes it easy to print or save the whole thing, and prevents two parts of the program from keeping their own, slightly different, copies. (In larger programs one would pass the struct around explicitly; for a single-window program a global is clear and short.)

10.4 Small helpers: time, clamping, toast

APP.HPP (LINES 75 TO 81)

```
  75  inline double nowSeconds() {
  76      static LARGE_INTEGER f = [] { LARGE_INTEGER x; QueryPerformanceFrequency(&x);
return x; }();
  77      LARGE_INTEGER c; QueryPerformanceCounter(&c);
  78      return (double)c.QuadPart / (double)f.QuadPart;
  79  }
  80  inline double clampd(double x, double lo, double hi) { return x < lo ? lo : (x > hi ?
hi : x); }
  81  inline void toast(const std::string& s) { A.toastText = s; A.toastUntil = nowSeconds()
+ 3.5; }
```

- **`nowSeconds()`** A high-resolution clock. `QueryPerformanceFrequency` asks Windows "how many ticks per second?" once (the `static` variable keeps the answer); `QueryPerformanceCounter` asks "what tick is it now?" The division converts ticks to seconds. It is accurate to well under a microsecond, which matters for smooth animation.
- **`clampd(x, lo, hi)`** Forces a number into a range. Used everywhere to keep values sensible (slider fractions, zoom limits).
- **`toast(s)`** Stores a message and a time **3.5 seconds in the future** at which it should vanish. The renderer shows it while `nowSeconds() < toastUntil`. Notice that `toast` never draws anything: it only changes state, and the artist does the rest.

10.5 Screen layout

APP.HPP (LINES 84 TO 95)

```
  84  inline Gdiplus::RectF sceneRect() { return Gdiplus::RectF(0, 0, (Gdiplus::REAL)(W -
PANEL_W), (Gdiplus::REAL)(H * SCENE_FRAC)); }
  85  inline Gdiplus::RectF timeRect() {
  86      double w = W - PANEL_W, top = H * SCENE_FRAC + 44;
  87      return Gdiplus::RectF(18, (Gdiplus::REAL)top, (Gdiplus::REAL)(w * 0.58 - 26),
(Gdiplus::REAL)(H - top - 16));
  88  }
  89  inline Gdiplus::RectF phaseRect() {
  90      double w = W - PANEL_W, top = H * SCENE_FRAC + 44;
  91      return Gdiplus::RectF((Gdiplus::REAL)(w * 0.58 + 14), (Gdiplus::REAL)top,
(Gdiplus::REAL)(w * 0.42 - 32), (Gdiplus::REAL)(H - top - 16));
  92  }
  93  inline bool inScene(double x, double y) { Gdiplus::RectF r = sceneRect(); return x >=
r.X && x < r.X + r.Width && y >= r.Y && y < r.Y + r.Height; }
  94
  95  inline void resetView() { Gdiplus::RectF r = sceneRect(); A.ox = r.Width / 2; A.oy =
r.Height / 2; A.scale = 80; }
```

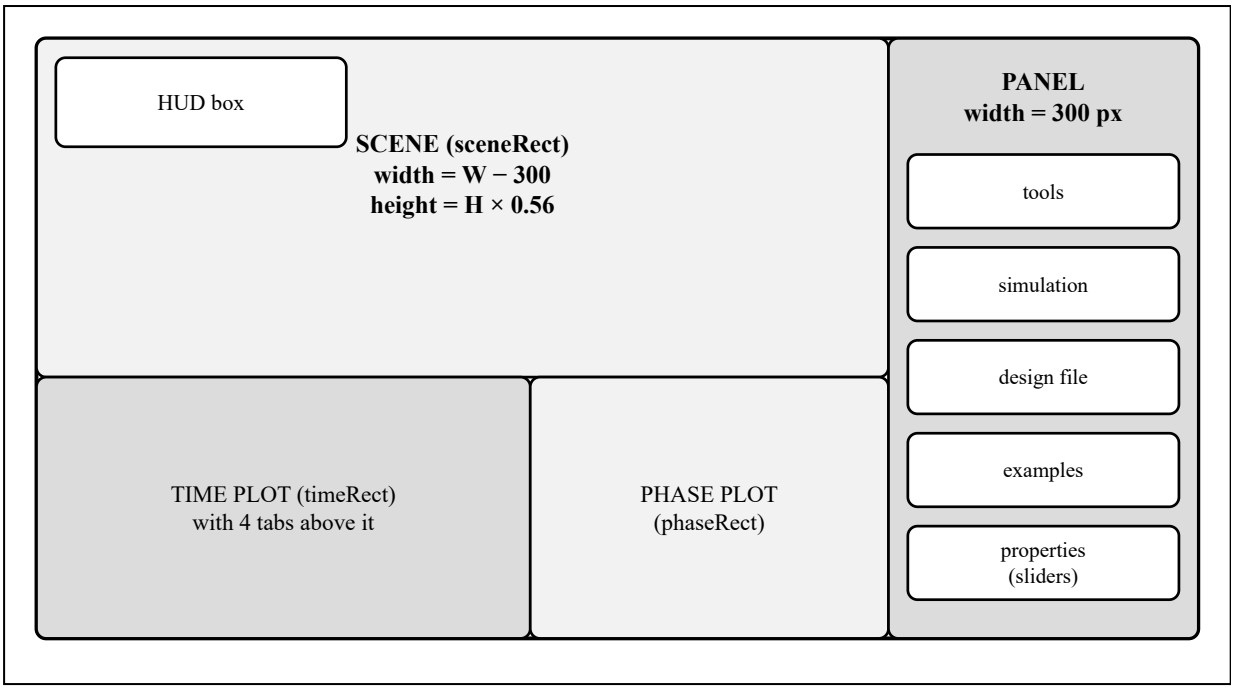


Figure 10.1. *The window is divided into the scene, two plots, and the panel. These four functions compute the rectangles.*

- **`sceneRect()`** Left of the panel and above the plots: width `W - PANEL_W`, height `H * SCENE_FRAC`.
- **`timeRect()`** and **`phaseRect()`** split the lower-left area: the time plot takes 58 % of the width, the phase portrait 42 %. The `top` is 44 pixels below the scene so there is room for the plot tabs.
- **`inScene(x, y)`** Is a point inside the scene? Used to decide whether a click belongs to the drawing.
- **`resetView()`** Puts the world origin in the middle of the scene and sets `scale = 80` pixels per metre.

All layout is computed from `W` and `H` *every time*, never stored. That is why the window can be resized freely: the next frame simply recomputes every rectangle.

10.6 Converting between world and screen

APP.HPP (LINES 97 TO 102)

```
  97  // world (metres) <-> screen (pixels)
  98  inline double toScreenX(double x) { return A.ox + x * A.scale; }
  99  inline double toScreenY(double y) { return A.oy + y * A.scale; }
 100  inline double toWorldX(double sx) { return (sx - A.ox) / A.scale; }
 101  inline double toWorldY(double sy) { return (sy - A.oy) / A.scale; }
 102  inline double snap(double v) { return std::round(v / 0.25) * 0.25; }     // grid of
0.25 m
```

- The four `toScreen` / `toWorld` functions are the equations `sx = ox + x*scale` and `x = (sx - ox)/scale`, one per axis.

- `snap(v)` rounds to the nearest 0.25 m: `std::round(v / 0.25) * 0.25`. When you place or drag something, it lands on a **0.25 m grid**. That keeps designs tidy and makes it easy to build symmetric systems.

APP.HPP (LINES 105 TO 108)

```
 105  inline void nodeWorld(int i, double& x, double& y) {
 106      if (A.mode != MODE_EDIT && 2 * i + 1 < (int)A.st.q.size()) { x = A.st.q[2 * i]; y =
A.st.q[2 * i + 1]; }
 107      else { x = A.model.nodes[i].x; y = A.model.nodes[i].y; }
 108  }
```

- **Where a node is drawn.** While designing (`MODE_EDIT`) it is drawn at its **design** position. While a simulation exists (run or pause) it is drawn at its **live** position from the state. The guard `2*i+1 < (int)A.st.q.size()` protects against asking for the live position of a node that was added after the simulation started.

APP.HPP (LINES 109 TO 112)

```
 109  inline double nodeRadius(int i) {         // on-screen size (pixels), also the click
target size
 110      const lag::Node& n = A.model.nodes[i];
 111      return n.kind == lag::NodeKind::Ground ? 12.0 : 13.0 + 7.0 * std::cbrt(n.m);
 112  }
```

- The on-screen size of a node in pixels: `12` for an anchor, and `13 + 7 * cbrt(m)` for a mass. The **cube root** makes the block's *volume* (not its width) grow with mass: a 8 kg block is twice as wide as a 1 kg block, a natural look. The same number is the click target size.

10.7 logFrame: recording the telemetry

APP.HPP (LINES 115 TO 133)

```
 115  inline void logFrame() {
 116      const lag::Model& M = A.model;
 117      Frame f;
 118      f.t = A.st.t; f.T = A.meter.T; f.V = A.meter.V; f.dissipated = A.meter.dissipated;
f.workIn = A.meter.workIn;
 119      f.nd.resize(4 * M.nodes.size());
 120      for (size_t i = 0; i < M.nodes.size(); ++i) {
 121          f.nd[4 * i]     = (float)A.st.q[2 * i];     f.nd[4 * i + 1] = (float)A.st.q[2 *
i + 1];
 122          f.nd[4 * i + 2] = (float)A.st.v[2 * i];     f.nd[4 * i + 3] = (float)A.st.v[2 *
i + 1];
 123      }
 124      f.lk.resize(3 * M.links.size());
 125      for (size_t li = 0; li < M.links.size(); ++li) {
 126          lag::LinkInfo in = lag::linkInfo(M, A.st.q, A.st.v, (int)li);
 127          f.lk[3 * li]     = (float)(M.links[li].kind == lag::LinkKind::Spring ?
in.stretch : in.rate);
 128          f.lk[3 * li + 1] = (float)in.force;
 129          f.lk[3 * li + 2] = (float)in.len;
 130      }
 131      A.log.push_back(std::move(f));
 132      while (A.log.size() > LOG_LIMIT) A.log.pop_front();
 133  }
```

- Builds one `Frame` from the current state and meter.

- `f.nd.resize(4 * nodes)` and the loop fill x, y, vx, vy for every node (anchors included, so indices match node numbers).
- `f.lk.resize(3 * links)` and the loop call `linkInfo` for each link and store the stretch (for a spring) or the rate (for a damper), the force, and the length.
- `A.log.push_back(std::move(f));` Append it. `std::move` hands over the frame's internal arrays instead of copying them.
- `while (A.log.size() > LOG_LIMIT) A.log.pop_front();` The cap from earlier.

10.8 Controlling the simulation

APP.HPP (LINES 136 TO 146)

```
136  inline void startRun() {
137      if (A.model.massCount() == 0) { toast("Add at least one mass first."); return; }
138      A.model.refreshAutoRest();
139      A.st = lag::initialState(A.model);
140      A.meter.reset(A.model, A.st);
141      A.ext.assign(A.st.q.size(), 0.0);
142      A.log.clear();
143      A.simAcc = 0; A.domFreq = 0;
144      A.mode = MODE_RUN;
145      logFrame();
146  }
```

- If there are no masses, there is nothing to simulate: show a toast and return.
- `A.model.refreshAutoRest();` Make sure automatic springs match the drawing.
- `A.st = lag::initialState(A.model);` Copy the design into a fresh state, with the starting velocities applied (and the rail rule).
- `A.meter.reset(...)` start the energy books. `A.ext.assign(size, 0.0)` zero the mouse forces. `A.log.clear()` forget the last run's telemetry.
- `A.mode = MODE_RUN; logFrame();` Enter run mode and log the very first row, so plots start at time zero.

APP.HPP (LINES 147 TO 151)

```
147  inline void stopRun() {
148      A.mode = MODE_EDIT;
149      A.dragNode = -1;
150      std::fill(A.ext.begin(), A.ext.end(), 0.0);
151  }
```

APP.HPP (LINES 152 TO 155)

```
152  inline void toggleRun() {
153      if (A.mode == MODE_EDIT) startRun();
154      else A.mode = (A.mode == MODE_RUN) ? MODE_PAUSE : MODE_RUN;
155  }
```

- `stopRun` returns to `MODE_EDIT`, cancels any drag, and zeroes the mouse force. The state itself is left alone; because the screen uses the design positions in edit mode, the masses jump back automatically.

- `toggleRun` is the Run button and the space bar: from edit mode it starts a run; from run it pauses; from pause it resumes.

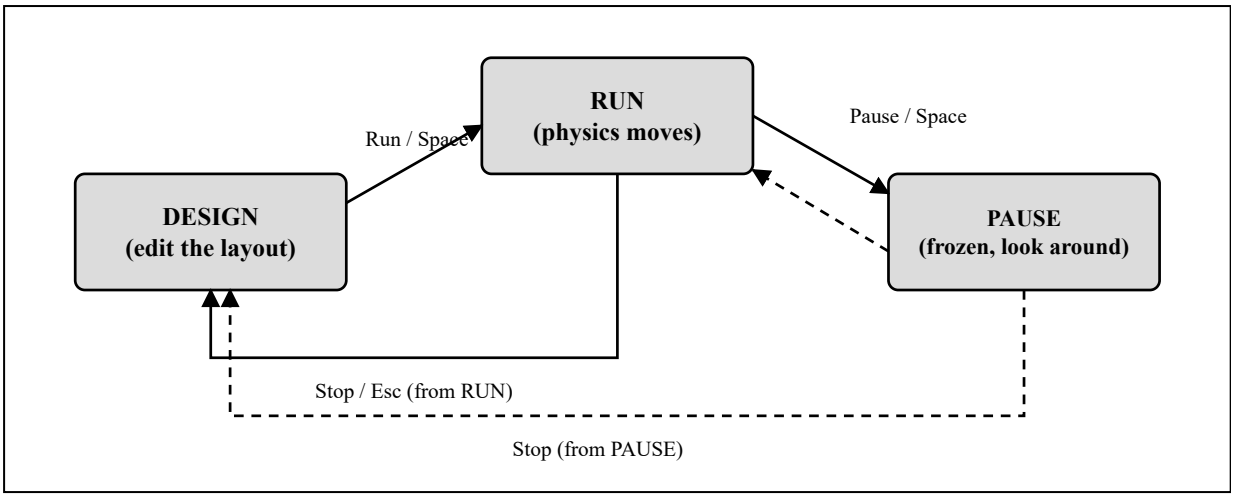


Figure 10.2. *The three modes and what moves between them.*

APP.HPP (LINES 158 TO 168)

```
 158  inline void applyMouseForce() {
 159      std::fill(A.ext.begin(), A.ext.end(), 0.0);
 160      int i = A.dragNode;
 161      if (A.mode != MODE_RUN || i < 0 || i >= (int)A.model.nodes.size() ||
A.model.nodes[i].kind != lag::NodeKind::Mass) return;
 162      double tx = toWorldX(A.mx), ty = toWorldY(A.my);
 163      double dx = tx - A.st.q[2 * i], dy = ty - A.st.q[2 * i + 1], d = std::hypot(dx,
dy);
 164      if (d > 3.0) { dx *= 3.0 / d; dy *= 3.0 / d; }                       // never pull
from further than 3 m away
 165      double m = A.model.nodes[i].m;
 166      A.ext[2 * i]     = m * (700.0 * dx - 45.0 * A.st.v[2 * i]);
 167      A.ext[2 * i + 1] = m * (700.0 * dy - 45.0 * A.st.v[2 * i + 1]);
 168  }
```

This is the "grab a mass with a rubber band" feature.

- `std::fill(A.ext.begin(), A.ext.end(), 0.0);` Clear last step's mouse force.
- The early return: do nothing unless we are running, a node is being dragged, the index is valid, and the node is a **mass** (you cannot drag a wall).
- `double tx = toWorldX(A.mx), ty = toWorldY(A.my);` The cursor in world coordinates: the spring's anchor point.
- `if (d > 3.0) { dx = 3.0/d; dy = 3.0/d; }` Never pull from more than 3 m away. Without this limit, dragging the cursor far from the mass would produce an enormous force (the force is proportional to distance) and fling the system out of control.
- `A.ext[2i] = m * (700.0 * dx - 45.0 * A.st.v[2i]);` The force from Chapter 3 Step 6: a spring (700 per metre of stretch, per kg) and a damper (45 per m/s, per kg). Multiplying by `m` makes the *acceleration* independent of mass, so every block feels equally easy to move. The y component is the same.

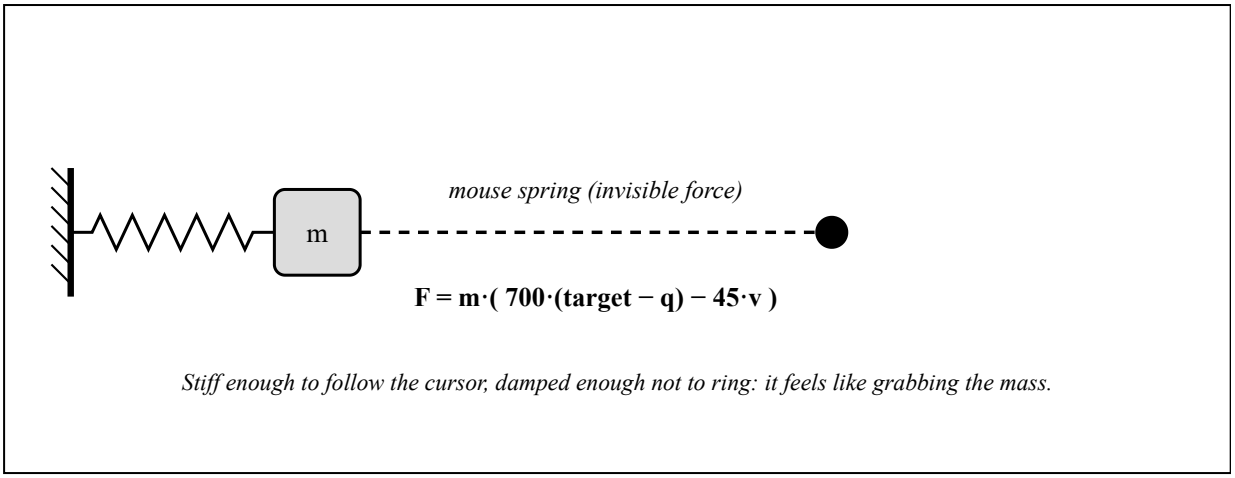


Figure 10.3. *The mouse spring: a stiff, well-damped rubber band between the cursor and the mass.*

WHAT IF IT WERE DIFFERENT?

What if the gains were not damped (45 set to 0)? The mass would oscillate around the cursor forever, like a bob on an elastic string, and never settle where you point. With the damping the mass moves to the cursor in about a tenth of a second and stays.

APP.HPP (LINES 170 TO 185)

```
 170  // Move the physics forward by `dt` seconds of "wall clock" (times the speed knob),
then log one telemetry row.
 171  inline void advanceSim(double dt) {
 172      if (A.mode != MODE_RUN) return;
 173      A.simAcc += dt * A.speed;
 174      int steps = 0;
 175      while (A.simAcc >= SIM_STEP && steps < 800) {
 176          applyMouseForce();
 177          lag::advance(A.model, A.st, A.ext, SIM_STEP, A.meter);
 178          A.simAcc -= SIM_STEP; ++steps;
 179      }
 180      if (steps == 800) A.simAcc = 0;                                        // fell
behind: drop the backlog
 181      A.stepsLastFrame = steps;
 182      for (double v : A.st.q)
 183          if (!std::isfinite(v) || std::fabs(v) > 1e4) { stopRun(); toast("Numbers blew
up (too stiff or too light). Try a heavier mass or a softer spring."); return; }
 184      logFrame();
 185  }
```

The function that moves time forward, called once per frame.

- `if (A.mode != MODE_RUN) return;` Nothing moves in edit or pause.
- `A.simAcc += dt * A.speed;` Add the real elapsed time (scaled by the speed knob) to the accumulator.
- `while (A.simAcc >= SIM_STEP && steps < 800)` Take 1 ms steps while there is enough time banked, but never more than 800 per frame (the spiral-of-death limit, Chapter 4).
- Inside: `applyMouseForce();` then `lag::advance(A.model, A.st, A.ext, SIM_STEP, A.meter);` then subtract one step from the accumulator. The mouse force is recomputed **every step**, because the mass moves between steps.
- `if (steps == 800) A.simAcc = 0;` Drop the backlog if we hit the limit.

- The `for (double v : A.st.q)` loop is a **safety net**: if any coordinate is not a finite number, or is more than 10 000 m from the origin, the simulation has blown up. We stop, and show a toast that suggests a remedy. A program that tells you what went wrong beats one that shows a blank screen.
- `logFrame();` Record one telemetry row for this frame.

10.9 Hit testing: what did the user click?

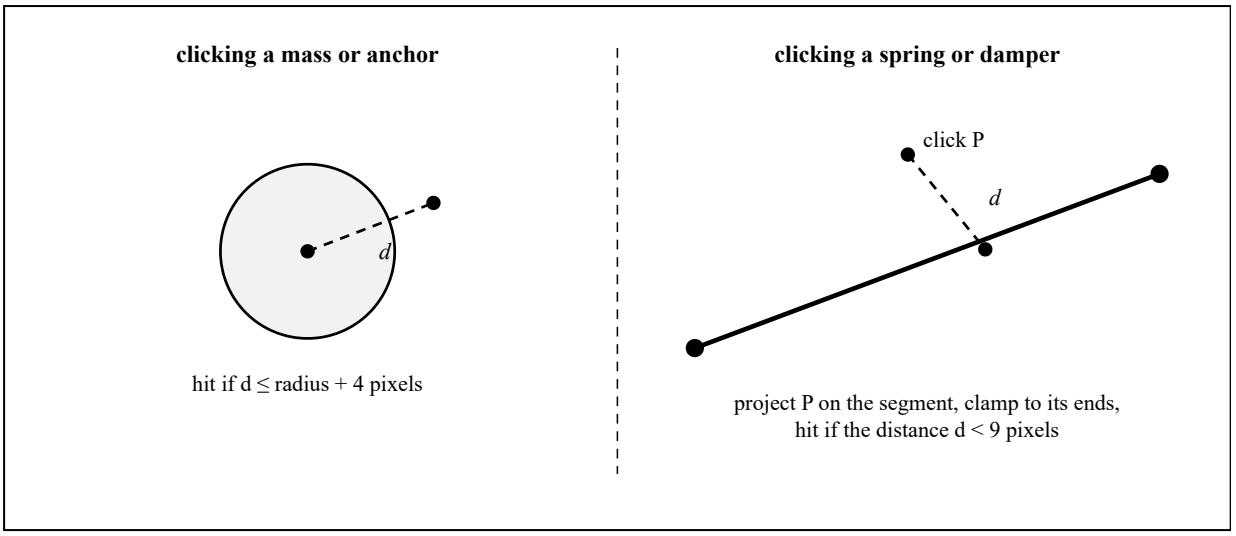


Figure 10.4. *Two tests: a distance to a point for masses and anchors, and a distance to a segment for springs and dampers.*

APP.HPP (LINES 188 TO 195)

```
 188  inline int hitNode(double sx, double sy) {
 189      for (int i = (int)A.model.nodes.size() - 1; i >= 0; --i) {           // topmost
(last drawn) first
 190          double x, y; nodeWorld(i, x, y);
 191          double dx = sx - toScreenX(x), dy = sy - toScreenY(y), r = nodeRadius(i) + 4;
 192          if (dx * dx + dy * dy <= r * r) return i;
 193      }
 194      return -1;
 195  }
```

- The loop runs from the **last** node to the first, so that if two nodes overlap, the one drawn on top (the later one) is chosen.
- `double dx = sx - toScreenX(x), dy = sy - toScreenY(y), r = nodeRadius(i) + 4;` The pixel distance from the click to the node's centre, and the click radius (the size of the node plus 4 pixels of forgiveness).
- `if (dxdx + dydy <= rr) return i;` *Compare the* squares*, which avoids a square root. Returns the node's index, or −1 if nothing was hit.

APP.HPP (LINES 196 TO 200)

```
 196  inline double distToSegment(double px, double py, double ax, double ay, double bx,
double by) {
 197      double dx = bx - ax, dy = by - ay, L2 = dx * dx + dy * dy;
 198      double t = L2 > 1e-9 ? clampd(((px - ax) * dx + (py - ay) * dy) / L2, 0, 1) : 0;
```

```
 199      return std::hypot(px - (ax + t * dx), py - (ay + t * dy));
 200  }
```

The distance from a point P to the line **segment** from A to B.

- `L2 = dx*dx + dy*dy` is the squared length of the segment.
- `t = ((px-ax)*dx + (py-ay)*dy) / L2` is the "how far along from A to B" of P's shadow on the line (a dot product), as a fraction.
- `clampd(t, 0, 1)` keeps the shadow *on the segment*, so clicking beyond an end measures to the end point.
- `std::hypot(px - (ax + t*dx), py - (ay + t*dy))` is the distance from P to that shadow point.
- `L2 > 1e-9 ? ... : 0` avoids dividing by zero for a zero-length segment.

APP.HPP (LINES 201 TO 210)

```
 201  inline int hitLink(double sx, double sy) {
 202      int best = -1; double bd = 9.0;
 203      for (size_t li = 0; li < A.model.links.size(); ++li) {
 204          const lag::Link& L = A.model.links[li];
 205          double ax, ay, bx, by; nodeWorld(L.a, ax, ay); nodeWorld(L.b, bx, by);
 206          double d = distToSegment(sx, sy, toScreenX(ax), toScreenY(ay), toScreenX(bx),
toScreenY(by));
 207          if (d < bd) { bd = d; best = (int)li; }
 208      }
 209      return best;
 210  }
```

- Loops over all links, measures the screen distance from the click to each link's segment, and remembers the nearest one **within 9 pixels** (`bd = 9.0` is both the starting "best distance" and the threshold).

10.10 Editing

APP.HPP (LINES 213 TO 226)

```
 213  inline void clearSelection() { A.selKind = SEL_NONE; A.sel = -1; }
 214  inline bool editingAllowed() {
 215      if (A.mode == MODE_EDIT) return true;
 216      toast("Press Stop to edit the layout. (You can still drag masses and move
sliders.)");
 217      return false;
 218  }
 219  inline void deleteSelection() {
 220      if (!editingAllowed()) return;
 221      if (A.selKind == SEL_NODE) A.model.removeNode(A.sel);
 222      else if (A.selKind == SEL_LINK) A.model.removeLink(A.sel);
 223      clearSelection();
 224      A.linkStart = -1;
 225  }
 226  inline void setTool(Tool t) { A.tool = t; A.linkStart = -1; }
```

- `clearSelection` sets the selection to "nothing".
- **`editingAllowed()`** is the gatekeeper for any change to the layout. If we are not in edit mode, it shows a toast ("Press Stop to edit the layout") and returns false. Every editing

function begins with `if (!editingAllowed()) return;`.

- `deleteSelection` removes the selected node (and, through `removeNode`, its links) or the selected link, then clears the selection and cancels any half-drawn link.
- `setTool` changes the tool and cancels any half-drawn link.

> **WHY?**
> **Why can't you edit the layout while it runs?** Imagine deleting a mass while the integrator is in the middle of a step: the arrays `q` and `v` would no longer match the model, and the next step would read the wrong numbers or run past the end of an array. Disallowing edits while running makes that whole class of bug impossible. Live edits that are *safe* (the sliders) are still allowed.

APP.HPP (LINES 228 TO 253)

```
 228  inline void loadPreset(int n) {
 229      if (A.mode != MODE_EDIT) stopRun();
 230      A.model = lag::presetByNumber(n);
 231      clearSelection(); A.linkStart = -1; A.log.clear();
 232      A.plotMode = 0;
 233      toast("Loaded example " + std::to_string(n));
 234  }
 235  inline std::string designPath() { return std::string(LAB_DIR) + "/design.vlab"; }
 236  inline void saveDesign() {
 237      toast(lag::saveModel(designPath(), A.model) ? "Design saved to design.vlab" :
"Could not save design.vlab");
 238  }
 239  inline void loadDesign() {
 240      lag::Model M;
 241      if (!lag::loadModel(designPath(), M)) { toast("No design.vlab found (press Save
first)"); return; }
 242      if (A.mode != MODE_EDIT) stopRun();
 243      A.model = M; clearSelection(); A.log.clear();
 244      toast("Design loaded");
 245  }
 246  inline void clearDesign() {
 247      if (!editingAllowed()) return;
 248      A.model = lag::Model();
 249      clearSelection(); A.linkStart = -1; A.log.clear();
 250      toast("Empty canvas: pick the Mass and Ground tools to start designing");
 251  }
 252
 253  // ------------------------------------------------------------------ the property
sliders on the right panel
```

- `loadPreset(n)` replaces the whole model with example *n* (stopping any run first), clears selection and telemetry, and shows a toast.
- `designPath()` is `LAB_DIR + "/design.vlab"`: the one design file.
- `saveDesign` and `loadDesign` call the engine-layer functions and report success or failure in a toast. `loadDesign` checks the file loaded into a *temporary* model first.
- `clearDesign` empties the canvas (edit mode only).

10.11 The property sliders

APP.HPP (LINES 254 TO 262)

```
 254  struct Prop {
 255      std::string name;
 256      double* p;                 // points straight into the model, so moving a slider
edits the physics
 257      double lo, hi;
 258      std::string unit;
 259      double step = 0;           // > 0: snap to multiples (used for the rail choice)
 260      const char* const* labels = nullptr;   // text for each snapped value
 261  };
 262  static const char* const RAIL_LABELS[3] = {"free (2-D)", "X rail", "Y rail"};
```

`Prop` describes one slider:

- `name` is its label, `unit` its unit text, `lo` and `hi` its range.
- **`double* p;`** is a **pointer straight into the model**. This is the key idea of the slider system: moving the slider *writes the number directly into the physics*, so there is nothing to "apply" or "update". A slider for a spring's stiffness has `p = &link.k`.
- `step`: if above zero, the value snaps to multiples of it. The rail slider uses `step = 1`, so it can only be 0, 1 or 2.
- `labels`: if not null, the value is shown as a word from this list (`"free (2-D)"`, `"X rail"`, `"Y rail"`).

APP.HPP (LINES 264 TO 290)

```
 264  inline std::vector<Prop> buildProps() {
 265      std::vector<Prop> v;
 266      lag::Model& M = A.model;
 267      if (A.selKind == SEL_NODE && A.sel >= 0 && A.sel < (int)M.nodes.size()) {
 268          lag::Node& n = M.nodes[A.sel];
 269          if (n.kind == lag::NodeKind::Mass) {
 270              v.push_back({"mass m", &n.m, 0.1, 20, "kg"});
 271              v.push_back({"motion", &n.rail, 0, 2, "", 1, RAIL_LABELS});
 272              v.push_back({"start vx", &n.vx0, -10, 10, "m/s"});
 273              v.push_back({"start vy", &n.vy0, -10, 10, "m/s"});
 274              v.push_back({"push force F", &n.driveF, 0, 60, "N"});
 275              v.push_back({"push angle", &n.driveAngle, 0, 360, "deg"});
 276              v.push_back({"push freq", &n.driveW, 0, 30, "rad/s"});
 277          }
 278      } else if (A.selKind == SEL_LINK && A.sel >= 0 && A.sel < (int)M.links.size()) {
 279          lag::Link& l = M.links[A.sel];
 280          if (l.kind == lag::LinkKind::Spring) {
 281              v.push_back({"stiffness k", &l.k, 0, 500, "N/m"});
 282              v.push_back({"rest length", &l.L0, 0.1, 8, "m"});
 283              v.push_back({"cubic k3", &l.k3, 0, 500, "N/m3"});
 284          } else {
 285              v.push_back({"damping c", &l.c, 0, 30, "N s/m"});
 286          }
 287      }
 288      v.push_back({"gravity g", &M.gravity, 0, 20, "m/s2"});
 289      return v;
 290  }
```

- This function builds, **every frame**, the list of sliders appropriate to the current selection. A selected mass gets seven sliders (mass, motion, start vx, start vy, push force, push angle, push frequency); a selected spring gets three (stiffness, rest length, cubic); a damper gets one (damping). Gravity is **always last**, whatever is selected.
- Because the list is rebuilt each frame from the model, it can never go out of date: if you delete the selected item, the next frame's list has no sliders for it.

- The checks like `A.sel >= 0 && A.sel < (int)M.nodes.size()` guard against a stale selection index.

> **WHAT IF IT WERE DIFFERENT?**
>
> **What if `Prop` stored a *copy* of the value instead of a pointer?** Every slider would need a "write back" step, and every property would need its own special-case code. With a pointer, one drag handler serves every slider in the program.

APP.HPP (LINES 291 TO 304)

```
 291  static const int PROPS_Y = 322;     // top of the property section of the panel
 292  inline Gdiplus::RectF propTrack(int i) { return Gdiplus::RectF((Gdiplus::REAL)(W -
PANEL_W + 24), (Gdiplus::REAL)(PROPS_Y + 58 + i * 46), (Gdiplus::REAL)(PANEL_W - 48), 8); }
 293
 294  inline void setPropFromX(int i, double x) {
 295      std::vector<Prop> props = buildProps();
 296      if (i < 0 || i >= (int)props.size()) return;
 297      Prop& p = props[i];
 298      Gdiplus::RectF tk = propTrack(i);
 299      double frac = clampd((x - tk.X) / tk.Width, 0, 1), val = p.lo + frac * (p.hi -
p.lo);
 300      if (p.step > 0) val = std::round(val / p.step) * p.step;
 301      *p.p = val;
 302      if (A.selKind == SEL_LINK && p.p == &A.model.links[A.sel].L0)
A.model.links[A.sel].autoRest = false;   // hand-set: stop following the drawing
 303      if (A.mode != MODE_EDIT) A.meter.rebase(A.model, A.st);                    // a
live edit changes energy for non-physical reasons
 304  }
```

- `propTrack(i)` gives the rectangle of the *i*-th slider's track: its position depends only on the slider number and the window width, so the renderer and the mouse handler agree about where each slider is.
- `setPropFromX` is called while you drag. It converts the mouse's x position to a fraction `(x - tk.X) / tk.Width`, clamped to 0..1, then maps it to the slider's range: `val = lo + frac * (hi - lo)`. If there is a step, it snaps: `std::round(val / step) * step`. Then `*p.p = val;` writes it into the model.
- **`if (A.selKind == SEL_LINK && p.p == &A.model.links[A.sel].L0) ... autoRest = false;`** The pointer trick pays off again: we can *ask whether the slider being moved is a spring's rest length* by comparing pointers. If so, the spring stops following the drawing (Chapter 5).
- **`if (A.mode != MODE_EDIT) A.meter.rebase(...)`** A live edit: restart the energy books (Chapter 7).

10.12 Buttons

APP.HPP (LINES 307 TO 313)

```
 307  enum Action {
 308      ACT_TOOL_SELECT, ACT_TOOL_MASS, ACT_TOOL_GROUND, ACT_TOOL_SPRING, ACT_TOOL_DAMPER,
ACT_DELETE,
 309      ACT_RUN, ACT_STOP, ACT_RECORD, ACT_CSV, ACT_SAVE, ACT_LOAD, ACT_CLEAR,
 310      ACT_PRESET1, ACT_PRESET2, ACT_PRESET3, ACT_PRESET4,
```

```
 311      ACT_TAB0, ACT_TAB1, ACT_TAB2, ACT_TAB3
 312  };
 313  struct Btn { Gdiplus::RectF r; std::string label; Action id; bool active; };
```

- `Action` is a numbered list of everything a button can do: the five tools, delete, run, stop, record, export, save, load, clear, four example buttons and four plot tabs.
- `Btn` is a button: its rectangle, label, action, and whether it is "active" (drawn highlighted).

APP.HPP (LINES 315 TO 342)

```
 315  inline std::vector<Btn> layoutButtons() {
 316      using Gdiplus::RectF;
 317      std::vector<Btn> b;
 318      const float px = (float)(W - PANEL_W);
 319      auto add = [&](float x, float y, float w, float h, const char* label, Action id,
bool active = false) {
 320          b.push_back({RectF(px + x, y, w, h), label, id, active});
 321      };
 322      add(16, 56, 84, 30, "Select",  ACT_TOOL_SELECT, A.tool == TOOL_SELECT);
 323      add(108, 56, 84, 30, "Mass",   ACT_TOOL_MASS,   A.tool == TOOL_MASS);
 324      add(200, 56, 84, 30, "Ground", ACT_TOOL_GROUND, A.tool == TOOL_GROUND);
 325      add(16, 92, 84, 30, "Spring",  ACT_TOOL_SPRING, A.tool == TOOL_SPRING);
 326      add(108, 92, 84, 30, "Damper", ACT_TOOL_DAMPER, A.tool == TOOL_DAMPER);
 327      add(200, 92, 84, 30, "Delete", ACT_DELETE);
 328      add(16, 142, 128, 30, A.mode == MODE_RUN ? "Pause" : (A.mode == MODE_PAUSE ?
"Resume" : "Run"), ACT_RUN, A.mode == MODE_RUN);
 329      add(156, 142, 128, 30, "Stop",  ACT_STOP);
 330      add(16, 178, 128, 30, A.recording ? "Stop REC" : "Record", ACT_RECORD,
A.recording);
 331      add(156, 178, 128, 30, "Export CSV", ACT_CSV);
 332      add(16, 226, 84, 28, "Save",   ACT_SAVE);
 333      add(108, 226, 84, 28, "Load",  ACT_LOAD);
 334      add(200, 226, 84, 28, "Clear", ACT_CLEAR);
 335      for (int i = 0; i < 4; ++i) add(16 + i * 70.0f, 276, 62, 28, std::to_string(i +
1).c_str(), (Action)(ACT_PRESET1 + i));
 336      // plot tabs sit just above the time plot
 337      const char* tabs[4] = {"Position", "Velocity", "Energy", "Link force"};
 338      Gdiplus::RectF tr = timeRect();
 339      for (int i = 0; i < 4; ++i)
 340          b.push_back({RectF(tr.X + i * 92.0f, tr.Y - 30, 86, 24), tabs[i], (Action)
(ACT_TAB0 + i), A.plotMode == i});
 341      return b;
 342  }
```

- Like `buildProps`, this builds the **whole list of buttons every frame**. The positions are fixed numbers relative to the panel's left edge `px = W - PANEL_W`.
- The small lambda `add(x, y, w, h, label, id, active)` is a shortcut for pushing one button.
- Note `A.mode == MODE_RUN ? "Pause" : (A.mode == MODE_PAUSE ? "Resume" : "Run")`: the label of the Run button depends on the mode. Nested conditional operators read like "if running say Pause, else if paused say Resume, else say Run".
- `A.recording ? "Stop REC" : "Record"`: the same idea for Record.
- The four tab buttons are positioned relative to `timeRect()`, so they sit above the time plot.

One list, two users. The renderer loops over this list to *draw* the buttons; the mouse handler loops over the *same list* to find which was clicked. Because both use the same function, the button you see is always exactly the button you click: they cannot drift apart.

APP.HPP (LINES 347 TO 365)

```
 347  inline void doAction(Action id) {
 348      switch (id) {
 349          case ACT_TOOL_SELECT: setTool(TOOL_SELECT); break;
 350          case ACT_TOOL_MASS:   setTool(TOOL_MASS); break;
 351          case ACT_TOOL_GROUND: setTool(TOOL_GROUND); break;
 352          case ACT_TOOL_SPRING: setTool(TOOL_SPRING); break;
 353          case ACT_TOOL_DAMPER: setTool(TOOL_DAMPER); break;
 354          case ACT_DELETE: deleteSelection(); break;
 355          case ACT_RUN:  toggleRun(); break;
 356          case ACT_STOP: stopRun(); break;
 357          case ACT_RECORD: toggleRecording(); break;
 358          case ACT_CSV: exportCsv(); break;
 359          case ACT_SAVE: saveDesign(); break;
 360          case ACT_LOAD: loadDesign(); break;
 361          case ACT_CLEAR: clearDesign(); break;
 362          case ACT_PRESET1: case ACT_PRESET2: case ACT_PRESET3: case ACT_PRESET4:
loadPreset(id - ACT_PRESET1 + 1); break;
 363          case ACT_TAB0: case ACT_TAB1: case ACT_TAB2: case ACT_TAB3: A.plotMode = id -
ACT_TAB0; break;
 364      }
 365  }
```

- A **switch** that turns a button's `Action` into a function call. Note that `ACT_RECORD` and `ACT_CSV` call `toggleRecording()` and `exportCsv()`, which are only *declared* just above (`void toggleRecording();`) and *defined* later in `live.cpp`. This is called a **forward declaration**: it promises the compiler the functions exist so `app.hpp` can call them without including the file that has the video code.

10.13 The mouse

APP.HPP (LINES 368 TO 407)

```
 368  inline void onMouseDown(double x, double y) {
 369      A.mx = x; A.my = y;
 370      for (const Btn& b : layoutButtons())                                    // 1) buttons
 371          if (x >= b.r.X && x <= b.r.X + b.r.Width && y >= b.r.Y && y <= b.r.Y +
b.r.Height) { doAction(b.id); return; }
 372      int n = (int)buildProps().size();                                        // 2)
sliders
 373      for (int i = 0; i < n; ++i) {
 374          Gdiplus::RectF tk = propTrack(i);
 375          if (x >= tk.X - 10 && x <= tk.X + tk.Width + 10 && y >= tk.Y - 16 && y <= tk.Y
+ 22) { A.dragSlider = i; setPropFromX(i, x); return; }
 376      }
 377      if (!inScene(x, y)) return;                                              // 3) the
drawing area
 378      int hn = hitNode(x, y);
 379      switch (A.tool) {
 380          case TOOL_SELECT:
 381              if (hn >= 0) { A.selKind = SEL_NODE; A.sel = hn; A.dragNode = hn; }
 382              else {
 383                  int hl = hitLink(x, y);
 384                  if (hl >= 0) { A.selKind = SEL_LINK; A.sel = hl; } else
clearSelection();
 385              }
 386              break;
 387          case TOOL_MASS: case TOOL_GROUND:
 388              if (!editingAllowed()) break;
 389              if (hn < 0) {
 390                  double wx = snap(toWorldX(x)), wy = snap(toWorldY(y));
 391                  int id = (A.tool == TOOL_MASS) ? A.model.addMass(wx, wy) :
A.model.addGround(wx, wy);
 392                  if (id >= 0) { A.selKind = SEL_NODE; A.sel = id; } else toast("Limit
```

```
reached: 24 nodes");
 393              }
 394              break;
 395          case TOOL_SPRING: case TOOL_DAMPER:
 396              if (!editingAllowed()) break;
 397              if (hn < 0) { A.linkStart = -1; break; }
 398              if (A.linkStart < 0) { A.linkStart = hn; break; }
 399              {
 400                  int li = A.model.addLink(A.tool == TOOL_SPRING ? lag::LinkKind::Spring
: lag::LinkKind::Damper, A.linkStart, hn);
 401                  if (li >= 0) { A.selKind = SEL_LINK; A.sel = li; }
 402                  else toast("Cannot connect those two (same node, two grounds, or
already connected)");
 403                  A.linkStart = -1;
 404              }
 405              break;
 406      }
 407  }
```

The click handler tries three targets in order, and stops at the first that accepts the click.

1. **Buttons.** Loop through `layoutButtons()`; if the click is inside one, run its action and `return`.
2. **Sliders.** Loop through the sliders; if the click is near a track (with 10 pixels of horizontal and −16/+22 pixels of vertical forgiveness, so the thin track is easy to hit), start dragging that slider and set its value.
3. **The drawing area.** First `if (!inScene(x, y)) return;`. Then `hitNode(x, y)` and a `switch` on the tool: * **Select:** a node hit selects it and starts dragging it (`dragNode`). Otherwise try a link. Otherwise clear the selection. * **Mass / Ground:** if editing is allowed and the click is on **empty** space (`hn < 0`), add the node at the snapped position and select it. * **Spring / Damper:** clicking empty space cancels; the first click on a node stores it in `linkStart`; the second click calls `addLink`. If `addLink` refuses (returns −1) a toast explains. Either way `linkStart` is reset.

APP.HPP (LINES 408 TO 418)

```
 408  inline void onMouseMove(double x, double y) {
 409      A.mx = x; A.my = y;
 410      if (A.panning) { A.ox += x - A.panX; A.oy += y - A.panY; A.panX = x; A.panY = y;
return; }
 411      if (A.dragSlider >= 0) { setPropFromX(A.dragSlider, x); return; }
 412      if (A.dragNode >= 0 && A.mode == MODE_EDIT && A.tool == TOOL_SELECT) {          //
move a node (edit mode only)
 413          lag::Node& nd = A.model.nodes[A.dragNode];
 414          nd.x = clampd(snap(toWorldX(x)), -40, 40);
 415          nd.y = clampd(snap(toWorldY(y)), -40, 40);
 416          A.model.refreshAutoRest();
 417      }
 418  }
```

- If a right-button pan is in progress, move the origin by the cursor's movement, and return.
- If a slider is being dragged, update it.
- If a node is being dragged **in edit mode with the select tool**, move it to the snapped mouse position, clamp it to ±40 m (so it cannot be lost off-screen), and call `refreshAutoRest()` so automatic springs stay relaxed.

- In *run* mode dragging does not move the node here at all: `applyMouseForce` reads the mouse position every physics step and the physics moves the node. The same mouse gesture has two meanings depending on the mode.

APP.HPP (LINES 419 TO 419)

```
419  inline void onMouseUp() { A.dragNode = -1; A.dragSlider = -1; }
```

APP.HPP (LINES 420 TO 426)

```
420  inline void onWheel(double x, double y, int notches) {
421      if (!inScene(x, y)) return;
422      double before = toWorldX(x), beforeY = toWorldY(y);
423      A.scale = clampd(A.scale * std::pow(1.12, notches), 25, 260);
424      A.ox = x - before * A.scale;                                    // keep the
point under the cursor fixed
425      A.oy = y - beforeY * A.scale;
426  }
```

- `onMouseUp` ends any drag (a one-line function).
- `onWheel` zooms by a factor of 1.12 per wheel notch, limited to 25 .. 260 pixels per metre. The two lines `A.ox = x - before * A.scale;` keep the world point under the cursor **fixed** while zooming. Without them the picture would zoom toward the world origin and the thing you were looking at would slide away.

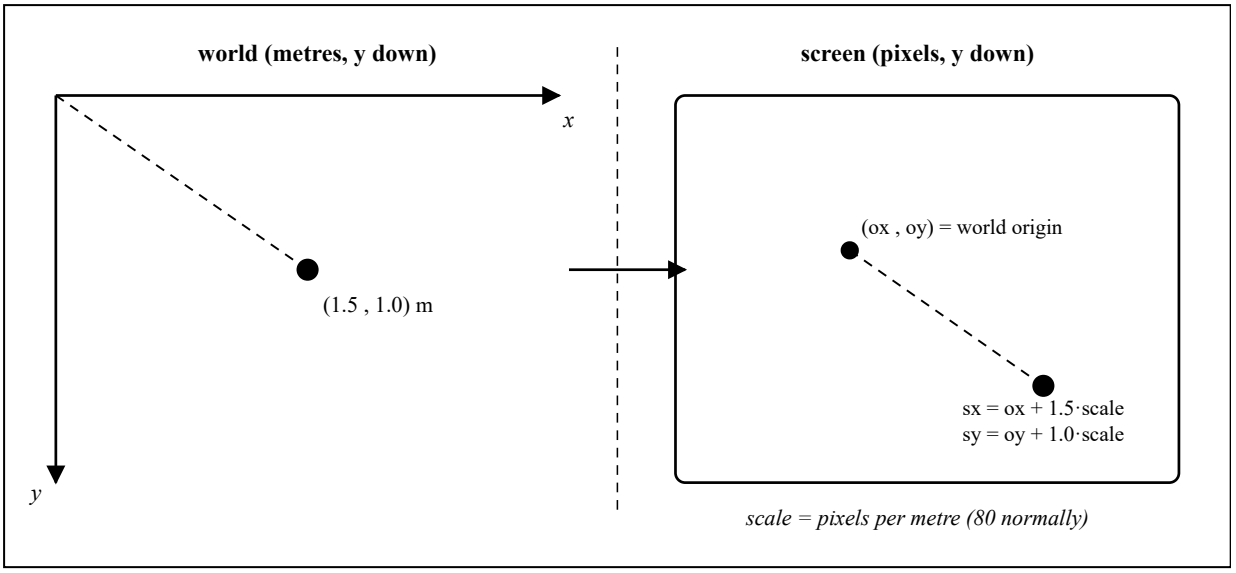


Figure 10.5. *Zooming about the cursor: the world point under the mouse stays where it is.*

10.14 Chapter checks

QUIZ

1. Why is there exactly one `App` object?
2. Why does `buildProps` run every frame instead of once?
3. What is the advantage of `Prop::p` being a pointer?
4. Why does `applyMouseForce` limit the pull to 3 m?

5. In `hitNode`, why compare squared distances?
6. Why is `editingAllowed()` called before every layout change?
7. How does `onWheel` keep the point under the cursor fixed?

ANSWERS

1. All parts of the interface must share the same state; one struct makes that obvious and prevents duplicate, drifting copies.
2. The correct sliders depend on the current selection and on the model, which can change at any moment; rebuilding makes it impossible to show stale sliders.
3. One drag handler can edit any property in the program, and the change goes straight into the physics with nothing to copy back.
4. The force is proportional to distance, so a far cursor would make a huge force and throw the system out of control.
5. It gives the same answer as comparing distances but avoids a slow square root.
6. Changing the layout while the integrator runs would make the model and state arrays disagree, which is a crash or garbage.
7. After changing `scale` it recomputes `ox, oy` so that the world point that was under the cursor maps to the same pixel as before.

CHAPTER 11

The Artist: render.hpp

`render.hpp` turns the state into a picture. It is the largest file, but it is simple in spirit: every function takes the `Graphics` canvas, looks at the state in `A`, and draws. It **never changes the state**.

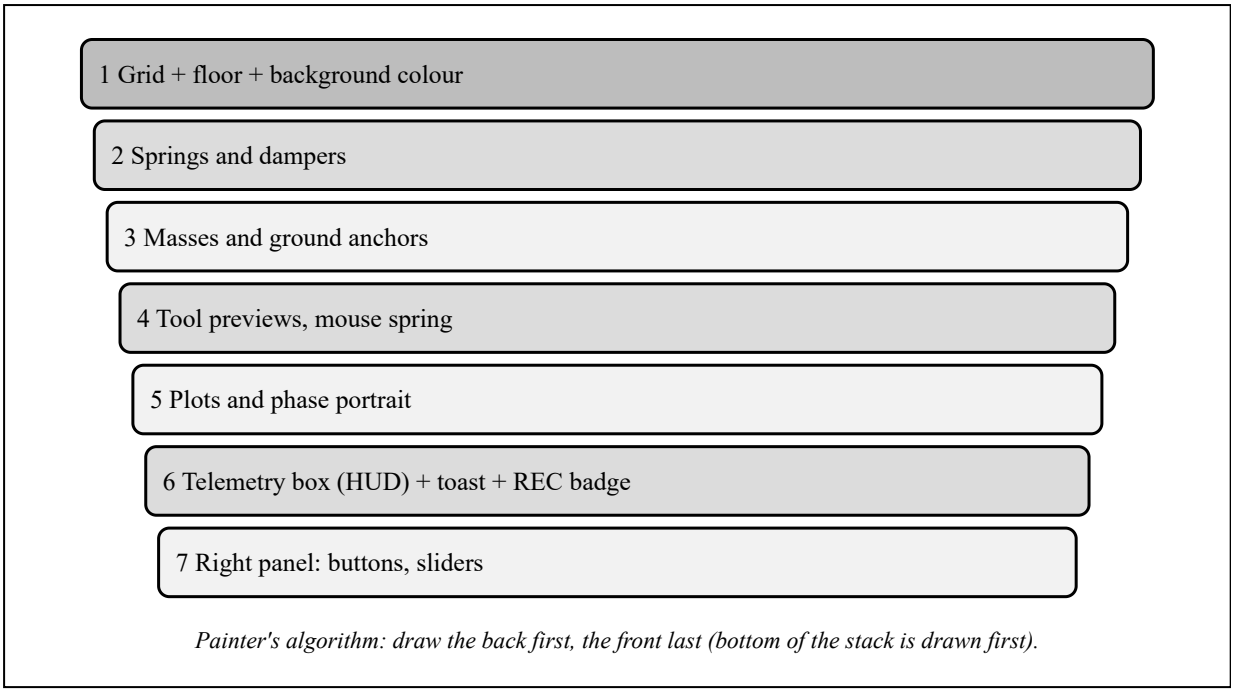


Figure 11.1. *The order of drawing. Each layer is painted over the one before it (the painter's algorithm).*

11.1 The header

RENDER.HPP (LINES 1 TO 7)

```
   1  // render.hpp -- everything that gets PAINTED. Reads the state in app.hpp, never
changes it.
   2  // Included by live.cpp after app.hpp.
   3  #pragma once
   4  #include <algorithm>
   5  #include <cstdarg>
   6
   7  using namespace Gdiplus;
```

- `#include <cstdarg>` brings in `va_list`, needed by `textf` below.
- `using namespace Gdiplus;` lets us write `Pen` and `Graphics` instead of `Gdiplus::Pen` and `Gdiplus::Graphics`.

11.2 Fonts, colours and the first helpers

RENDER.HPP (LINES 10 TO 15)

```
  10  static FontFamily* gMono = nullptr;
  11  static Font *fSmall, *fNorm, *fBig;
  12
  13  static Color C(int r, int g, int b, int a = 255) { return Color((BYTE)a, (BYTE)r,
(BYTE)g, (BYTE)b); }
  14  static const Color MASS_COL[8] = {Color(255, 53, 116, 240), Color(255, 232, 88, 84),
Color(255, 46, 190, 110), Color(255, 235, 170, 20),
  15                                    Color(255, 150, 100, 250), Color(255, 20, 180, 210),
Color(255, 220, 60, 150), Color(255, 160, 160, 170)};
```

- **`gMono, fSmall, fNorm, fBig`** are the program's fonts, created once in `main()` (Consolas, a clean monospaced font, in three sizes). They are pointers because they must be built *after* GDI+ has started.
- **`C(r, g, b, a)`** is a shorthand for building a colour: `C(53, 116, 240)` is a blue. The alpha `a` (opacity) defaults to 255, fully solid.
- **`MASS_COL[8]`** is the palette used for masses: blue, red, green, amber, violet, cyan, pink, grey. The *k*-th mass always gets the same colour in the block, the plot lines and the phase portrait.

RENDER.HPP (LINES 16 TO 20)

```
  16  static Color massColor(int nodeIndex) {          // colour by order among the masses,
so it matches plots and blocks
  17      int k = 0;
  18      for (int i = 0; i < nodeIndex; ++i) k += (A.model.nodes[i].kind ==
lag::NodeKind::Mass);
  19      return MASS_COL[k % 8];
  20  }
```

RENDER.HPP (LINES 21 TO 25)

```
  21  static int massOrdinal(int nodeIndex) {
  22      int k = 0;
  23      for (int i = 0; i < nodeIndex; ++i) k += (A.model.nodes[i].kind ==
lag::NodeKind::Mass);
  24      return k;
  25  }
```

- **A subtle point:** a node's *index* counts anchors too. If the nodes are [anchor, mass, mass], the masses have indices 1 and 2 but are called **m0** and **m1**, and take colours 0 and 1. Both functions count how many **masses** come before the node. `massOrdinal` returns that count (the m-number); `massColor` uses it to pick a palette entry, wrapping around after eight (`k % 8`).

RENDER.HPP (LINES 27 TO 35)

```
  27  static void text(Graphics& g, const char* s, REAL x, REAL y, Font* f, Color c,
StringAlignment al = StringAlignmentNear) {
  28      WCHAR w[256];
  29      MultiByteToWideChar(CP_UTF8, 0, s, -1, w, 256);
  30      SolidBrush b(c);
  31      StringFormat sf;
  32      sf.SetAlignment(al);
  33      sf.SetFormatFlags(StringFormatFlagsNoWrap);
```

```
  34      g.DrawString(w, -1, f, PointF(x, y), &sf, &b);
  35  }
```

- `MultiByteToWideChar` converts our UTF-8 text to the wide characters (UTF-16) that Windows drawing wants. `CP_UTF8` names the encoding; the buffer is 256 wide characters.
- `SolidBrush b(c);` A brush in the requested colour.
- `StringFormat sf; sf.SetAlignment(al);` Controls whether the point we give is the left end of the text (`StringAlignmentNear`), its middle (`Center`) or its right end (`Far`). We use this to centre labels on masses and right-align slider values.
- `StringFormatFlagsNoWrap` keeps each string on one line.
- `g.DrawString(w, -1, f, PointF(x, y), &sf, &b);` Draws it. The `-1` means "the string is null-terminated, work out its length".

RENDER.HPP (LINES 36 TO 42)

```
  36  static void textf(Graphics& g, REAL x, REAL y, Font* f, Color c, const char* fmtStr,
...) {
  37      char buf[200];
  38      va_list ap; va_start(ap, fmtStr);
  39      std::vsnprintf(buf, sizeof buf, fmtStr, ap);
  40      va_end(ap);
  41      text(g, buf, x, y, f, c);
  42  }
```

- A `printf`-style text function: `textf(g, x, y, font, colour, "t = %8.3f s", value)`. The `...` in the parameter list accepts any number of extra arguments; `va_start`, `vsnprintf` and `va_end` are the standard C way to forward them to a formatting function. `vsnprintf` writes at most 200 characters, so a long message can never overflow the buffer.

11.3 Springs and dashpots, drawn flat

RENDER.HPP (LINES 44 TO 55)

```
  44  // A zig-zag spring drawn along the x axis from x1 to x2 at height y (we rotate the
canvas to aim it).
  45  static void coil(Graphics& g, Pen& pen, double x1, double x2, double y, int coils,
double amp) {
  46      std::vector<PointF> p;
  47      double len = x2 - x1, lead = std::fmin(10.0, len * 0.15), span = len - 2 * lead;
  48      p.push_back(PointF((REAL)x1, (REAL)y));
  49      p.push_back(PointF((REAL)(x1 + lead), (REAL)y));
  50      for (int i = 0; i < coils * 2; ++i)
  51          p.push_back(PointF((REAL)(x1 + lead + span * (i + 0.5) / (coils * 2)), (REAL)(y
+ (i % 2 ? amp : -amp))));
  52      p.push_back(PointF((REAL)(x2 - lead), (REAL)y));
  53      p.push_back(PointF((REAL)x2, (REAL)y));
  54      g.DrawLines(&pen, p.data(), (INT)p.size());
  55  }
```

- The spring is a **polyline**: a list of points joined by straight segments. It starts at `x1`, runs a short straight **lead** (at most 10 pixels, at most 15 % of the length) and then zig-zags up and down by `amp` pixels `coils * 2` times, and ends with another straight lead to `x2`.

- `span = len - 2lead` *is the length available for zig-zags. Each zig-zag point is placed at* `x1 + lead + span * (i + 0.5) / (coils 2)`: evenly spaced, with half a gap at each end. The `(i % 2 ? amp : -amp)` alternates the height: up, down, up, down.
- The spring is drawn **horizontal**, along the x axis at height `y`. Turning it to any angle is the next function's trick, shown below.

RENDER.HPP (LINES 56 TO 65)

```
  56  // A dashpot (cylinder + piston) drawn along the x axis.
  57  static void dashpot(Graphics& g, Pen& pen, double x1, double x2, double y) {
  58      double len = x2 - x1, c1 = std::fmax(14.0, len * 0.5), c0 = x1 + 8, c1 =
std::fmin(c0 + c1, x2 - 8);
  59      double p = clampd(x2 - 14, c0 + 6, c1 - 2);
  60      g.DrawLine(&pen, (REAL)x1, (REAL)y, (REAL)c0, (REAL)y);
  61      PointF cyl[4] = {PointF((REAL)c1, (REAL)(y - 8)), PointF((REAL)c0, (REAL)(y - 8)),
PointF((REAL)c0, (REAL)(y + 8)), PointF((REAL)c1, (REAL)(y + 8))};
  62      g.DrawLines(&pen, cyl, 4);
  63      g.DrawLine(&pen, (REAL)x2, (REAL)y, (REAL)p, (REAL)y);
  64      g.DrawLine(&pen, (REAL)p, (REAL)(y - 6), (REAL)p, (REAL)(y + 6));
  65  }
```

- A dashpot (shock-absorber symbol) has three parts: a **cylinder** drawn as an open box (`cyl[4]`: four points forming a "[" shape), a **rod** from the far end into the cylinder, and a **piston** (a short vertical bar on the rod).
- `cl = fmax(14, len*0.5)` is the cylinder length: half the link, at least 14 pixels. `c0, c1` are its two ends.
- `p = clampd(x2 - 14, c0 + 6, c1 - 2)` is the piston's position, **clamped** to stay inside the cylinder. As the link compresses or stretches, the piston slides in and out but cannot leave the cylinder. A visual cue that the damper is working.

RENDER.HPP (LINES 66 TO 72)

```
  66  static void panelFrame(Graphics& g, RectF r, const char* title) {
  67      SolidBrush bg(C(20, 23, 29));
  68      Pen pen(C(52, 58, 70), 1.0f);
  69      g.FillRectangle(&bg, r);
  70      g.DrawRectangle(&pen, r.X, r.Y, r.Width, r.Height);
  71      if (title && *title) text(g, title, r.X, r.Y - 18, fSmall, C(150, 158, 175));
  72  }
```

- Draws a dark rectangle with a thin border, and (if a title is given) the title just above it. The time plot passes an empty title because its tabs already say what it shows.

11.4 drawNodeBody: masses and anchors

RENDER.HPP (LINES 75 TO 113)

```
  75  static void drawNodeBody(Graphics& g, int i) {
  76      const lag::Node& n = A.model.nodes[i];
  77      double wx, wy; nodeWorld(i, wx, wy);
  78      float x = (float)toScreenX(wx), y = (float)toScreenY(wy);
  79      bool selected = (A.selKind == SEL_NODE && A.sel == i);
  80      if (n.kind == lag::NodeKind::Ground) {                        // a fixed anchor: pin
+ hatched base
  81          Pen base(C(220, 224, 232), 2.5f), hatch(C(110, 116, 130), 1.5f);
  82          g.DrawLine(&base, x - 16, y + 8, x + 16, y + 8);
```

```
  83          for (int k = -16; k < 16; k += 6) g.DrawLine(&hatch, x + k + 6, y + 8, x + k, y
+ 16);
  84          SolidBrush pin(selected ? C(90, 205, 255) : C(235, 238, 246));
  85          g.FillEllipse(&pin, x - 6, y - 6, 12.0f, 12.0f);
  86          Pen ring(C(10, 12, 16), 2.0f);
  87          g.DrawEllipse(&ring, x - 6, y - 6, 12.0f, 12.0f);
  88          return;
  89      }
  90      float R = (float)nodeRadius(i);
  91      int rail = (int)std::lround(n.rail);
  92      if (rail != lag::FREE && A.mode == MODE_EDIT) {             // faint rail guide
  93          Pen rp(C(70, 76, 92), 1.0f);
  94          rp.SetDashStyle(DashStyleDash);
  95          if (rail == lag::RAIL_X) g.DrawLine(&rp, x - 90, y, x + 90, y); else
g.DrawLine(&rp, x, y - 90, x, y + 90);
  96      }
  97      if (A.mode != MODE_EDIT) {                                  // a ghost where it
started
  98          Pen gp(C(60, 66, 80), 1.0f);
  99          gp.SetDashStyle(DashStyleDot);
 100          g.DrawRectangle(&gp, (REAL)toScreenX(n.x) - R, (REAL)toScreenY(n.y) - R, 2 * R,
2 * R);
 101      }
 102      RectF r(x - R, y - R, 2 * R, 2 * R);
 103      Color base = massColor(i);
 104      LinearGradientBrush gb(r, Color(255, (BYTE)std::min(255, base.GetR() + 55),
(BYTE)std::min(255, base.GetG() + 55), (BYTE)std::min(255, base.GetB() + 55)), base,
LinearGradientModeVertical);
 105      g.FillRectangle(&gb, r);
 106      Pen outline(selected ? C(255, 255, 255) : C(10, 12, 16), selected ? 3.0f : 2.0f);
 107      g.DrawRectangle(&outline, r);
 108      char b[40];
 109      std::snprintf(b, sizeof b, "m%d", massOrdinal(i));
 110      text(g, b, x, y - 15, fNorm, C(255, 255, 255), StringAlignmentCenter);
 111      std::snprintf(b, sizeof b, "%.3gkg", n.m);
 112      text(g, b, x, y + 1, fSmall, C(255, 255, 255), StringAlignmentCenter);
 113  }
```

- **Setup.** `wx, wy` are the node's world position (design or live, via `nodeWorld`); `x, y` are its pixel position. `selected` is true if this node is the current selection.
- **A ground anchor** is drawn as a horizontal bar with hatch marks beneath (the standard engineering symbol for a fixed support) and a round pin on top. It turns **cyan** when selected. Then `return`: anchors have no more to draw.
- **A mass.** `R = nodeRadius(i)` is half its width. `rail = (int)std::lround(n.rail);` reads the rail setting.
- **The rail guide.** In edit mode, a mass on a rail gets a faint dashed line through it (horizontal for an X rail, vertical for a Y rail), 90 pixels each way, to show where it can go.
- **The ghost.** While a simulation exists, a faint dotted square marks where the mass **started**. It makes it easy to see how far the mass has moved, and is how the sag in example 4 is visible.
- `LinearGradientBrush gb(r, lighter, base, LinearGradientModeVertical);` Fills the block with a gradient from a lighter shade at the top to the base colour at the bottom, which gives a gentle 3-D look. The lighter shade is the base colour with 55 added to each channel, limited to 255 (`std::min(255, ...)`).
- The outline is **white and thick** when selected, dark and thin otherwise.

- The two labels: `"m%d"` (the mass number) above the middle and `"%.3gkg"` (the mass in kg, with at most 3 significant figures) below it.

11.5 drawLink: a spring or damper at any angle

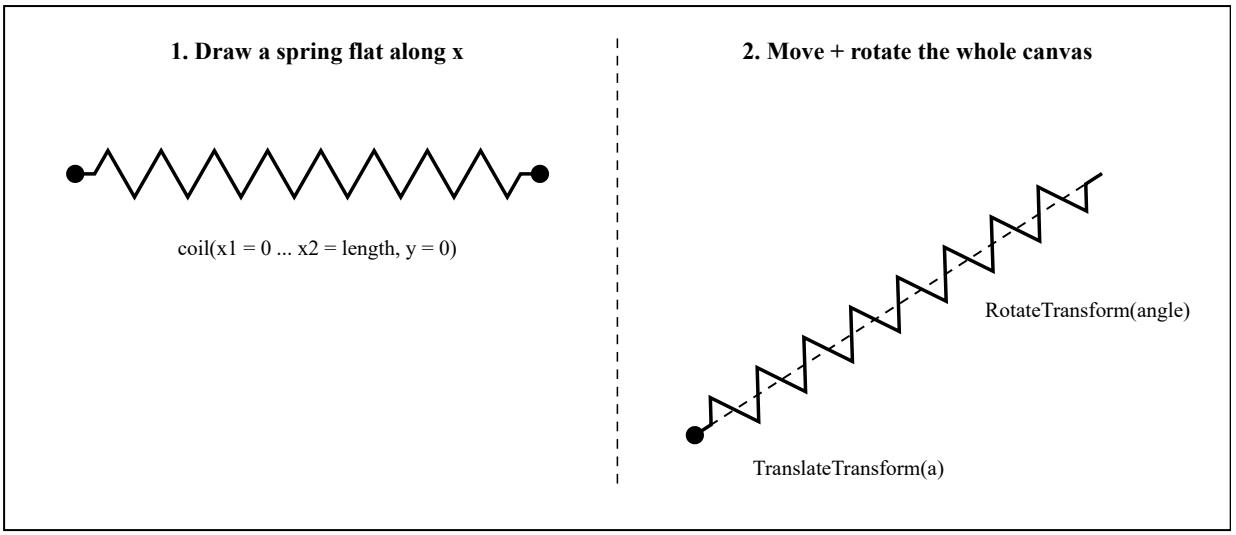


Figure 11.2. *The trick for drawing a spring at any angle: draw it flat, then move and turn the whole canvas.*

RENDER.HPP (LINES 115 TO 153)

```
 115  static void drawLink(Graphics& g, int li, double forceRef) {
 116      const lag::Link& L = A.model.links[li];
 117      double ax, ay, bx, by; nodeWorld(L.a, ax, ay); nodeWorld(L.b, bx, by);
 118      float pax = (float)toScreenX(ax), pay = (float)toScreenY(ay), pbx =
(float)toScreenX(bx), pby = (float)toScreenY(by);
 119      double dx = pbx - pax, dy = pby - pay, len = std::hypot(dx, dy);
 120      if (len < 4) return;
 121      double ux = dx / len, uy = dy / len;
 122      // links that join the same two nodes are fanned out sideways so they do not
overlap
 123      int same = 0, before = 0;
 124      for (int j = 0; j < (int)A.model.links.size(); ++j) {
 125          const lag::Link& o = A.model.links[j];
 126          if ((o.a == L.a && o.b == L.b) || (o.a == L.b && o.b == L.a)) { if (j < li)
++before; ++same; }
 127      }
 128      double off = (before - (same - 1) / 2.0) * 16.0;
 129      double ra = nodeRadius(L.a), rb = nodeRadius(L.b);
 130      bool sel = (A.selKind == SEL_LINK && A.sel == li);
 131
 132      Color col = (L.kind == lag::LinkKind::Spring) ? C(235, 238, 246) : C(150, 158,
175);
 133      if (A.mode != MODE_EDIT && L.kind == lag::LinkKind::Spring) {          // colour
the spring by how hard it pulls (warm) or pushes (cool)
 134          lag::LinkInfo in = lag::linkInfo(A.model, A.st.q, A.st.v, li);
 135          double f = clampd(in.force / forceRef, -1, 1);
 136          col = f >= 0 ? C((int)(235 + 20 * f), (int)(238 - 110 * f), (int)(246 - 190 *
f)) : C((int)(235 + 150 * f), (int)(238 + 20 * f), 246);
 137      }
 138      if (sel) col = C(90, 205, 255);
 139      Pen pen(col, sel ? 2.8f : 2.2f);
 140
 141      GraphicsState state = g.Save();
 142      g.TranslateTransform((REAL)(pax - uy * off), (REAL)(pay + ux * off));      // move
to the (offset) start
 143      g.RotateTransform((REAL)(std::atan2(dy, dx) * 180.0 / lag::PI));            //
point the x axis along the link
```

```
 144      double s0 = std::fmin(ra, len * 0.4), s1 = len - std::fmin(rb, len * 0.4);
 145      if (s1 - s0 < 12) {
 146          g.DrawLine(&pen, (REAL)s0, 0.0f, (REAL)s1, 0.0f);
 147      } else if (L.kind == lag::LinkKind::Spring) {
 148          coil(g, pen, s0, s1, 0, (int)clampd(std::lround((s1 - s0) / 16.0), 3, 22),
8.0);
 149      } else {
 150          dashpot(g, pen, s0, s1, 0);
 151      }
 152      g.Restore(state);
 153  }
```

- **Geometry.** `pax, pay, pbx, pby` are the pixel positions of the two ends. `dx, dy, len` give the on-screen vector and its length; if the link is shorter than 4 pixels we skip it (`return`), because nothing sensible can be drawn. `ux, uy` is the unit vector along it.
- **Fanning out parallel links.** The loop counts how many links join the same two nodes (`same`) and how many of those come before this one (`before`). The sideways offset is `off = (before - (same - 1)/2.0) * 16.0` pixels: for two links it is −8 and +8, so they appear as two parallel lines; for a single link it is 0. That is how a spring and damper between the same two nodes are drawn side by side instead of on top of each other.
- **Colour.** The default is near-white for springs and grey for dampers. **While running**, a spring is colored by how hard it pulls: its force `in.force` is divided by a reference force (`forceRef`) and limited to −1..+1. Positive (stretched, pulling) turns it **orange**; negative (compressed, pushing) turns it **blue**. You can *see* the forces. A selected link is cyan.
- **The rotation trick.** `GraphicsState state = g.Save();` remembers the canvas's current transform. `g.TranslateTransform(pax - uyoff, pay + uxoff);` moves the canvas's origin to the start of the link, shifted sideways by the offset (the perpendicular to **u** is ($-u_y$, u_x)). `g.RotateTransform(atan2(dy, dx) * 180/π);` turns the canvas so its x axis points along the link. Now we can draw the spring **flat**, along x, using the simple `coil` function, and it appears at the right angle on screen. `g.Restore(state);` puts the canvas back so the next drawing is unaffected.
- **Shortening.** `s0 = fmin(ra, len0.4)` *and* `s1 = len - fmin(rb, len0.4)` shrink the drawn part so the spring starts at the edge of the mass blocks, not their centres. The `0.4 * len` limit prevents the ends crossing if two nodes are very close.
- **What to draw.** If the visible part is under 12 pixels, a plain line; for a spring, `coil` with a coil count of `round((s1 - s0)/16)`, limited to 3..22 (longer springs get more coils, like a real spring); for a damper, `dashpot`.

> **WHY?**
> **Why rotate the canvas instead of computing every zig-zag point at an angle?** The rotation takes two lines and works for every shape, now and in the future. Computing the rotated points by hand for the spring and again for the dashpot would be more code, more arithmetic, and more chances for a sign error.

11.6 drawScene: assembling the canvas

RENDER.HPP (LINES 155 TO 203)

```
 155  static void drawScene(Graphics& g) {
 156      RectF sc = sceneRect();
 157      g.SetClip(sc);
 158      // grid: a line every metre, brighter through the origin
 159      Pen grid(C(26, 30, 38), 1.0f), axis(C(44, 50, 62), 1.0f);
 160      double x0 = toWorldX(0), x1 = toWorldX(sc.Width), y0 = toWorldY(0), y1 =
toWorldY(sc.Height);
 161      for (int k = (int)std::floor(x0); k <= (int)std::ceil(x1); ++k) g.DrawLine(k == 0 ?
&axis : &grid, (REAL)toScreenX(k), 0.0f, (REAL)toScreenX(k), sc.Height);
 162      for (int k = (int)std::floor(y0); k <= (int)std::ceil(y1); ++k) g.DrawLine(k == 0 ?
&axis : &grid, 0.0f, (REAL)toScreenY(k), sc.Width, (REAL)toScreenY(k));
 163
 164      if (A.model.nodes.empty() && A.mode == MODE_EDIT) {
 165          text(g, "Empty canvas.  Pick  Ground  or  Mass  on the right, then click
here.", sc.Width / 2, sc.Height / 2 - 20, fNorm, C(150, 158, 175), StringAlignmentCenter);
 166          text(g, "Pick  Spring  or  Damper, then click two things to join them.  Press
Run.", sc.Width / 2, sc.Height / 2 + 4, fSmall, C(110, 118, 135), StringAlignmentCenter);
 167      }
 168      // reference for spring colours: the biggest spring force seen recently (never
below 5 N)
 169      double forceRef = 5;
 170      if (!A.log.empty())
 171          for (size_t li = 0; li < A.model.links.size(); ++li)
 172              if (A.model.links[li].kind == lag::LinkKind::Spring && 3 * li + 1 <
A.log.back().lk.size())
 173                  forceRef = std::fmax(forceRef, std::fabs(A.log.back().lk[3 * li + 1]) *
1.5);
 174      for (int li = 0; li < (int)A.model.links.size(); ++li) drawLink(g, li, forceRef);
 175      for (int i = 0; i < (int)A.model.nodes.size(); ++i) drawNodeBody(g, i);
 176
 177      // tool previews
 178      bool over = inScene(A.mx, A.my);
 179      if (over && A.mode == MODE_EDIT && (A.tool == TOOL_MASS || A.tool == TOOL_GROUND)
&& hitNode(A.mx, A.my) < 0) {
 180          float x = (float)toScreenX(snap(toWorldX(A.mx))), y =
(float)toScreenY(snap(toWorldY(A.my)));
 181          Pen gp(C(90, 205, 255, 160), 1.5f);
 182          gp.SetDashStyle(DashStyleDash);
 183          if (A.tool == TOOL_MASS) g.DrawRectangle(&gp, x - 20, y - 20, 40.0f, 40.0f);
else g.DrawEllipse(&gp, x - 8, y - 8, 16.0f, 16.0f);
 184      }
 185      if (A.linkStart >= 0 && A.linkStart < (int)A.model.nodes.size()) {
 186          double ax, ay; nodeWorld(A.linkStart, ax, ay);
 187          Pen rb(C(90, 205, 255), 1.8f);
 188          rb.SetDashStyle(DashStyleDot);
 189          g.DrawLine(&rb, (REAL)toScreenX(ax), (REAL)toScreenY(ay), (REAL)A.mx,
(REAL)A.my);
 190          text(g, A.tool == TOOL_SPRING ? "click the second end of the spring" : "click
the second end of the damper", (REAL)A.mx + 12, (REAL)A.my - 18, fSmall, C(90, 205, 255));
 191      }
 192      if (A.mode == MODE_RUN && A.dragNode >= 0 && A.model.nodes[A.dragNode].kind ==
lag::NodeKind::Mass) {   // the mouse spring
 193          double x, y; nodeWorld(A.dragNode, x, y);
 194          Pen mp(C(255, 110, 100), 1.6f);
 195          mp.SetDashStyle(DashStyleDot);
 196          g.DrawLine(&mp, (REAL)toScreenX(x), (REAL)toScreenY(y), (REAL)A.mx,
(REAL)A.my);
 197          SolidBrush hb(C(255, 110, 100));
 198          g.FillEllipse(&hb, (REAL)A.mx - 6, (REAL)A.my - 6, 12.0f, 12.0f);
 199      }
 200      g.ResetClip();
 201      Pen divider(C(52, 58, 70), 1.0f);
 202      g.DrawLine(&divider, 0.0f, sc.Height, sc.Width, sc.Height);
 203  }
```

- `g.SetClip(sc);` Restrict all drawing to the scene rectangle; `g.ResetClip();` at the end lifts the restriction. A **clip region** stops a stretched spring or a zoomed-in view from painting over the plots below.
- **The grid.** One vertical and one horizontal line per metre, from `floor(x0)` to `ceil(x1)` (the world limits of the visible area). The line through zero is a slightly brighter pen (`axis`). The grid scrolls and scales with the view, so you always have a ruler.
- **The empty-canvas hint.** If there are no nodes in edit mode, two lines of instructions are drawn in the middle.
- **Spring colour reference.** `forceRef` starts at 5 N and grows to 1.5 times the biggest spring force in the latest telemetry row. This is how the colour scale adapts: a gentle system and a violent one both show the full range of colours.
- **Drawing order.** First every link (`drawLink`), then every node (`drawNodeBody`), so masses cover the ends of springs.
- **Tool previews.** In edit mode with the Mass or Ground tool, over empty space, a dashed cyan outline shows where the new node will land, at the **snapped** position. With a half-drawn link (`linkStart >= 0`) a dotted cyan rubber band is drawn from the first node to the cursor, with a prompt.
- **The mouse spring.** In run mode while dragging a mass, a dotted red line and a red dot show the rubber band.

11.7 drawHud: the telemetry box

RENDER.HPP (LINES 206 TO 270)

```
 206  static void drawHud(Graphics& g) {
 207      const lag::Model& M = A.model;
 208      RectF hud(12, 10, 392, 170);
 209      SolidBrush hudBg(C(8, 10, 14, 215));
 210      Pen hudPen(C(210, 214, 224), 1.2f);
 211      g.FillRectangle(&hudBg, hud);
 212      g.DrawRectangle(&hudPen, hud);
 213      const char* mode = A.mode == MODE_EDIT ? "DESIGN" : (A.mode == MODE_RUN ? "RUNNING"
: "PAUSED");
 214      Color mc = A.mode == MODE_EDIT ? C(255, 214, 120) : (A.mode == MODE_RUN ? C(110,
230, 150) : C(255, 120, 110));
 215      text(g, "VIBRATION LAB  //  TELEMETRY", 22, 15, fNorm, C(240, 242, 248));
 216      text(g, mode, 380, 15, fNorm, mc, StringAlignmentFar);
 217      float y = 38;
 218      const Color dim = C(190, 196, 210), hi = C(120, 215, 255);
 219      textf(g, 22, y, fSmall, dim, "t = %8.3f s     speed %.3gx     FPS %3.0f", A.mode ==
MODE_EDIT ? 0.0 : A.st.t, A.speed, A.fps); y += 16;
 220      if (A.mode != MODE_EDIT) {
 221          textf(g, 22, y, fSmall, dim, "T = %8.3f J   V = %8.3f J", A.meter.T,
A.meter.V); y += 16;
 222          textf(g, 22, y, fSmall, hi,  "E = %8.3f J   dissipated %.3f J", A.meter.E,
A.meter.dissipated); y += 16;
 223          textf(g, 22, y, fSmall, dim, "energy-law error %.1e J   masses %d, links %d",
std::fabs(A.meter.residual()), M.massCount(), (int)M.links.size()); y += 20;
 224      } else {
 225          textf(g, 22, y, fSmall, dim, "%d masses, %d springs/dampers, gravity %.3g",
M.massCount(), (int)M.links.size(), M.gravity); y += 20;
 226      }
```

```
 227      if (A.selKind == SEL_NODE && A.sel >= 0 && A.sel < (int)M.nodes.size() &&
M.nodes[A.sel].kind == lag::NodeKind::Mass) {
 228          int i = A.sel;
 229          textf(g, 22, y, fNorm, massColor(i), "selected m%d  (%.3g kg)", massOrdinal(i),
M.nodes[i].m); y += 18;
 230          if (A.mode != MODE_EDIT && 2 * i + 1 < (int)A.st.q.size()) {
 231              double vx = A.st.v[2 * i], vy = A.st.v[2 * i + 1];
 232              std::vector<double> a;
 233              lag::accel(M, A.st.t, A.st.q, A.st.v, A.ext, a);
 234              textf(g, 22, y, fSmall, dim, "x %+7.3f  y %+7.3f m   |v| %6.3f m/s",
A.st.q[2 * i] - M.nodes[i].x, A.st.q[2 * i + 1] - M.nodes[i].y, std::hypot(vx, vy)); y += 16;
 235              textf(g, 22, y, fSmall, dim, "|a| %6.3f m/s2     dominant freq %s",
std::hypot(a[2 * i], a[2 * i + 1]),
 236                    A.domFreq > 0 ? (std::to_string(A.domFreq).substr(0, 5) + "
Hz").c_str() : "--");
 237          } else {
 238              textf(g, 22, y, fSmall, dim, "press Run to see x, v, a and frequency");
 239          }
 240      } else if (A.selKind == SEL_NODE && A.sel >= 0 && A.sel < (int)M.nodes.size()) {
 241          textf(g, 22, y, fNorm, C(220, 224, 232), "selected ground anchor"); y += 18;
 242          textf(g, 22, y, fSmall, dim, "at (%.2f, %.2f) m  - never moves",
M.nodes[A.sel].x, M.nodes[A.sel].y);
 243      } else if (A.selKind == SEL_LINK && A.sel >= 0 && A.sel < (int)M.links.size()) {
 244          const lag::Link& L = M.links[A.sel];
 245          textf(g, 22, y, fNorm, C(90, 205, 255), "selected %s", L.kind ==
lag::LinkKind::Spring ? "spring" : "damper"); y += 18;
 246          if (A.mode != MODE_EDIT) {
 247              lag::LinkInfo in = lag::linkInfo(M, A.st.q, A.st.v, A.sel);
 248              if (L.kind == lag::LinkKind::Spring) {
 249                  textf(g, 22, y, fSmall, dim, "length %.3f m   stretch %+.3f m", in.len,
in.stretch); y += 16;
 250                  textf(g, 22, y, fSmall, dim, "force %+8.3f N  (+ = pulling)",
in.force);
 251              } else {
 252                  textf(g, 22, y, fSmall, dim, "length %.3f m   rate %+.3f m/s", in.len,
in.rate); y += 16;
 253                  textf(g, 22, y, fSmall, dim, "force %+8.3f N   power lost %.3f W",
in.force, L.c * in.rate * in.rate);
 254              }
 255          } else {
 256              textf(g, 22, y, fSmall, dim, "drawn length %.3f m", M.distance(L.a, L.b));
 257          }
 258      } else {
 259          text(g, "click something to see its numbers", 22, y, fSmall, C(120, 128, 145));
 260      }
 261      // recording badge, top-right of the drawing area
 262      RectF sc = sceneRect();
 263      if (A.recording) {
 264          SolidBrush red(C(235, 60, 60));
 265          g.FillEllipse(&red, sc.Width - 140, 16.0f, 14.0f, 14.0f);
 266          textf(g, sc.Width - 120, 14, fNorm, C(255, 130, 120), "REC %02d:%02d",
(A.recFrames / 60) / 60, (A.recFrames / 60) % 60);
 267      }
 268      text(g, "wheel = zoom   right-drag = pan   Space = run   Esc = stop", 14, sc.Height
- 20, fSmall, C(110, 118, 135));
 269      if (A.toastUntil > nowSeconds()) text(g, A.toastText.c_str(), sc.Width / 2,
sc.Height - 48, fNorm, C(110, 230, 150), StringAlignmentCenter);
 270  }
```

- The box is a semi-transparent black rectangle (`C(8, 10, 14, 215)`: alpha 215 of 255) with a white border, in the top-left of the scene.
- The **title line** and the **mode word** (DESIGN in amber, RUNNING in green, PAUSED in red).
- **Global numbers.** Time, speed and FPS. While a simulation exists: T, V, E = T + V and the dissipated energy, and the **energy-law error** `std::fabs(A.meter.residual())` with the

count of masses and links. In design mode: a count of masses and springs, and the gravity.

- **The selection section.** What it shows depends on what is selected:
- **A mass:** its name and colour, then (if a simulation exists) its displacement from the start (`q - design position`), its speed |**v**|, its acceleration |**a**| (computed by calling the physics function `lag::accel` once more, just for the readout) and the dominant frequency.
- **A ground anchor:** its position and the note that it never moves.
- **A spring or damper:** length and stretch (spring) or rate (damper), force, and for a damper the **power lost** `c ṡ²` in watts, which is how much heat it is making at this instant.
- **Nothing selected:** a hint.
- **The REC badge.** While recording: a red dot and the elapsed time `mm:ss`, computed from the frame count at 60 frames per second.
- **The toast** appears centred near the bottom of the scene until its expiry time.

> **CHECKPOINT**
> Run example 4, select the damper, and read "power lost". It is zero when the damper is not moving (rate = 0) and greatest when the suspension is moving fastest: a damper makes heat only when it moves.

11.8 Plots

RENDER.HPP (LINES 273 TO 273)

```
 273  struct Series { std::string name; Color col; std::vector<double> v; };
```

- A `Series` is one curve: a name for the legend, a colour, and its list of values. All curves of a plot share one list of times.

RENDER.HPP (LINES 275 TO 319)

```
 275  // Draw several curves against time (the last PLOT_SECONDS seconds of the log).
 276  static void drawTimePlot(Graphics& g, RectF r, const std::vector<Series>& series, const
std::vector<double>& times, int modeIndex, bool symmetric) {
 277      g.SetClip(r);
 278      double lo = 1e300, hi = -1e300;
 279      for (const Series& s : series) for (double v : s.v) { lo = std::fmin(lo, v); hi =
std::fmax(hi, v); }
 280      if (lo > hi) { lo = -1; hi = 1; }
 281      if (symmetric) { double m = std::fmax(std::fabs(lo), std::fabs(hi)); lo = -m; hi =
m; }
 282      if (hi - lo < 1e-6) { hi += 0.5; lo -= 0.5; }
 283      double pad = (hi - lo) * 0.1;
 284      lo -= pad; hi += pad;
 285      // smooth the vertical range so the picture does not jitter
 286      double& sm = A.plotScale[modeIndex];
 287      if (sm <= 1.0001 && sm >= 0.9999) sm = hi - lo;
 288      sm += ((hi - lo) - sm) * 0.2;
 289      double mid = (hi + lo) / 2;
 290      lo = mid - sm / 2; hi = mid + sm / 2;
 291      double t1 = times.empty() ? 1.0 : times.back(), t0 = t1 - PLOT_SECONDS;
 292      Pen gridPen(C(34, 39, 48), 1.0f);
 293      for (int k = 0; k <= 4; ++k) {
 294          double v = lo + (hi - lo) * k / 4;
 295          float yy = (float)(r.Y + r.Height * (1 - (v - lo) / (hi - lo)));
```

```
 296          g.DrawLine(&gridPen, r.X, yy, r.X + r.Width, yy);
 297          textf(g, r.X + 4, yy - 14, fSmall, C(120, 128, 145), "%.3g", v);
 298      }
 299      if (lo < 0 && hi > 0) {
 300          Pen zero(C(70, 76, 92), 1.0f);
 301          float yy = (float)(r.Y + r.Height * (1 - (0 - lo) / (hi - lo)));
 302          g.DrawLine(&zero, r.X, yy, r.X + r.Width, yy);
 303      }
 304      for (const Series& s : series) {
 305          std::vector<PointF> p;
 306          for (size_t i = 0; i < s.v.size() && i < times.size(); ++i)
 307              p.push_back(PointF((REAL)(r.X + (times[i] - t0) / PLOT_SECONDS * r.Width),
(REAL)(r.Y + r.Height * (1 - (s.v[i] - lo) / (hi - lo)))));
 308          Pen lp(s.col, 1.8f);
 309          if (p.size() > 1) g.DrawLines(&lp, p.data(), (INT)p.size());
 310      }
 311      float lx = r.X + 60;
 312      for (const Series& s : series) {                    // legend
 313          SolidBrush b(s.col);
 314          g.FillRectangle(&b, lx, r.Y + 7, 10.0f, 10.0f);
 315          text(g, s.name.c_str(), lx + 14, r.Y + 3, fSmall, C(190, 196, 210));
 316          lx += 24 + 8.0f * (float)s.name.size();
 317      }
 318      g.ResetClip();
 319  }
```

The generic time plot. It draws any set of curves, so the Position, Velocity, Energy and Link-force tabs all use it.

- **Finding the range.** It scans every value for the smallest and largest. If `symmetric` is true (position, velocity, force) the range is made symmetric about zero, so zero sits in the middle and positive and negative look equal. Energy is not symmetric. A tiny range is widened so a flat line does not crash the scaling (`hi - lo < 1e-6`).
- **Padding.** The range is widened 10 % so curves do not touch the frame.
- **Smoothing.** `sm` remembers the previous range for each tab (`A.plotScale[modeIndex]`) and moves 20 % of the way to the new one each frame (`sm += (target - sm) * 0.2`). Without this, the whole plot would jump whenever a new peak appeared. With it the axis glides.
- **The grid and labels.** Five horizontal lines with numeric labels formatted `%.3g` (three significant figures); a zero line if zero is in range.
- **The curves.** For each series, every sample becomes a point: x from its time (`(times[i] - t0) / PLOT_SECONDS * r.Width`, where `t0 = t1 - 10`, so the plot is a 10-second sliding window) and y from its value scaled to the rectangle. `g.DrawLines` joins them.
- **The legend.** A small coloured square and the series name for each curve, moving right by an amount based on the name length.

RENDER.HPP (LINES 321 TO 421)

```
 321  static void drawPlots(Graphics& g) {
 322      const lag::Model& M = A.model;
 323      RectF tr = timeRect(), pr = phaseRect();
 324      panelFrame(g, tr, "");      // the tab buttons above the plot already say what it
shows
 325      text(g, "last 10 s", tr.X + tr.Width - 8, tr.Y + tr.Height - 18, fSmall, C(110,
118, 135), StringAlignmentFar);
 326
```

```
 327      // collect the frames of the last PLOT_SECONDS
 328      std::vector<const Frame*> fr;
 329      if (!A.log.empty()) {
 330          double tEnd = A.log.back().t;
 331          for (auto it = A.log.rbegin(); it != A.log.rend() && it->t >= tEnd -
PLOT_SECONDS; ++it) fr.push_back(&*it);
 332          std::reverse(fr.begin(), fr.end());
 333      }
 334      std::vector<double> times;
 335      for (const Frame* f : fr) times.push_back(f->t);
 336      std::vector<Series> series;
 337      if (A.plotMode == 0 || A.plotMode == 1) {
 338          for (int i = 0; i < (int)M.nodes.size(); ++i) {
 339              if (M.nodes[i].kind != lag::NodeKind::Mass) continue;
 340              int rail = (int)std::lround(M.nodes[i].rail);
 341              Color c = massColor(i);
 342              for (int comp = 0; comp < 2; ++comp) {
 343                  if ((comp == 0 && rail == lag::RAIL_Y) || (comp == 1 && rail ==
lag::RAIL_X)) continue;   // that axis cannot move
 344                  Series s;
 345                  char nm[24];
 346                  std::snprintf(nm, sizeof nm, "m%d.%c", massOrdinal(i), comp == 0 ? 'x'
: 'y');
 347                  s.name = nm;
 348                  s.col = comp == 0 ? c : Color(255, (BYTE)(c.GetR() * 0.6), (BYTE)
(c.GetG() * 0.6), (BYTE)(c.GetB() * 0.6));
 349                  for (const Frame* f : fr) {
 350                      size_t b = 4 * (size_t)i;
 351                      if (b + 3 >= f->nd.size()) { s.v.push_back(0); continue; }
 352                      s.v.push_back(A.plotMode == 0 ? (f->nd[b + comp] - (comp == 0 ?
M.nodes[i].x : M.nodes[i].y)) : f->nd[b + 2 + comp]);
 353                  }
 354                  series.push_back(s);
 355              }
 356          }
 357          drawTimePlot(g, tr, series, times, A.plotMode, true);
 358      } else if (A.plotMode == 2) {
 359          Series T{"T kinetic", C(232, 88, 84), {}}, V{"V potential", C(53, 116, 240),
{}}, E{"E = T+V", C(235, 238, 246), {}}, D{"dissipated", C(235, 170, 20), {}};
 360          for (const Frame* f : fr) { T.v.push_back(f->T); V.v.push_back(f->V);
E.v.push_back(f->T + f->V); D.v.push_back(f->dissipated); }
 361          series = {T, V, E, D};
 362          drawTimePlot(g, tr, series, times, 2, false);
 363      } else {
 364          if (A.selKind == SEL_LINK && A.sel >= 0 && A.sel < (int)M.links.size()) {
 365              size_t li = (size_t)A.sel;
 366              Series F{M.links[li].kind == lag::LinkKind::Spring ? "spring force N" :
"damper force N", C(90, 205, 255), {}};
 367              Series S{M.links[li].kind == lag::LinkKind::Spring ? "stretch m" : "rate
m/s", C(235, 170, 20), {}};
 368              for (const Frame* f : fr) {
 369                  if (3 * li + 2 >= f->lk.size()) { F.v.push_back(0); S.v.push_back(0);
continue; }
 370                  F.v.push_back(f->lk[3 * li + 1]); S.v.push_back(f->lk[3 * li]);
 371              }
 372              series = {F, S};
 373              drawTimePlot(g, tr, series, times, 3, true);
 374          } else {
 375              drawTimePlot(g, tr, series, times, 3, true);
 376              text(g, "select a spring or damper (Select tool) to plot its force", tr.X +
tr.Width / 2, tr.Y + tr.Height / 2 - 8, fSmall, C(150, 158, 175), StringAlignmentCenter);
 377          }
 378      }
 379
 380      // phase portrait of the selected mass (or the first mass)
 381      int pm = -1;
 382      if (A.selKind == SEL_NODE && A.sel >= 0 && A.sel < (int)M.nodes.size() &&
M.nodes[A.sel].kind == lag::NodeKind::Mass) pm = A.sel;
 383      else for (int i = 0; i < (int)M.nodes.size(); ++i) if (M.nodes[i].kind ==
lag::NodeKind::Mass) { pm = i; break; }
 384      char title[96];
 385      if (pm >= 0) std::snprintf(title, sizeof title, "PHASE PORTRAIT of m%d  (velocity
```

```
vs position)", massOrdinal(pm));
 386      else std::snprintf(title, sizeof title, "PHASE PORTRAIT");
 387      panelFrame(g, pr, title);
 388      if (pm >= 0 && A.log.size() > 2) {
 389          int comp = ((int)std::lround(M.nodes[pm].rail) == lag::RAIL_Y) ? 1 : 0;
 390          size_t n = std::min<size_t>(A.log.size(), 700), start = A.log.size() - n;
 391          double xm = 0.3, vm = 0.3;
 392          for (size_t i = start; i < A.log.size(); ++i) {
 393              const Frame& f = A.log[i];
 394              size_t b = 4 * (size_t)pm;
 395              if (b + 3 >= f.nd.size()) continue;
 396              xm = std::fmax(xm, std::fabs(f.nd[b + comp] - (comp == 0 ? M.nodes[pm].x :
M.nodes[pm].y)));
 397              vm = std::fmax(vm, std::fabs(f.nd[b + 2 + comp]));
 398          }
 399          xm *= 1.15; vm *= 1.15;
 400          g.SetClip(pr);
 401          Pen axis(C(52, 58, 70), 1.0f);
 402          g.DrawLine(&axis, pr.X, pr.Y + pr.Height / 2, pr.X + pr.Width, pr.Y + pr.Height
/ 2);
 403          g.DrawLine(&axis, pr.X + pr.Width / 2, pr.Y, pr.X + pr.Width / 2, pr.Y +
pr.Height);
 404          Color mc = massColor(pm);
 405          PointF prev; bool havePrev = false;
 406          for (size_t i = start; i < A.log.size(); ++i) {
 407              const Frame& f = A.log[i];
 408              size_t b = 4 * (size_t)pm;
 409              if (b + 3 >= f.nd.size()) continue;
 410              PointF p((REAL)(pr.X + pr.Width / 2 + (f.nd[b + comp] - (comp == 0 ?
M.nodes[pm].x : M.nodes[pm].y)) / xm * pr.Width / 2),
 411                       (REAL)(pr.Y + pr.Height / 2 - f.nd[b + 2 + comp] / vm * pr.Height
/ 2));
 412              if (havePrev) {
 413                  Pen tp(Color((BYTE)(30 + 225 * (i - start) / n), mc.GetR(), mc.GetG(),
mc.GetB()), 1.8f);
 414                  g.DrawLine(&tp, prev, p);
 415              }
 416              prev = p; havePrev = true;
 417          }
 418          if (havePrev) { SolidBrush dot(C(255, 255, 255)); g.FillEllipse(&dot, prev.X -
4, prev.Y - 4, 8.0f, 8.0f); }
 419          g.ResetClip();
 420      }
 421  }
```

- **Collecting the frames.** It walks the telemetry log **backwards** from the newest row while the rows are within the last 10 seconds, then reverses the list (`std::reverse`) so it is oldest first. It builds the list of times.
- **Tabs 0 and 1 (Position and Velocity).** For each mass it makes up to **two** series, x and y. A component is *skipped* if the mass's rail forbids it (an X-rail mass has no y curve), so a rail mass shows a single clean curve. The y curve is drawn in a darker shade of the same colour (`c * 0.6`). Position is plotted **relative to the design position** so every curve starts near zero.
- **Tab 2 (Energy).** Four series: kinetic (red), potential (blue), total E = T + V (white), and the running total of dissipated energy (amber). You can watch energy trade between T and V and drain into heat.
- **Tab 3 (Link force).** If a spring or damper is selected: its force (cyan) and its stretch or rate (amber). If not, an instruction is shown on the empty plot.

- **The phase portrait.** It picks the selected mass, or the first mass if none is selected, and plots **velocity against position** for the last 700 rows. For a mass on a Y rail it uses y; otherwise x. The oldest points are drawn **fainter** (alpha grows from 30 to 255 with age), leaving a fading trail, and the newest point gets a white dot.

> **KEY WORD**
>
> **Phase portrait.** A plot of velocity (vertical) against position (horizontal). A frictionless oscillator traces a closed loop (an ellipse); a damped one spirals inward toward the middle; a driven one settles onto a loop. Shape tells you the *kind* of motion at a glance.

11.9 The right-hand panel

RENDER.HPP (LINES 424 TO 473)

```
 424  static void drawPanel(Graphics& g) {
 425      REAL px = (REAL)(W - PANEL_W);
 426      SolidBrush pbg(C(22, 25, 31));
 427      g.FillRectangle(&pbg, px, 0.0f, (REAL)PANEL_W, (REAL)H);
 428      Pen divider(C(52, 58, 70), 1.0f);
 429      g.DrawLine(&divider, px, 0.0f, px, (REAL)H);
 430      text(g, "DESIGNER", px + 16, 14, fBig, C(235, 238, 246));
 431      text(g, "TOOLS", px + 16, 40, fSmall, C(120, 128, 145));
 432      text(g, "SIMULATION", px + 16, 126, fSmall, C(120, 128, 145));
 433      text(g, "DESIGN FILE", px + 16, 210, fSmall, C(120, 128, 145));
 434      text(g, "EXAMPLES", px + 16, 260, fSmall, C(120, 128, 145));
 435
 436      for (const Btn& b : layoutButtons()) {
 437          bool hover = A.mx >= b.r.X && A.mx <= b.r.X + b.r.Width && A.my >= b.r.Y &&
A.my <= b.r.Y + b.r.Height;
 438          bool rec = (b.id == ACT_RECORD && A.recording);
 439          Color fill = rec ? C(170, 50, 50) : (b.active ? C(53, 116, 240) : (hover ?
C(52, 58, 72) : C(38, 43, 54)));
 440          SolidBrush bf(fill);
 441          Pen bp(b.active || rec ? C(120, 170, 255) : C(60, 66, 80), 1.0f);
 442          g.FillRectangle(&bf, b.r);
 443          g.DrawRectangle(&bp, b.r);
 444          text(g, b.label.c_str(), b.r.X + b.r.Width / 2, b.r.Y + b.r.Height / 2 - 8,
fSmall, C(240, 242, 248), StringAlignmentCenter);
 445      }
 446
 447      // selection title + sliders
 448      text(g, "PROPERTIES", px + 16, PROPS_Y - 4, fSmall, C(120, 128, 145));
 449      char s[120];
 450      const lag::Model& M = A.model;
 451      if (A.selKind == SEL_NODE && A.sel >= 0 && A.sel < (int)M.nodes.size())
 452          std::snprintf(s, sizeof s, M.nodes[A.sel].kind == lag::NodeKind::Mass ? "mass
m%d" : "ground anchor", massOrdinal(A.sel));
 453      else if (A.selKind == SEL_LINK && A.sel >= 0 && A.sel < (int)M.links.size())
 454          std::snprintf(s, sizeof s, "%s", M.links[A.sel].kind == lag::LinkKind::Spring ?
"spring" : "damper");
 455      else std::snprintf(s, sizeof s, "nothing selected (world settings)");
 456      text(g, s, px + 16, PROPS_Y + 14, fNorm, C(215, 220, 232));
 457      std::vector<Prop> props = buildProps();
 458      for (int i = 0; i < (int)props.size(); ++i) {
 459          const Prop& p = props[i];
 460          RectF tk = propTrack(i);
 461          double val = *p.p, frac = clampd((val - p.lo) / (p.hi - p.lo), 0, 1);
 462          text(g, p.name.c_str(), tk.X, tk.Y - 22, fSmall, C(200, 206, 220));
 463          if (p.labels) std::snprintf(s, sizeof s, "%s",
p.labels[(int)clampd(std::lround(val), 0, 2)]);
 464          else std::snprintf(s, sizeof s, "%.3g %s", val, p.unit.c_str());
 465          text(g, s, tk.X + tk.Width, tk.Y - 22, fSmall, C(120, 215, 255),
```

```
StringAlignmentFar);
 466          SolidBrush track(C(48, 54, 66)), fill(C(60, 140, 240)), knob(C(245, 247, 252));
 467          g.FillRectangle(&track, tk);
 468          g.FillRectangle(&fill, tk.X, tk.Y, (REAL)(tk.Width * frac), tk.Height);
 469          g.FillEllipse(&knob, (REAL)(tk.X + tk.Width * frac - 7), tk.Y - 3, 14.0f,
14.0f);
 470      }
 471      if (A.mode != MODE_EDIT) text(g, "sliders edit the physics LIVE", px + 16, (REAL)(H
- 40), fSmall, C(110, 230, 150));
 472      else text(g, "start v and push apply at Run", px + 16, (REAL)(H - 40), fSmall,
C(120, 128, 145));
 473  }
```

- **The background and headings.** A dark strip the full height of the window, a divider line, and the small section titles TOOLS, SIMULATION, DESIGN FILE and EXAMPLES at the same fixed positions as the buttons.
- **Drawing the buttons.** One loop over `layoutButtons()`. For each: is the mouse over it (`hover`)? Is the Record button active (`rec`)? The fill colour is red for an active Record, blue for an active button, lighter grey for a hover, plain grey otherwise. Then the rectangle, border and centred label.
- **The properties section.** A title built from the selection ("mass m0", "spring", "damper", "ground anchor", or "nothing selected (world settings)"), then one slider per entry of `buildProps()`: the name on the left, the value on the right (a word if the slider has labels, else `"%.3g unit"`), a grey **track**, a blue **fill** up to the value, and a round white **knob**.
- **The footer note.** "sliders edit the physics LIVE" in green while a simulation exists; in design mode a reminder that starting velocity and push apply at Run.

11.10 render: putting the layers together

RENDER.HPP (LINES 476 TO 486)

```
 476  static void render(Graphics& g, int w, int h) {
 477      W = w; H = h;
 478      g.SetSmoothingMode(SmoothingModeAntiAlias);
 479      g.SetTextRenderingHint(TextRenderingHintAntiAlias);
 480      SolidBrush bg(C(12, 14, 18));
 481      g.FillRectangle(&bg, 0, 0, w, h);
 482      drawScene(g);
 483      drawPlots(g);
 484      drawHud(g);
 485      drawPanel(g);       // also paints every button, including the plot tabs (they come
from the same list)
 486  }
```

- `W = w; H = h;` Keeps the global size in step with the canvas.
- `SetSmoothingMode(SmoothingModeAntiAlias)` turns on smooth edges; `SetTextRenderingHint(TextRenderingHintAntiAlias)` does the same for text.
- Fill the background, then `drawScene`, `drawPlots`, `drawHud`, `drawPanel` in that order (the painter's algorithm of Figure 11.1).

This eight-line function is *the entire picture*. Screen and video both come from calling it.

11.11 Chapter checks

QUIZ

1. What does `g.Save()` / `g.Restore()` let `drawLink` do?
2. How are a spring and a damper between the same two nodes kept from overlapping?
3. Why does the plot's vertical range move 20 % at a time?
4. Why is a Y-rail mass shown with only one curve in the Position plot?
5. How does the program colour a spring by its force?
6. What would go wrong if `drawScene` did not set a clip region?

ANSWERS

1. Rotate and shift the canvas to draw flat shapes at an angle, then put the canvas back exactly as it was.
2. Links with the same endpoints are offset sideways by (before − (same−1)/2) × 16 pixels.
3. Smoothing: without it the axis would jump every time a new peak appears; with it the axis glides.
4. The rail forbids x motion, so only the y curve is meaningful (the x series is skipped).
5. The spring's force divided by a reference force, limited to −1..+1: positive blends toward orange, negative toward blue.
6. A long spring or a zoomed-in view could paint over the plots and panel.

CHAPTER 12

The Receptionist: live.cpp

`live.cpp` is the only file the compiler builds directly. It connects everything to Windows. This chapter covers every part except recording (Chapter 13) and the self-test (Chapter 14).

12.1 The header and includes

LIVE.CPP (LINES 1 TO 26)

```
   1  // live.cpp -- Vibration Lab: design your own spring / mass / damper system, run it,
watch the telemetry,
   2  // and record the whole thing to an .mp4 video.  Windows only (plain Win32 + GDI+ +
Media Foundation).
   3  //
   4  //   engine.hpp    the Lagrangian physics
   5  //   model_io.hpp  save / load / example designs
   6  //   app.hpp       state + what every click does
   7  //   render.hpp    all the drawing
   8  //   encoder.hpp   H.264 video writer
   9  //   live.cpp      (this file) window, main loop, recording, CSV export, self-test
  10  //
  11  //   Command line:   vibration_live --selftest
  12  //                   vibration_live --shot out.png  [--preset N] [--run SECONDS] [--
select node:I|link:I] [--tab N]
  13  //                   vibration_live --demo out.mp4  [--preset N] [--run SECONDS]
(records without opening a window)
  14  #include <algorithm>
  15  namespace Gdiplus { using std::min; using std::max; }
  16  #include <windows.h>
  17  #include <gdiplus.h>
  18
  19  #include <ctime>
  20  #include <fstream>
  21
  22  #include "app.hpp"
  23  #include "encoder.hpp"
  24  #include "render.hpp"
  25
  26  static HWND gWnd = nullptr;
```

- The big comment is the **map of the program**: one line per file, plus the command-line modes. The first thing a new reader sees explains the whole layout.
- `namespace Gdiplus { using std::min; using std::max; }` is a workaround. The GDI+ headers use the names `min` and `max` without saying where they come from; this line tells the compiler "inside `Gdiplus`, `min` means `std::min`". It must come **before** `#include <gdiplus.h>`.
- The **order of the includes matters**: `<windows.h>` first, then `<gdiplus.h>`, then our headers, because our headers use Windows types such as `LARGE_INTEGER` and GDI+ types such as `RectF`.

- `static HWND gWnd = nullptr;` The handle of our window. `HWND` is "handle to a window", a number Windows uses to identify it. `nullptr` means "no window yet". The headless modes never create a window, and `present()` checks for that.

12.2 The Canvas: our hidden picture

LIVE.CPP (LINES 29 TO 45)

```
  29  struct Canvas {
  30      HDC dc = nullptr; HBITMAP bmp = nullptr, old = nullptr; void* bits = nullptr; int w
= 0, h = 0;
  31      void make(int nw, int nh) {
  32          destroy();
  33          BITMAPINFO bi{};
  34          bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
  35          bi.bmiHeader.biWidth = nw; bi.bmiHeader.biHeight = -nh;            // negative
height = rows run top to bottom
  36          bi.bmiHeader.biPlanes = 1; bi.bmiHeader.biBitCount = 32;
  37          HDC screen = GetDC(nullptr);
  38          dc = CreateCompatibleDC(screen);
  39          bmp = CreateDIBSection(screen, &bi, DIB_RGB_COLORS, &bits, nullptr, 0);
  40          ReleaseDC(nullptr, screen);
  41          old = (HBITMAP)SelectObject(dc, bmp);
  42          w = nw; h = nh;
  43      }
  44      void destroy() { if (dc) { SelectObject(dc, old); DeleteObject(bmp); DeleteDC(dc);
dc = nullptr; } }
  45  } canvas;
```

- A **struct with its own methods**. It holds a device context `dc`, a bitmap `bmp`, the previous bitmap `old`, a pointer `bits` to the **pixel memory**, and the size.
- **`make(nw, nh)`**: first `destroy()` any old one. Then fill a `BITMAPINFO` describing a 32-bit-per-pixel image: `biWidth = nw` and **`biHeight = -nh`**. The *negative* height is a Windows convention meaning "rows run **top to bottom**", which is what we want and what the recorder expects.
- `GetDC(nullptr)` gets the screen's device context, used only as a model for colour format. `CreateCompatibleDC` makes a memory device context that can hold a picture. `CreateDIBSection(...)` makes the actual bitmap and returns a pointer to its pixels in `bits`. `SelectObject(dc, bmp)` puts the bitmap into the device context so drawing goes into it. `ReleaseDC` gives the screen DC back.
- **`destroy()`** undoes it in the reverse order: select the old bitmap back, delete the new one, delete the DC. Windows resources must be released by hand; forgetting leaks memory every time the window is resized.

LIVE.CPP (LINES 47 TO 51)

```
  47  static void drawFrameToCanvas() {
  48      if (canvas.w != W || canvas.h != H) canvas.make(W, H);
  49      Graphics g(canvas.dc);
  50      render(g, W, H);
  51  }
```

LIVE.CPP (LINES 52 TO 58)

```
52  static void present() {
53      if (!gWnd || W < 10 || H < 10) return;
54      drawFrameToCanvas();
55      HDC hdc = GetDC(gWnd);
56      BitBlt(hdc, 0, 0, W, H, canvas.dc, 0, 0, SRCCOPY);
57      ReleaseDC(gWnd, hdc);
58  }
```

- `drawFrameToCanvas` makes sure the canvas has the right size (a resize makes a new one), creates a `Graphics` object that draws **into the canvas's device context**, and calls `render`. This is the moment a picture exists in memory.
- `present` returns immediately if there is no window or the window is tiny. Otherwise it draws the frame, then `BitBlt`s the finished canvas to the window in one go: `SRCCOPY` means "copy the pixels exactly". Double buffering in six lines.

12.3 Files and folders

LIVE.CPP (LINES 61 TO 66)

```
61  static std::string stamp() {
62      time_t t = time(nullptr);
63      char b[32];
64      strftime(b, sizeof b, "%Y%m%d_%H%M%S", localtime(&t));
65      return b;
66  }
```

LIVE.CPP (LINES 67 TO 71)

```
67  static std::string recordingsDir() {
68      std::string d = std::string(LAB_DIR) + "/recordings";
69      CreateDirectoryA(d.c_str(), nullptr);
70      return d;
71  }
```

- `stamp()` returns a timestamp such as `20261112_221530` (year, month, day, hour, minute, second), made with `strftime`. File names built from it sort in time order and never collide.
- `recordingsDir()` builds `LAB_DIR/recordings` and calls `CreateDirectoryA`, which creates the folder or quietly does nothing if it already exists. Videos and CSV files all go there.

12.4 Exporting the telemetry to CSV

LIVE.CPP (LINES 103 TO 132)

```
103 void exportCsv() {
104     if (A.log.empty()) { toast("Nothing to export yet: press Run first"); return; }
105     std::string path = recordingsDir() + "/telemetry_" + stamp() + ".csv";
106     std::ofstream f(path);
107     if (!f) { toast("Could not write the CSV file"); return; }
108     const lag::Model& M = A.model;
109     f << "t,T,V,E,dissipated,work_in";
110     int k = 0;
111     for (size_t i = 0; i < M.nodes.size(); ++i) {
112         if (M.nodes[i].kind != lag::NodeKind::Mass) continue;
113         f << ",m" << k << "_x,m" << k << "_y,m" << k << "_vx,m" << k << "_vy";
114         ++k;
```

```
 115      }
 116      for (size_t li = 0; li < M.links.size(); ++li) {
 117          const char* kind = M.links[li].kind == lag::LinkKind::Spring ? "spring" :
"damper";
 118          f << "," << kind << li << (M.links[li].kind == lag::LinkKind::Spring ?
"_stretch" : "_rate") << "," << kind << li << "_force," << kind << li << "_length";
 119      }
 120      f << "\n";
 121      f.precision(7);
 122      for (const Frame& fr : A.log) {
 123          f << fr.t << "," << fr.T << "," << fr.V << "," << fr.T + fr.V << "," <<
fr.dissipated << "," << fr.workIn;
 124          for (size_t i = 0; i < M.nodes.size(); ++i)
 125              if (M.nodes[i].kind == lag::NodeKind::Mass && 4 * i + 3 < fr.nd.size())
 126                  f << "," << fr.nd[4 * i] << "," << fr.nd[4 * i + 1] << "," << fr.nd[4 *
i + 2] << "," << fr.nd[4 * i + 3];
 127          for (size_t li = 0; li < M.links.size(); ++li)
 128              if (3 * li + 2 < fr.lk.size()) f << "," << fr.lk[3 * li] << "," << fr.lk[3
* li + 1] << "," << fr.lk[3 * li + 2];
 129          f << "\n";
 130      }
 131      toast("Telemetry saved: recordings/" + path.substr(path.find_last_of('/') + 1));
 132  }
```

CSV ("comma-separated values") is a plain text table that Excel, Python, MATLAB and others can all open.

- If the log is empty, tell the user and stop.
- Build a path with `stamp()` and open an `std::ofstream`; if that fails, say so.
- **The header row.** `t,T,V,E,dissipated,work_in` then, for every **mass**, four columns: `m0_x,m0_y,m0_vx,m0_vy`, and for every link three columns named after its kind and index: `spring3_stretch, spring3_force, spring3_length` (or `damper2_rate, ...`).
- `f.precision(7);` Seven significant digits.
- **One row per telemetry frame**, in the same column order: the times and energies, then the four numbers of each mass, then the three of each link. The guards `4*i+3 < fr.nd.size()` protect against a frame that was recorded with fewer nodes.
- Finally a toast with the file name.

> **TRY IT**
> Run an example for ten seconds, press **Export CSV**, open the file from the `recordings` folder in a spreadsheet, and make a chart of `m0_x` against `t`. It should look like the plot on screen.

12.5 Reading the dominant frequency

LIVE.CPP (LINES 135 TO 144)

```
 135  static void updateDominantFrequency() {
 136      if (A.mode != MODE_RUN || A.selKind != SEL_NODE || A.sel < 0 || A.sel >=
(int)A.model.nodes.size()) { A.domFreq = 0; return; }
 137      if (A.model.nodes[A.sel].kind != lag::NodeKind::Mass || A.log.size() < 64) {
A.domFreq = 0; return; }
 138      int comp = ((int)std::lround(A.model.nodes[A.sel].rail) == lag::RAIL_Y) ? 1 : 0;
 139      size_t n = std::min<size_t>(A.log.size(), 360), start = A.log.size() - n;
 140      std::vector<double> x;
```

```
 141      for (size_t i = start; i < A.log.size(); ++i) x.push_back(A.log[i].nd[4 *
(size_t)A.sel + comp]);
 142      double span = A.log.back().t - A.log[start].t;
 143      A.domFreq = span > 0.5 ? lag::dominantFrequency(x, (double)(n - 1) / span) : 0;
 144  }
```

- It works only in run mode, with a **mass** selected, and once at least 64 rows exist (about one second): otherwise `domFreq = 0` and the display shows "--".
- `comp` chooses which coordinate to analyse: y for a Y-rail mass, x otherwise.
- It copies the **last 360 rows** (about six seconds) of that coordinate into a vector.
- `span = last time - first time` and the sample rate is `(n - 1) / span`: the number of intervals divided by the time they took. We *measure* the sample rate instead of assuming 60, because the frames are not perfectly evenly spaced.
- It calls `lag::dominantFrequency` (Chapter 7) only if the span is over half a second.

The main loop calls this about every 0.3 seconds. The calculation is not free (a Fourier test of several hundred samples at over a thousand frequencies), so it is *rationed*.

12.6 The keyboard

LIVE.CPP (LINES 147 TO 172)

```
 147  static void onKey(WPARAM k) {
 148      bool ctrl = (GetKeyState(VK_CONTROL) & 0x8000) != 0;
 149      if (ctrl && k == 'S') { saveDesign(); return; }
 150      if (ctrl && k == 'O') { loadDesign(); return; }
 151      switch (k) {
 152          case VK_SPACE: toggleRun(); break;
 153          case VK_ESCAPE:
 154              if (A.linkStart >= 0) A.linkStart = -1;
 155              else if (A.mode != MODE_EDIT) stopRun();
 156              else setTool(TOOL_SELECT);
 157              break;
 158          case 'S': setTool(TOOL_SELECT); break;
 159          case 'M': setTool(TOOL_MASS); break;
 160          case 'G': setTool(TOOL_GROUND); break;
 161          case 'K': setTool(TOOL_SPRING); break;
 162          case 'D': setTool(TOOL_DAMPER); break;
 163          case VK_DELETE: case VK_BACK: deleteSelection(); break;
 164          case VK_F9: toggleRecording(); break;
 165          case VK_TAB: A.plotMode = (A.plotMode + 1) % 4; break;
 166          case VK_HOME: resetView(); break;
 167          case VK_OEM_COMMA:  A.speed = std::fmax(0.125, A.speed / 2); break;
 168          case VK_OEM_PERIOD: A.speed = std::fmin(4.0, A.speed * 2); break;
 169          case '1': case '2': case '3': case '4': loadPreset((int)(k - '0')); break;
 170          case VK_NUMPAD1: case VK_NUMPAD2: case VK_NUMPAD3: case VK_NUMPAD4:
loadPreset((int)(k - VK_NUMPAD0)); break;
 171      }
 172  }
```

- `GetKeyState(VK_CONTROL) & 0x8000` asks whether the Ctrl key is held down. The `0x8000` picks the "is down" bit.
- **Ctrl+S** saves the design, **Ctrl+O** loads it.
- A `switch` maps each key to a function you have already met: `VK_SPACE` toggles run; `VK_ESCAPE` cancels a half-drawn link, or else stops the run, or else returns to the Select tool

(three different meanings, chosen by what is going on); letters S, M, G, K, D pick the tools; Delete or Backspace deletes; F9 toggles recording; Tab cycles the plot tabs; Home resets the view; comma and period halve and double the speed (clamped to 0.125..4); the digits 1 to 4 (and the numeric keypad) load the examples.

- `VK_...` are *virtual-key codes*: names Windows gives to keys. Letters and digits use their ASCII codes ('S', '1').

12.7 The window procedure

LIVE.CPP (LINES 174 TO 200)

```
 174  static LRESULT CALLBACK WndProc(HWND h, UINT m, WPARAM wp, LPARAM lp) {
 175      switch (m) {
 176          case WM_SIZE:   W = LOWORD(lp); H = HIWORD(lp); return 0;
 177          case WM_GETMINMAXINFO: { MINMAXINFO* mi = (MINMAXINFO*)lp; mi->ptMinTrackSize.x
= 1100; mi->ptMinTrackSize.y = 700; return 0; }
 178          case WM_ERASEBKGND: return 1;
 179          case WM_PAINT:  { PAINTSTRUCT ps; BeginPaint(h, &ps); EndPaint(h, &ps);
present(); return 0; }
 180          case WM_LBUTTONDOWN: SetCapture(h); onMouseDown((short)LOWORD(lp),
(short)HIWORD(lp)); return 0;
 181          case WM_MOUSEMOVE:   onMouseMove((short)LOWORD(lp), (short)HIWORD(lp)); return
0;
 182          case WM_LBUTTONUP:   ReleaseCapture(); onMouseUp(); return 0;
 183          case WM_RBUTTONDOWN: case WM_MBUTTONDOWN:
 184              SetCapture(h); A.panning = true; A.panX = (short)LOWORD(lp); A.panY =
(short)HIWORD(lp); return 0;
 185          case WM_RBUTTONUP: case WM_MBUTTONUP: ReleaseCapture(); A.panning = false;
return 0;
 186          case WM_MOUSEWHEEL: {
 187              POINT p = {(short)LOWORD(lp), (short)HIWORD(lp)};
 188              ScreenToClient(h, &p);
 189              onWheel(p.x, p.y, GET_WHEEL_DELTA_WPARAM(wp) / 120);
 190              return 0;
 191          }
 192          case WM_KEYDOWN: onKey(wp); return 0;
 193          case WM_DESTROY:
 194              stopRecording();
 195              lag::saveModel(designPath(), A.model);          // your design is still
there next time
 196              PostQuitMessage(0);
 197              return 0;
 198      }
 199      return DefWindowProc(h, m, wp, lp);
 200  }
```

The function Windows calls for every message. Read the cases in turn:

- **`WM_SIZE`**: the window changed size. `LOWORD(lp)` and `HIWORD(lp)` are the new width and height, packed into one number; we store them in `W` and `H`. Next frame, every rectangle is recomputed from them.
- **`WM_GETMINMAXINFO`**: Windows asks "what is the smallest size you allow?" We say 1100 × 700, so the panel and plots never get squashed beyond use.
- **`WM_ERASEBKGND`**: returns 1 ("I did it") without erasing. If Windows cleared the window to white before every repaint, it would flash. We paint every pixel ourselves.

- `WM_PAINT`: Windows says the window needs repainting. `BeginPaint/EndPaint` tell it we noticed; then `present()` draws.
- `WM_LBUTTONDOWN / MOUSEMOVE / LBUTTONUP`: the left button. `SetCapture(h)` on press makes Windows keep sending mouse messages to our window *even if the pointer leaves it*, so a drag continues past the edge; `ReleaseCapture()` on release lets go. The position is packed in `lp`, extracted with `LOWORD`/`HIWORD` and cast to `(short)` so that positions left of or above the window come out **negative** instead of huge positive numbers. The handlers are the ones from `app.hpp`.
- `WM_RBUTTONDOWN / MBUTTONDOWN` start a **pan**: set `A.panning` and remember the start. The matching UP messages end it.
- `WM_MOUSEWHEEL`: the wheel position arrives in **screen** coordinates, so `ScreenToClient` converts it to window coordinates. `GET_WHEEL_DELTA_WPARAM(wp) / 120` is the number of notches (one notch = 120 units).
- `WM_KEYDOWN`: a key was pressed; hand it to `onKey`.
- `WM_DESTROY`: the window is closing. We **stop any recording** (so the .mp4 is finished properly), **save the design** to `design.vlab` (so it is waiting for you next time), and `PostQuitMessage(0)` which puts `WM_QUIT` into the queue and ends the main loop.
- `DefWindowProc` handles every message we did not mention, with Windows' default behaviour (moving the window, minimising, and so on). Forgetting it makes a window that cannot be dragged.

12.8 Reading the command line

LIVE.CPP (LINES 343 TO 343)

```
 343  struct CliOptions { int preset = 0; double runSeconds = 0; std::string select; int tab
= 0; bool empty = false; };
```

LIVE.CPP (LINES 344 TO 355)

```
 344  static CliOptions parseOptions(int argc, char** argv) {
 345      CliOptions o;
 346      for (int i = 1; i + 1 < argc; ++i) {
 347          std::string a = argv[i];
 348          if (a == "--preset") o.preset = std::atoi(argv[i + 1]);
 349          else if (a == "--run") o.runSeconds = std::atof(argv[i + 1]);
 350          else if (a == "--select") o.select = argv[i + 1];
 351          else if (a == "--tab") o.tab = std::atoi(argv[i + 1]);
 352          else if (a == "--empty") o.empty = std::atoi(argv[i + 1]) != 0;
 353      }
 354      return o;
 355  }
```

LIVE.CPP (LINES 356 TO 362)

```
 356  static void applyOptions(const CliOptions& o) {
 357      if (o.preset > 0) A.model = lag::presetByNumber(o.preset);
 358      if (o.empty) A.model = lag::Model();
 359      A.plotMode = o.tab;
 360      if (o.select.rfind("node:", 0) == 0) { A.selKind = SEL_NODE; A.sel =
std::atoi(o.select.c_str() + 5); }
```

```
 361      if (o.select.rfind("link:", 0) == 0) { A.selKind = SEL_LINK; A.sel =
std::atoi(o.select.c_str() + 5); }
 362  }
```

LIVE.CPP (LINES 363 TO 368)

```
 363  static void stepOffscreen(double seconds) {            // advance exactly like the
recorder does: 1/60 s per frame
 364      for (int i = 0; i < (int)(seconds * REC_FPS); ++i) {
 365          advanceSim(1.0 / REC_FPS);
 366          if (i % 18 == 0) updateDominantFrequency();
 367      }
 368  }
```

- These functions support the headless modes (`--shot` and `--demo`) used for testing and for taking this guide's screenshots.
- `parseOptions` scans the arguments in pairs: `--preset 3`, `--run 4` (seconds), `--select node:1` or `link:2`, `--tab 2`, `--empty 1`.
- `applyOptions` loads the chosen example and selection into the app state.
- `stepOffscreen` advances the simulation by whole seconds in steps of 1/60 s, exactly like the recorder does, refreshing the dominant frequency every 18 frames.

12.9 main: the front door

LIVE.CPP (LINES 371 TO 384)

```
 371  int main(int argc, char** argv) {
 372      if (argc > 1 && std::strcmp(argv[1], "--selftest") == 0) return selfTest();
 373
 374      GdiplusStartupInput gsi; ULONG_PTR tok;
 375      GdiplusStartup(&tok, &gsi, nullptr);
 376      gMono = new FontFamily(L"Consolas");
 377      fSmall = new Font(gMono, 12, FontStyleRegular, UnitPixel);
 378      fNorm  = new Font(gMono, 14, FontStyleBold, UnitPixel);
 379      fBig   = new Font(gMono, 20, FontStyleBold, UnitPixel);
 380
 381      if (!lag::loadModel(designPath(), A.model)) A.model = lag::presetTwoDof();     //
last session's design, or an example
 382      resetView();
 383
 384      int rc = 0;
```

- `--selftest` is handled first, before anything else starts, because the tests need no window and no GDI+.
- `GdiplusStartup(&gsi, &tok, nullptr)` starts the drawing library and returns a token; `GdiplusShutdown(tok)` at the end closes it. The fonts are created *after* the start.
- `lag::loadModel(designPath(), A.model)` loads last session's design; if there is none (the first run), `presetTwoDof()` is used so the user always sees something.
- `resetView()` centres the view.

LIVE.CPP (LINES 385 TO 420)

```
 385      if (argc > 2 && (std::strcmp(argv[1], "--shot") == 0 || std::strcmp(argv[1], "--
demo") == 0)) {
 386          bool video = std::strcmp(argv[1], "--demo") == 0;
 387          CliOptions o = parseOptions(argc, argv);
```

```
 388          W = 1320; H = 800;
 389          if (o.preset == 0 && !video) o.preset = 2;
 390          if (o.preset == 0) o.preset = 4;
 391          resetView();
 392          applyOptions(o);
 393          A.mx = A.my = -100;
 394          if (video) {
 395              std::string err;
 396              recW = (W - PANEL_W) & ~1; recH = H & ~1;
 397              if (!encoder.open(argv[2], recW, recH, REC_FPS, err)) { std::printf("cannot
record: %s\n", err.c_str()); rc = 1; }
 398              else {
 399                  startRun();
 400                  A.recording = true;
 401                  int frames = (int)(std::fmax(1.0, o.runSeconds) * REC_FPS);
 402                  for (int i = 0; i < frames; ++i) {
 403                      advanceSim(1.0 / REC_FPS);
 404                      if (i % 18 == 0) updateDominantFrequency();
 405                      drawFrameToCanvas();
 406                      captureFrame();
 407                  }
 408                  A.recording = false;
 409                  bool ok = encoder.close();
 410                  std::printf("demo %s: %d frames (%s)\n", argv[2], A.recFrames, ok ?
"ok" : "FAILED");
 411                  rc = ok ? 0 : 1;
 412              }
 413          } else {
 414              if (o.runSeconds > 0) { startRun(); stepOffscreen(o.runSeconds); }
 415              drawFrameToCanvas();
 416              bool ok = savePng(argv[2]);
 417              std::printf("shot %s (%s)\n", argv[2], ok ? "ok" : "FAILED");
 418              rc = ok ? 0 : 1;
 419          }
 420      } else {
```

- The **headless** branch. `--shot` renders **one picture** to a PNG: set the size (1320 × 800), apply the options, optionally run N seconds, draw once, save. `--demo` **records a video** without any window: it starts the encoder, runs the simulation frame by frame at exactly 1/60 s, draws each frame into the canvas, and writes it. These modes are why this guide's screenshots are real, and why we could test the recorder with no one at the keyboard.

LIVE.CPP (LINES 421 TO 455)

```
 421          std::printf("Vibration Lab: a window has opened. Design a system, press
Run.\n");
 422          WNDCLASSA wc{};
 423          wc.lpfnWndProc = WndProc; wc.hInstance = GetModuleHandle(nullptr);
wc.lpszClassName = "VibLab";
 424          wc.hCursor = LoadCursor(nullptr, IDC_ARROW);
 425          RegisterClassA(&wc);
 426          RECT r = {0, 0, W, H};
 427          AdjustWindowRect(&r, WS_OVERLAPPEDWINDOW, FALSE);
 428          gWnd = CreateWindowA("VibLab", "NOVA Vibration Lab  -  design, simulate,
record", WS_OVERLAPPEDWINDOW | WS_VISIBLE,
 429                               CW_USEDEFAULT, CW_USEDEFAULT, r.right - r.left, r.bottom -
r.top, nullptr, nullptr, wc.hInstance, nullptr);
 430          SetForegroundWindow(gWnd);
 431          double last = nowSeconds(), fpsT = last, domT = last;
 432          int frames = 0;
 433          MSG msg;
 434          bool running = true;
 435          while (running) {
 436              while (PeekMessage(&msg, nullptr, 0, 0, PM_REMOVE)) {
 437                  if (msg.message == WM_QUIT) running = false;
 438                  TranslateMessage(&msg);
 439                  DispatchMessage(&msg);
```

```
 440             }
 441             double now = nowSeconds(), dt = clampd(now - last, 0.0, 0.05);
 442             last = now;
 443             // while recording, every video frame is exactly 1/60 s of simulated time,
so the movie is smooth and true-to-life
 444             advanceSim(A.recording ? 1.0 / REC_FPS : dt);
 445             if (now - domT > 0.3) { updateDominantFrequency(); domT = now; }
 446             present();
 447             captureFrame();
 448             ++frames;
 449             if (now - fpsT >= 0.5) { A.fps = frames / (now - fpsT); frames = 0; fpsT =
now; }
 450             if (A.recording) {                                  // pace to the video
frame rate so dragging feels natural
 451                 double spent = nowSeconds() - now;
 452                 if (spent < 1.0 / REC_FPS) Sleep((DWORD)((1.0 / REC_FPS - spent) *
1000));
 453             } else Sleep(1);
 454         }
 455     }
```

The **interactive** branch:

- `WNDCLASSA wc{};` describes a *window class*: a template for windows. We set the window procedure (`lpfnWndProc = WndProc`), the program instance, a class name, and the arrow cursor. `RegisterClassA(&wc)` registers it with Windows.
- `AdjustWindowRect` converts the *client* size we want (1320 × 800 for the drawing area) into the *outer* window size, which includes the title bar and borders.
- `CreateWindowA(...)` makes the window. `WS_OVERLAPPEDWINDOW | WS_VISIBLE` asks for a normal resizable window that is shown at once.
- `SetForegroundWindow` brings it to the front.
- **The loop** (Figure 9.1): drain the message queue with `PeekMessage`; read the clock and compute `dt = clampd(now - last, 0, 0.05)` (never more than 50 ms, so a stall cannot produce a giant jump); `advanceSim(A.recording ? 1.0/REC_FPS : dt)`; every 0.3 s refresh the frequency; `present()`; `captureFrame()`; count frames for the FPS readout every half second.
- **The pacing.** While recording, the loop sleeps until a full 1/60 s has passed, so dragging a mass during a recording feels natural and the video's time matches the real time you spent. Otherwise `Sleep(1)` yields the processor briefly so the program does not use 100 % of a CPU core for nothing.

LIVE.CPP (LINES 456 TO 459)

```
 456     canvas.destroy();
 457     GdiplusShutdown(tok);
 458     return rc;
 459 }
```

- Release the canvas and shut down GDI+, then return the exit code (0 for success).

WHAT IF IT WERE DIFFERENT?

What if `dt` were not clamped? Drag the window by its title bar: Windows halts the main loop while you hold it, perhaps for several seconds. When you let go, `dt` would be several seconds and the physics would try to simulate all of that at once. The clamp turns a long pause into a tiny hiccup instead.

12.10 Chapter checks

QUIZ

1. Why do we handle `WM_ERASEBKGND` by returning 1?
2. What does `SetCapture` do for a drag?
3. Why cast the mouse coordinates to `(short)`?
4. What happens in `WM_DESTROY`, and why in that order?
5. Why is `dt` clamped to 50 ms?
6. How does the program decide between the interactive and headless modes?

ANSWERS

1. To stop Windows erasing to white before each repaint, which would flicker; we draw every pixel ourselves.
2. It makes Windows send mouse messages to our window even when the pointer leaves it, so drags continue outside.
3. The values are packed 16-bit numbers; casting to short gives the right negative values for positions left of or above the window.
4. Recording is stopped (finishing the file), the design is saved, then the quit message ends the loop.
5. After a long stall (for example while dragging the window) one frame would otherwise demand a huge amount of simulation.
6. By the command-line arguments: `--shot` and `--demo` take the headless branch, no arguments opens the window.

Part IV — The Recorder and the Tests

CHAPTER 13

The Direct-to-Video Recorder

The Record button turns what you see into a normal video file you can play, share and put in a presentation. This chapter explains how video works, how Windows' built-in encoder is driven, and every line of `encoder.hpp`.

13.1 What a video really is

A video is a **flipbook**: a list of still pictures (called **frames**) shown quickly one after another. Show 60 pictures every second and the eye sees smooth motion.

Three numbers describe it:

- the **resolution**: how many pixels wide and tall each frame is (our videos are 1020 × 800 with the standard window size);
- the **frame rate**: how many frames per second (we use 60);
- the **duration**: how long it plays.

Why compression is unavoidable

Each of our pixels takes 4 bytes. One frame is 1020 × 800 × 4 = 3.26 million bytes. At 60 frames per second that is **196 million bytes every second**: one minute of raw video would be almost 12 gigabytes. Nobody could save or share that.

A **codec** (**co**der-**dec**oder) is a method for squeezing video. The one we use is **H.264** (also called AVC), the most widely supported codec in the world. It exploits two facts:

1. **Neighbouring pixels are similar** (a flat dark background). Store the *difference* instead of every pixel.
2. **Consecutive frames are similar** (only the masses and curves moved). Store only *what changed* since the last frame.

Our test video: 6 seconds, 360 frames, **5.0 million bytes**: about 0.8 million bytes per second. That is a squeeze of more than 200 to 1.

KEY WORD

Codec. The compression method (H.264).

Container. The file format that holds the compressed pictures plus an index (MP4). People often say "MP4" for both; they are different things: the container is the box, the codec is how the contents were packed.

Bitrate. How many bits per second the compressed video uses. We ask for 8 million (8 Mbit/s, about 1 MB/s); simple content like ours needs less, and the encoder uses what it needs.

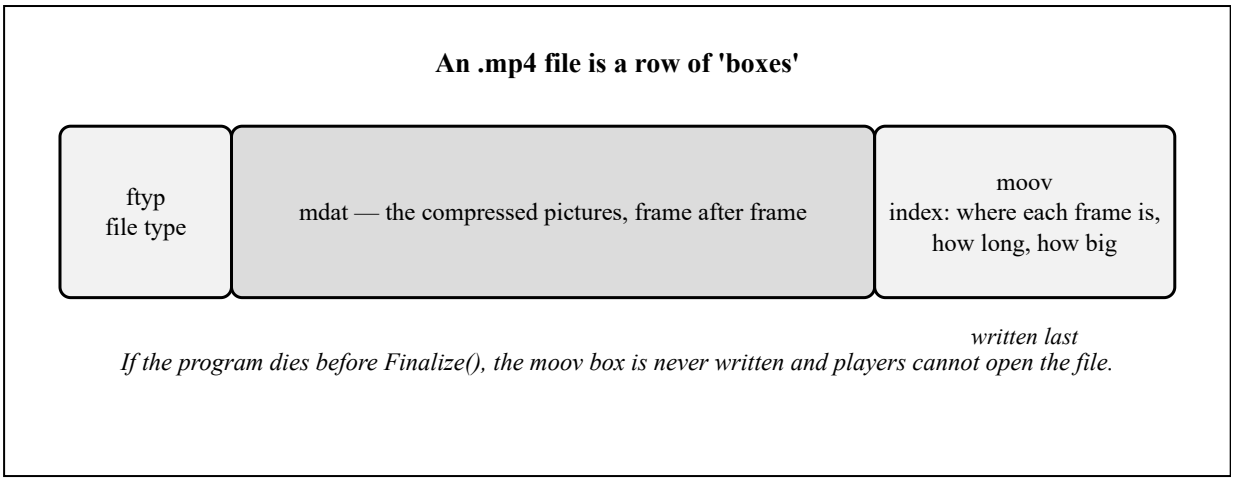


Figure 13.1. *An MP4 file is a series of labelled "boxes". The index (moov) is written at the very end, which is why the encoder must be finalised.*

13.2 Why we did not write our own encoder, and did not use ffmpeg

Writing an H.264 encoder from scratch takes a large team and years. The popular tool **ffmpeg** is excellent, but it is a separate download that every user would have to install, and we wanted the program to work on a plain Windows PC. Windows already contains an H.264 encoder as part of **Media Foundation**, its multimedia framework. We use that. The cost: Windows only. The gain: zero installation.

13.3 A very short COM primer

Media Foundation is built with **COM** (Component Object Model), an older Microsoft technology. You only need four ideas:

- An **interface** is a set of functions on an object you cannot see inside. Our code holds a **pointer to an interface**, such as `IMFSinkWriter*`.
- COM objects **count how many users they have**. When you are finished you must call `Release()`, which lowers the count; at zero the object destroys itself. Forgetting

`Release()` is a *memory leak*.

- Almost every COM function returns an **HRESULT**, a number where success is non-negative and failure is negative. The macros `SUCCEEDED(hr)` and `FAILED(hr)` test it. **Always check.**
- A program must **initialise** COM (`CoInitializeEx`) and Media Foundation (`MFStartup`) once before using them.

13.4 The Sink Writer pipeline

The Media Foundation object we use is the **Sink Writer**. You give it *uncompressed* pictures; it finds an encoder, compresses them and writes the MP4 file.

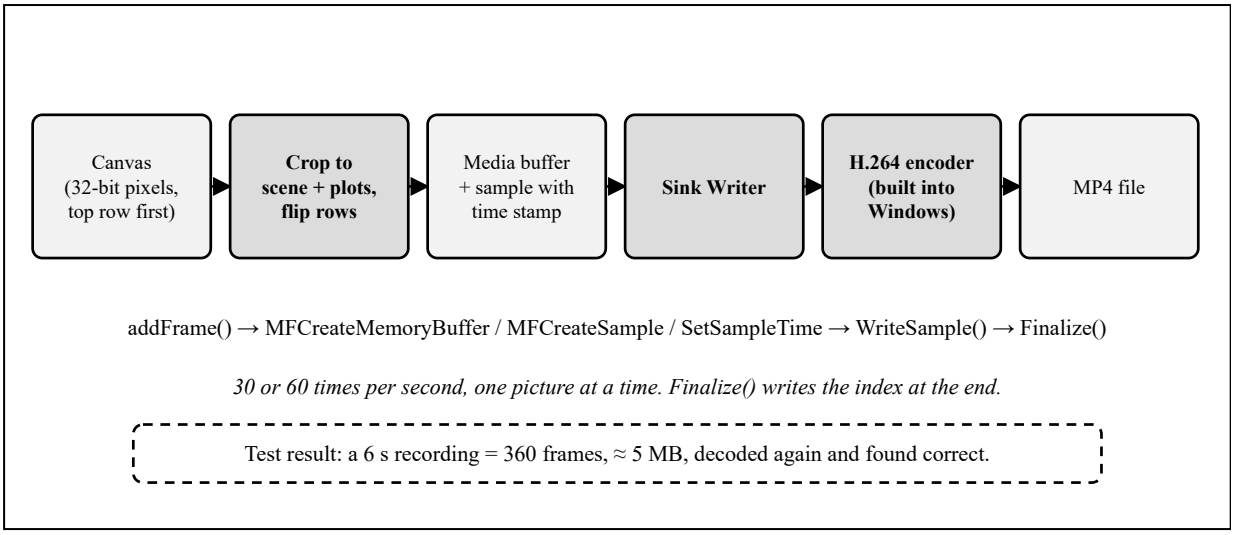


Figure 13.2. *From a drawn frame to a finished MP4 file. Our code does the first three boxes; Windows does the rest.*

Giving a picture to the Sink Writer takes four steps: describe **what you will send in**, describe **what you want stored**, say **begin**, then send frames one at a time and say **finalize**.

13.5 encoder.hpp: the header comment and includes

ENCODER.HPP (LINES 1 TO 19)

```
   1  // encoder.hpp -- "direct to video": writes the picture on screen into an H.264 .mp4
file,
   2  // using the video encoder that is already built into Windows (Media Foundation Sink
Writer).
   3  // No ffmpeg or any other install needed.
   4  //
   5  //   VideoEncoder enc;
   6  //   enc.open("movie.mp4", 1000, 700, 60, error);     // width/height must be even
   7  //   enc.addFrame(pixels, rowBytes, 1000, 700);        // call once per video frame,
top-down BGRA pixels
   8  //   enc.close();                                      // finishes the file (mp4 is
unplayable without this!)
   9  #pragma once
  10  #include <windows.h>
```

```
  11  #include <mfapi.h>
  12  #include <mferror.h>
  13  #include <mfidl.h>
  14  #include <mfreadwrite.h>
  15
  16  #include <cstdio>
  17  #include <cstring>
  18  #include <string>
  19  #include <vector>
```

- The comment shows the whole interface in six lines of example code. A good header tells you how to *use* it before you read how it works.
- `mfapi.h, mfidl.h, mfreadwrite.h, mferror.h` declare the Media Foundation functions and types.
- The matching libraries (`mfplat`, `mfreadwrite`, `mfuuid`, `ole32`) are listed in `CMakeLists.txt`. `mfuuid` supplies the long identifier numbers (GUIDs) such as `MFVideoFormat_H264`; forgetting it gives "undefined reference" errors at link time.

13.6 The class and its small accessors

ENCODER.HPP (LINES 21 TO 26)

```
  21  class VideoEncoder {
  22  public:
  23      ~VideoEncoder() { if (writer_) close(); }
  24      bool isOpen() const { return writer_ != nullptr; }
  25      int  framesWritten() const { return (int)frame_; }
  26      double seconds() const { return fps_ > 0 ? (double)frame_ / fps_ : 0.0; }
```

- `~VideoEncoder() { if (writer_) close(); }` The **destructor**: if the encoder object disappears while a file is still open, it finishes the file automatically. This is RAII again: cleanup is guaranteed even if the program is shutting down in a hurry.
- `isOpen()` tells whether a recording is in progress; `framesWritten()` and `seconds()` report progress (frames divided by the frame rate).

13.7 open: set everything up

ENCODER.HPP (LINES 28 TO 76)

```
  28      bool open(const std::string& path, int width, int height, int fps, std::string&
err, int bitrate = 8000000) {
  29          if (writer_) { err = "already recording"; return false; }
  30          if (width < 16 || height < 16 || (width & 1) || (height & 1)) { err = "video
size must be even and at least 16"; return false; }
  31          if (!startup(err)) return false;
  32
  33          WCHAR wpath[1024];
  34          MultiByteToWideChar(CP_UTF8, 0, path.c_str(), -1, wpath, 1024);
  35
  36          IMFAttributes* attr = nullptr;
  37          MFCreateAttributes(&attr, 2);
  38          attr->SetUINT32(MF_READWRITE_ENABLE_HARDWARE_TRANSFORMS, TRUE);     // use the
GPU encoder if there is one
  39          attr->SetUINT32(MF_SINK_WRITER_DISABLE_THROTTLING, TRUE);           // never
skip frames to "keep up"
  40          HRESULT hr = MFCreateSinkWriterFromURL(wpath, nullptr, attr, &writer_);
```

```
  41          attr->Release();
  42          if (FAILED(hr)) { err = hrText("could not create the .mp4 file", hr); writer_ =
nullptr; return false; }
  43
  44          // 1) what we want to STORE: H.264 video
  45          IMFMediaType* out = nullptr;
  46          MFCreateMediaType(&out);
  47          out->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
  48          out->SetGUID(MF_MT_SUBTYPE, MFVideoFormat_H264);
  49          out->SetUINT32(MF_MT_AVG_BITRATE, (UINT32)bitrate);
  50          out->SetUINT32(MF_MT_INTERLACE_MODE, MFVideoInterlace_Progressive);
  51          out->SetUINT64(MF_MT_FRAME_SIZE, pack((UINT32)width, (UINT32)height));
  52          out->SetUINT64(MF_MT_FRAME_RATE, pack((UINT32)fps, 1));
  53          out->SetUINT64(MF_MT_PIXEL_ASPECT_RATIO, pack(1, 1));
  54          hr = writer_->AddStream(out, &stream_);
  55          out->Release();
  56          if (FAILED(hr)) return fail("no H.264 encoder available", hr, err);
  57
  58          // 2) what we will FEED it: plain 32-bit RGB pictures
  59          IMFMediaType* in = nullptr;
  60          MFCreateMediaType(&in);
  61          in->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
  62          in->SetGUID(MF_MT_SUBTYPE, MFVideoFormat_RGB32);
  63          in->SetUINT32(MF_MT_INTERLACE_MODE, MFVideoInterlace_Progressive);
  64          in->SetUINT64(MF_MT_FRAME_SIZE, pack((UINT32)width, (UINT32)height));
  65          in->SetUINT64(MF_MT_FRAME_RATE, pack((UINT32)fps, 1));
  66          in->SetUINT64(MF_MT_PIXEL_ASPECT_RATIO, pack(1, 1));
  67          hr = writer_->SetInputMediaType(stream_, in, nullptr);
  68          in->Release();
  69          if (FAILED(hr)) return fail("encoder rejected the picture format", hr, err);
  70
  71          hr = writer_->BeginWriting();
  72          if (FAILED(hr)) return fail("could not start writing", hr, err);
  73
  74          w_ = width; h_ = height; fps_ = fps; frame_ = 0;
  75          return true;
  76      }
```

Walk through it in the order the pipeline needs.

Checks first.

- `if (writer_) { err = "already recording"; return false; }` One recording at a time.
- `if (width < 16 || ... || (width & 1) || (height & 1))` The size must be **even** and at least 16. H.264 stores colour at half resolution (pairs of pixels share a colour: *4:2:0 subsampling*), and most encoders refuse odd sizes. `(width & 1)` tests the lowest bit: 1 means odd. The caller makes the size even with `(W - PANEL_W) & ~1`, which clears that bit.
- `startup(err)` starts COM and Media Foundation (once).

Creating the writer.

- `MultiByteToWideChar(CP_UTF8, ...)` converts the file name to wide characters, as Windows wants.
- `MFCreateAttributes(&attr, 2)` makes a small settings bag with room for two.
- `MF_READWRITE_ENABLE_HARDWARE_TRANSFORMS = TRUE`: "use the graphics card's video encoder if there is one", which is faster and frees the CPU. If there is none, the software

encoder is used automatically.

- `MF_SINK_WRITER_DISABLE_THROTTLING = TRUE`: "never drop frames to keep up". For a recording we want *every* frame, even if encoding is slow.
- `MFCreateSinkWriterFromURL(wpath, nullptr, attr, &writer_)` creates the writer and the (empty) file. It chooses the container from the extension `.mp4`. We then `Release()` the settings bag, which we no longer need, and test the result with `FAILED(hr)`.

What to store (the output type).

- `MFCreateMediaType(&out)` makes a media-type description. We set: major type = video; subtype = `MFVideoFormat_H264`; average bitrate; progressive (not interlaced); the frame size; the frame rate as the fraction `fps / 1`; and a pixel aspect ratio of `1 / 1` (square pixels).
- The frame size and rate are *pairs* of numbers. Media Foundation stores a pair in one 64-bit number with one value in the top half and the other in the bottom half. Our helper `pack(hi, lo)` does `((UINT64)hi << 32) | lo`.
- `writer_->AddStream(out, &stream_)` tells the writer "there will be a video stream with this format" and returns the stream's number. The type object is released afterwards.

What we will feed it (the input type).

- The same kind of description, but the subtype is `MFVideoFormat_RGB32`: plain 32-bit pixels, which is what our canvas holds. `SetInputMediaType` says "we will send you these". The writer looks for a converter and an encoder that bridge RGB32 to H.264. If none exists, this call fails: the reason we can report "encoder rejected the picture format".

Start.

- `BeginWriting()` opens the pipeline. Finally the size, rate and frame counter are remembered (`w_, h_, fps_, frame_ = 0`).

> **WHY?**
> **Why describe the input and output separately?** Because they are different things. What *you* have (raw pixels) is not what the *file* wants (compressed H.264). The Sink Writer sits between the two and inserts whatever conversion and encoding is needed. If you later wanted H.265 or a different container, only the output description would change.

13.8 addFrame: send one picture

ENCODER.HPP (LINES 78 TO 103)

```
  78      // pixels: top-down 32-bit (B,G,R,x) rows, `rowBytes` apart. Only the left w_ x h_
block is stored.
  79      bool addFrame(const void* pixels, int rowBytes) {
  80          if (!writer_) return false;
  81          const DWORD bytes = (DWORD)(w_ * h_ * 4);
```

```
  82          IMFMediaBuffer* buf = nullptr;
  83          if (FAILED(MFCreateMemoryBuffer(bytes, &buf))) return false;
  84          BYTE* dst = nullptr;
  85          buf->Lock(&dst, nullptr, nullptr);
  86          const BYTE* src = (const BYTE*)pixels;
  87          for (int y = 0; y < h_; ++y)                                    // Media
Foundation wants RGB bottom-up: flip rows
  88              std::memcpy(dst + (size_t)(h_ - 1 - y) * w_ * 4, src + (size_t)y *
rowBytes, (size_t)w_ * 4);
  89          buf->Unlock();
  90          buf->SetCurrentLength(bytes);
  91
  92          IMFSample* sample = nullptr;
  93          MFCreateSample(&sample);
  94          sample->AddBuffer(buf);
  95          sample->SetSampleTime(frame_ * 10000000LL / fps_);              // times are in
units of 100 nanoseconds
  96          sample->SetSampleDuration(10000000LL / fps_);
  97          HRESULT hr = writer_->WriteSample(stream_, sample);
  98          sample->Release();
  99          buf->Release();
 100          if (FAILED(hr)) return false;
 101          ++frame_;
 102          return true;
 103      }
```

- `const DWORD bytes = w_ * h_ * 4;` The size of one frame in bytes.
- `MFCreateMemoryBuffer(bytes, &buf)` allocates a block of memory that Media Foundation owns. `buf->Lock(&dst, ...)` gives us a pointer to it so we can write into it; `Unlock` hands it back.
- **The row flip.** The loop `for (int y = 0; y < h_; ++y) memcpy(dst + (h_ - 1 - y) * w_ * 4, src + y * rowBytes, w_ * 4);` copies the picture row by row, **top row of our canvas to the bottom of the buffer**. Our canvas stores rows top to bottom; Media Foundation expects uncompressed RGB rows **bottom to top** (a tradition from the old BMP file format). Without the flip every video would be upside down.
- `(size_t)y * rowBytes` uses `rowBytes` (the canvas's row length) for the source, so the encoder can copy a **left part** of a wider canvas: our video leaves out the control panel by copying only the first `w_` pixels of each row.
- `buf->SetCurrentLength(bytes)` says how much of the buffer is valid.
- **The sample.** `MFCreateSample(&sample)` makes a container for "one unit of media data with a time stamp"; `AddBuffer` puts our pixels in it.
- **The times.** `sample->SetSampleTime(frame_ * 10000000LL / fps_);` Media Foundation measures time in units of **100 nanoseconds**, so one second is 10 000 000. The time of frame *n* at 60 fps is `n * 10000000 / 60`. We compute it **from the frame number** each time rather than adding 1/60 repeatedly: repeated addition of a rounded value would *drift*, whereas multiplying fresh from the integer frame number is exact to within one unit. `SetSampleDuration(10000000LL / fps_)` says how long the frame is shown.
- `writer_->WriteSample(stream_, sample)` sends it to the encoder. Then **both the sample and the buffer are released**; the writer keeps what it needs. Releasing a frame's

worth of memory 60 times a second is essential: forgetting would leak about 200 MB per second.

- `++frame_;` counts the frame only after success.

13.9 close: finish the file

ENCODER.HPP (LINES 105 TO 111)

```
105      bool close() {
106          if (!writer_) return false;
107          HRESULT hr = writer_->Finalize();                              // writes the
index so players can open the file
108          writer_->Release();
109          writer_ = nullptr;
110          return SUCCEEDED(hr);
111      }
```

- `writer_->Finalize()` flushes any frames still inside the encoder and writes the index (the `moov` box). **Without this call the MP4 is unplayable**, because players need the index to find the frames. `Release()` then frees the writer; the pointer is set to `nullptr` so a second `close()` does nothing harmful.

> **WHAT IF IT WERE DIFFERENT?**
>
> **What if the program crashes, or the power fails, while recording?** The index is never written, and the file will not open. That is a real weakness of the MP4 format. A safeguard, if you want one, is to record in short **segments** (say, 30 seconds each) so a crash loses at most one segment. The program does handle the *polite* ways of ending: Stop Record, closing the window (`WM_DESTROY` calls `stopRecording`), and the destructor.

13.10 The private part: helpers and cleanup

ENCODER.HPP (LINES 113 TO 119)

```
113  private:
114      IMFSinkWriter* writer_ = nullptr;
115      DWORD stream_ = 0;
116      int w_ = 0, h_ = 0, fps_ = 0;
117      long long frame_ = 0;
118
119      static UINT64 pack(UINT32 hi, UINT32 lo) { return ((UINT64)hi << 32) | lo; }    //
Media Foundation stores pairs this way
```

ENCODER.HPP (LINES 121 TO 129)

```
121      static bool startup(std::string& err) {
122          static bool done = false, ok = false;
123          if (done) { if (!ok) err = "Media Foundation failed to start"; return ok; }
124          done = true;
125          CoInitializeEx(nullptr, COINIT_MULTITHREADED);
126          ok = SUCCEEDED(MFStartup(MF_VERSION));
127          if (!ok) err = "Media Foundation failed to start";
128          return ok;
129      }
```

ENCODER.HPP (LINES 130 TO 134)

```
130      static std::string hrText(const char* what, HRESULT hr) {
131          char b[160];
132          std::snprintf(b, sizeof b, "%s (error 0x%08lX)", what, (unsigned long)hr);
133          return b;
134      }
```

ENCODER.HPP (LINES 135 TO 139)

```
135      bool fail(const char* what, HRESULT hr, std::string& err) {
136          err = hrText(what, hr);
137          if (writer_) { writer_->Release(); writer_ = nullptr; }
138          return false;
139      }
```

- The private fields: the writer pointer `writer_`, the stream number, size, rate and frame counter. A trailing underscore (`writer_`) is a naming habit for "this is a member variable".
- **`startup`** uses `static bool done, ok`: variables that survive between calls, so the expensive start-up runs **once** however many recordings you make. `CoInitializeEx(nullptr, COINIT_MULTITHREADED)` starts COM; `MFStartup(MF_VERSION)` starts Media Foundation and checks that the library version matches the headers.
- **`hrText`** turns a failure code into readable text: `"encoder rejected the picture format (error 0x...)"`. The hexadecimal code can be looked up if the message is not enough.
- **`fail`** is the single place that cleans up after a failed `open`: it builds the error text and releases the half-built writer, so every error path in `open` is one line: `return fail("...", hr, err);`.

13.11 Connecting the recorder to the program (live.cpp)

LIVE.CPP (LINES 74 TO 76)

```
 74  static VideoEncoder encoder;
 75  static int recW = 0, recH = 0;
 76  static const int REC_FPS = 60;
```

- One global `encoder` object, the recording size (`recW, recH`), and `REC_FPS = 60`.

LIVE.CPP (LINES 78 TO 85)

```
 78  static void startRecording() {
 79      recW = (W - PANEL_W) & ~1;          // record the drawing + plots only (not the
control panel), even size
 80      recH = H & ~1;
 81      std::string path = recordingsDir() + "/vibration_" + stamp() + ".mp4", err;
 82      if (!encoder.open(path, recW, recH, REC_FPS, err)) { toast("Cannot record: " +
err); return; }
 83      A.recording = true; A.recFrames = 0; A.recPath = path;
 84      toast("Recording... press Record again (or F9) to finish");
 85  }
```

- **The size.** `recW = (W - PANEL_W) & ~1; recH = H & ~1;` The video holds the scene and plots **without the control panel**, cut to an even size. The size is fixed *at the moment*

recording starts: if you resize the window later, the video size cannot change.

- The path is `recordings/vibration_YYYYMMDD_HHMMSS.mp4`. `encoder.open(...)` either starts the encoder or returns the error text, which becomes a toast: "Cannot record: ...".
- On success, set `A.recording = true`, reset the frame counter, store the path, and tell the user how to stop.

LIVE.CPP (LINES 86 TO 91)

```
  86  static void stopRecording() {
  87      if (!A.recording) return;
  88      A.recording = false;
  89      bool ok = encoder.close();
  90      toast(ok ? "Saved video: recordings/" +
A.recPath.substr(A.recPath.find_last_of('/') + 1) : "Video could not be finalised");
  91  }
```

LIVE.CPP (LINES 92 TO 92)

```
  92  void toggleRecording() { if (A.recording) stopRecording(); else startRecording(); }
```

- `stopRecording` clears the flag and calls `encoder.close()`, then reports where the file went (the toast shows only the file name, found by cutting the path after its last `/`) or that finalising failed. It does nothing if no recording is active, so it is safe to call from `WM_DESTROY` unconditionally.
- `toggleRecording` is the function declared in `app.hpp`: the Record button and F9 call it.

LIVE.CPP (LINES 94 TO 100)

```
  94  // called once per drawn frame while recording
  95  static void captureFrame() {
  96      if (!A.recording) return;
  97      if (canvas.w < recW || canvas.h < recH) { stopRecording(); toast("Window got
smaller than the video: recording stopped"); return; }
  98      if (!encoder.addFrame(canvas.bits, canvas.w * 4)) { stopRecording(); toast("Encoder
error: recording stopped"); return; }
  99      ++A.recFrames;
 100  }
```

- **Safety check:** if the canvas became smaller than the video (the user shrank the window), stop, because copying `recW × recH` pixels from a smaller buffer would read memory that is not ours.
- `encoder.addFrame(canvas.bits, canvas.w * 4)` passes the canvas pixels and the length of a row in bytes (`w * 4`). On error: stop and say so.
- `++A.recFrames` counts the frame; the REC badge converts it to minutes and seconds.

13.12 The recording clock: why the video has the right speed

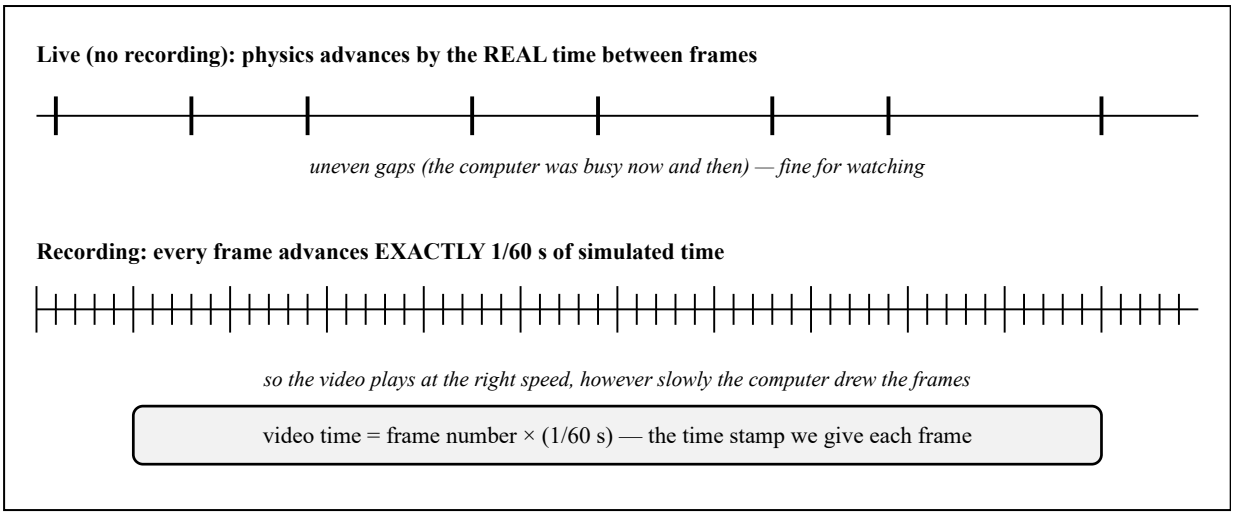


Figure 13.3. *When not recording, the physics advances by the real time between frames. While recording, every frame advances exactly 1/60 s, and gets the time stamp frame × (1/60 s).*

This is the most important design decision in the recorder. Think about what could go wrong. Suppose drawing a frame and encoding it takes 25 ms on a slow computer, and we advanced the physics by the *real* elapsed time. Then each video frame would show 25 ms of motion but be *played* for 16.7 ms: the video would run too fast.

Our solution is in the main loop: `advanceSim(A.recording ? 1.0 / REC_FPS : dt)`. While recording, **every frame advances the physics by exactly 1/60 of a second**, and the encoder stamps it as frame number n at $n/60$ s. The video is therefore an exact record of the simulation, no matter how slow the computer was while making it. The loop then **paces** itself to 60 frames per second (it sleeps if a frame took less than 16.7 ms) so that the app still *feels* real-time to you while you drag things.

> **WHAT IF IT WERE DIFFERENT?**
>
> **What if we recorded with the real elapsed time and variable time stamps?** The video would be correct in principle but jerky: many players struggle with variable frame rates, and the physics would take uneven steps. Fixed steps give smooth, repeatable video.

13.13 How we verified the recorder

A recorder that produces a file is not enough; the file must be *correct*. We checked in four ways:

1. **Orientation and colour test.** We wrote 90 test frames in which the top 40 rows are a green band and a red square moves to the right. We decoded the file with a video player (a Chromium `<video>` element), grabbed the frame at 0.5 s and looked at it: the green band

was at the **top** (not flipped), the colours were right (not swapped), and the square was at the right place for frame 30. That test is what proved the row-flip loop correct.

2. **Real recording.** Using the headless mode, `vibration_live --demo out.mp4 --preset 3 --run 6` recorded 6 seconds of the real interface: 360 frames, 1020 × 800, 5.0 million bytes, a duration of 5.999983 s. Frames at 0.2 s, 1.0 s, 2.5 s, 3.9 s and 5.5 s all showed the real picture (the REC badge, the telemetry and the plots).
3. **The built-in self-test.** Check 8 of the self-test writes 30 frames and requires the file to exist and be larger than 1000 bytes.
4. **Test the tests.** In our first attempt to inspect some frames, some came out blank. The *video* was fine; our *checker* had grabbed the picture before the player had finished drawing it. After adding a short wait, every frame decoded. The lesson: when a test fails, suspect the test as well as the code.

13.14 Chapter checks

QUIZ

1. Why is raw video too big to store? Give the calculation.
2. What is the difference between a codec and a container?
3. Why must the video width and height be even?
4. What does `Release()` do, and what happens if you forget it?
5. Why does `addFrame` flip the rows?
6. Why are frame times computed from the frame number?
7. What does `Finalize()` do, and why does a crash while recording lose the file?
8. Why does the recording advance the physics by exactly 1/60 s per frame?

ANSWERS

1. 1020 × 800 pixels × 4 bytes × 60 frames per second ≈ 196 MB per second, nearly 12 GB per minute.
2. The codec compresses the pictures (H.264); the container (MP4) holds the compressed data plus an index.
3. H.264 stores colour at half resolution (4:2:0), which needs even dimensions.
4. It lowers a COM object's user count; at zero the object frees itself. Forgetting it leaks memory.
5. Our canvas stores the top row first; Media Foundation wants uncompressed RGB with the bottom row first.
6. Adding 1/60 repeatedly would accumulate rounding error (drift); multiplying the integer frame number is exact.
7. It flushes the encoder and writes the index. Without the index players cannot find any frames.

8. So the video's speed is exactly right however slowly the computer produced the frames.

CHAPTER 14

Testing: How We Know It Is Right

Anyone can write a program that produces numbers. The real question is: *are the numbers true?* This chapter shows how the Vibration Lab proves itself with **nine automatic checks** that run in a fraction of a minute, without opening a window.

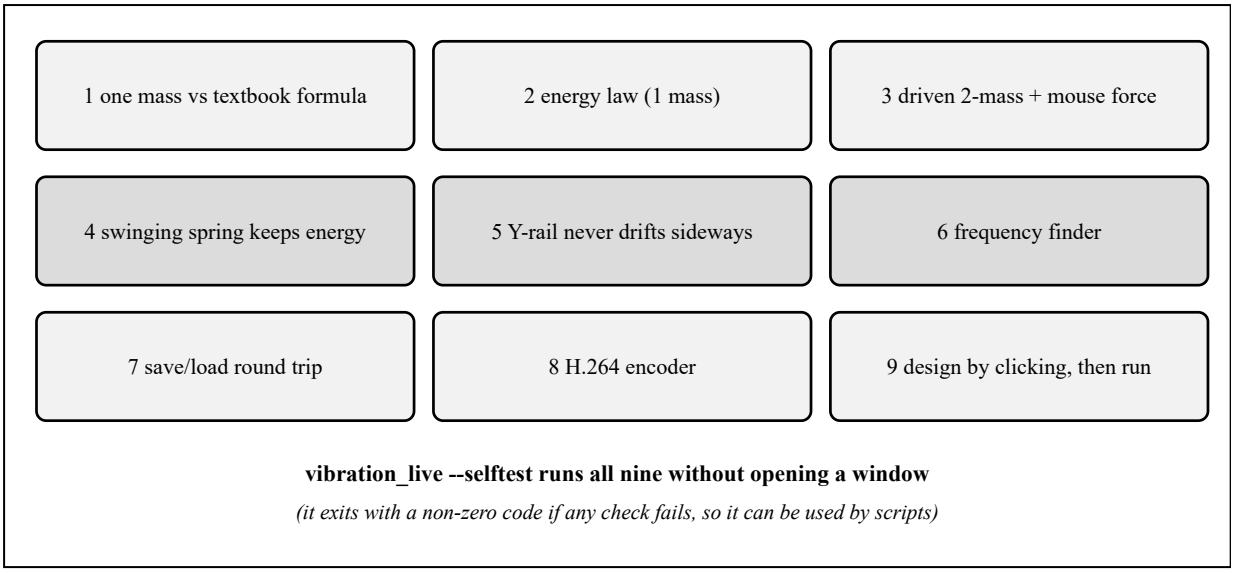


Figure 14.1. *The nine checks, in the order they run.*

14.1 Running the tests

```
vibration_live.exe --selftest
```

The program runs the checks, prints one line per check marked `[PASS]` or `[FAIL]`, prints a summary, and **exits with code 0 if everything passed or 1 if anything failed**. A script or a build system can therefore use it directly.

14.2 The test machinery

LIVE.CPP (LINES 203 TO 207)

```
203  static int gFail = 0;
204  static void check(const char* name, bool ok, const char* detail) {
205      std::printf("  [%s] %s  %s\n", ok ? "PASS" : "FAIL", name, detail);
206      if (!ok) ++gFail;
207  }
```

- `gFail` counts failures. `check(name, ok, detail)` prints the line (`[PASS]` or `[FAIL]`, the name, and a detail string with the measured number) and increments `gFail` if `ok` is false. All nine checks go through this one function, so the output is uniform.

14.3 Check 1 and 2: one mass against the textbook

LIVE.CPP (LINES 212 TO 228)

```
 212      {   // 1) single mass vs the textbook formula
 213          lag::Model M = lag::presetOneDof();
 214          lag::State s = lag::initialState(M);
 215          lag::Meter meter; meter.reset(M, s);
 216          std::vector<double> ext(s.q.size(), 0.0);
 217          const double m = 2.0, k = 30.0, c = 1.5, wn = std::sqrt(k / m), zeta = c / (2 *
std::sqrt(k * m)), wd = wn * std::sqrt(1 - zeta * zeta);
 218          double worst = 0;
 219          for (int i = 0; i < 10000; ++i) {
 220              lag::advance(M, s, ext, 0.001, meter);
 221              double t = s.t, u = std::exp(-zeta * wn * t) * (std::cos(wd * t) + zeta *
wn / wd * std::sin(wd * t));   // x0 = 1, v0 = 0
 222              worst = std::fmax(worst, std::fabs((-1.0 + u) - s.q[2]));
 223          }
 224          std::snprintf(d, sizeof d, "worst gap %.2e m after 10 s", worst);
 225          check("1-DOF matches theory", worst < 1e-6, d);
 226          std::snprintf(d, sizeof d, "energy-law error %.2e J",
std::fabs(meter.residual()));
 227          check("1-DOF energy law", std::fabs(meter.residual()) < 1e-4, d);
 228      }
```

- Loads example 1, makes a state and a meter, and a zero `ext` list (no mouse force).
- The constants `m, k, c` are the example's; `wn, zeta, wd` are computed from equation (3.16).
- The loop advances **10 000 steps of 1 ms** (10 seconds). At every step it computes the exact answer from formula (3.17): `u = e^(-ζωn t) (cos ωd t + ζωn/ωd sin ωd t)`, for $x_0 = 1$ and $v_0 = 0$. The mass is at `-1 + u` because its equilibrium is 1 m to the left of where it started (rest length 2 m, equilibrium position −1, release at 0). It keeps the **worst gap** between exact and simulated.
- The result: **worst gap 6.91 × 10⁻¹² m**. The required bound is 10^{-6} m, a million times looser; we are far inside it.
- Check 2 uses the same run: the meter's residual must be below 10^{-4} J. We measured **3.70 × 10⁻⁹ J**.

WHY?

Why test against a formula and an energy law? They catch different bugs. A wrong sign in a spring force would break both. A missing damper term might keep the shape right but fail the energy law; a wrong time-step bookkeeping might pass the formula but break the energy law. Using two independent yardsticks makes it very unlikely that a bug slips through both.

14.4 Check 3: a driven pair with a wiggling mouse

LIVE.CPP (LINES 229 TO 243)

```
 229      {   // 2) two masses, driven, with mouse-like extra force: the energy law must
still hold
 230          lag::Model M = lag::presetTwoDof();
 231          M.nodes[1].driveF = 6; M.nodes[1].driveW = 5;
 232          lag::State s = lag::initialState(M);
 233          lag::Meter meter; meter.reset(M, s);
 234          std::vector<double> ext(s.q.size(), 0.0);
 235          double worst = 0;
 236          for (int i = 0; i < 20000; ++i) {
 237              ext[2 * 2] = 2.0 * std::sin(0.002 * i);                          // wiggle the
second mass in x as well
 238              lag::advance(M, s, ext, 0.001, meter);
 239              worst = std::fmax(worst, std::fabs(meter.residual()));
 240          }
 241          std::snprintf(d, sizeof d, "worst error %.2e J over 20 s (E up to %.1f J)",
worst, meter.E);
 242          check("2-DOF driven energy law", worst < 5e-3, d);
 243      }
```

- Takes example 2, adds a drive force on mass 1 (`driveF = 6`, `driveW = 5`), *and* every step applies an extra, time-varying force on the second mass in x, `2.0 * sin(0.002 * i)`, imitating a mouse.
- Over 20 seconds it records the worst energy-law error. Result: **1.01×10^{-5} J** against a limit of 5×10^{-3} J. This proves that the drive, the mouse force, the dampers and RK4 all agree with (3.14).

14.5 Check 4: a frictionless spring pendulum

LIVE.CPP (LINES 244 TO 254)

```
 244      {   // 3) swinging spring pendulum without any damper: total energy must stay put
 245          lag::Model M = lag::presetSpringPendulum();
 246          for (size_t i = M.links.size(); i-- > 0;) if (M.links[i].kind ==
lag::LinkKind::Damper) M.removeLink((int)i);
 247          lag::State s = lag::initialState(M);
 248          lag::Meter meter; meter.reset(M, s);
 249          std::vector<double> ext(s.q.size(), 0.0);
 250          double worst = 0;
 251          for (int i = 0; i < 30000; ++i) { lag::advance(M, s, ext, 0.001, meter); worst
= std::fmax(worst, std::fabs(meter.E - meter.E0)); }
 252          std::snprintf(d, sizeof d, "energy wandered by %.2e J (E0 = %.3f J)", worst,
meter.E0);
 253          check("spring pendulum conserves energy", worst < 5e-3, d);
 254      }
```

- Example 3 with its damper **removed**: there is gravity and a spring, but nothing to burn energy. Equation (3.13) says E must stay constant forever.
- Over 30 seconds of a swinging, bouncing, partly chaotic motion the total energy wandered by **1.7×10^{-9} J** out of 23.467 J: about one part in ten billion.
- This checks the **non-linear geometry** (the angle changes, the unit vector rotates) and the gravity term, which the straight-line tests do not touch.

14.6 Check 5: rails

LIVE.CPP (LINES 255 TO 264)

```
 255      {   // 4) rails really remove a degree of freedom
 256          lag::Model M = lag::presetQuarterCar();
 257          lag::State s = lag::initialState(M);
 258          lag::Meter meter; meter.reset(M, s);
 259          std::vector<double> ext(s.q.size(), 0.0);
 260          for (int i = 0; i < 3000; ++i) lag::advance(M, s, ext, 0.001, meter);
 261          double sideways = std::fabs(s.q[2]) + std::fabs(s.q[4]);          // x of wheel
and body: must be exactly where they started (0)
 262          std::snprintf(d, sizeof d, "sideways drift %.1e m", sideways);
 263          check("Y-rail masses never move sideways", sideways < 1e-12, d);
 264      }
```

- The quarter-car has both masses on Y rails. After 3 seconds, their x coordinates must be **exactly** as they started (0). The measured sideways drift is **0.0 m**, exactly zero, not merely small. That is expected: the blocked acceleration is set to exactly zero and the blocked velocity starts at exactly zero, so every arithmetic step adds zero.

14.7 Check 6: the frequency finder

LIVE.CPP (LINES 265 TO 271)

```
 265      {   // 5) frequency finder
 266          std::vector<double> x;
 267          for (int i = 0; i < 360; ++i) x.push_back(std::sin(2 * lag::PI * 2.5 * i /
60.0) * std::exp(-0.05 * i / 60.0));
 268          double f = lag::dominantFrequency(x, 60.0);
 269          std::snprintf(d, sizeof d, "found %.3f Hz (true 2.5)", f);
 270          check("frequency finder", std::fabs(f - 2.5) < 0.06, d);
 271      }
```

- Builds a **known** signal: a 2.5 Hz sine, fading slowly, 360 samples at 60 per second. The finder must return 2.5 Hz to within 0.06 Hz. It returned **2.500 Hz**.
- Testing a measuring tool on a signal whose answer you know is the oldest trick in engineering.

14.8 Check 7: save and load

LIVE.CPP (LINES 272 TO 279)

```
 272      {   // 6) save / load round trip
 273          lag::Model a = lag::presetQuarterCar(), b;
 274          std::string p = recordingsDir() + "/_selftest.vlab";
 275          bool ok = lag::saveModel(p, a) && lag::loadModel(p, b);
 276          ok = ok && a.nodes.size() == b.nodes.size() && a.links.size() == b.links.size()
&& std::fabs(a.links[2].k - b.links[2].k) < 1e-6 && std::fabs(a.gravity - b.gravity) < 1e-9;
 277          std::remove(p.c_str());
 278          check("design file round trip", ok, "");
 279      }
```

- Saves the quarter-car to a temporary file, loads it into a new model, and checks that the node count, link count, one stiffness and the gravity survived. Then deletes the temporary

file. A **round trip** test catches mistakes in either direction, such as a field written but never read.

14.9 Check 8: the video encoder

LIVE.CPP (LINES 280 TO 293)

```
 280      {   // 7) the video encoder
 281          std::string p = recordingsDir() + "/_selftest.mp4", err;
 282          VideoEncoder e;
 283          bool ok = e.open(p, 320, 240, 30, err);
 284          std::vector<unsigned char> px(320 * 240 * 4, 128);
 285          for (int f = 0; ok && f < 30; ++f) ok = e.addFrame(px.data(), 320 * 4);
 286          ok = e.close() && ok;
 287          std::ifstream in(p, std::ios::binary | std::ios::ate);
 288          long long size = in ? (long long)in.tellg() : 0;
 289          in.close();
 290          std::remove(p.c_str());
 291          std::snprintf(d, sizeof d, "%s (%lld bytes)", ok ? "30 frames written" :
err.c_str(), size);
 292          check("H.264 video encoder", ok && size > 1000, d);
 293      }
```

- Opens a 320 × 240 file at 30 fps, writes 30 grey frames, closes it, and requires the file to exist and exceed 1000 bytes. We measured **250 857 bytes** (a solid grey frame compresses well, but the file also contains headers).
- The temporary file is deleted afterwards.

14.10 Check 9: designing by clicking, then running

LIVE.CPP (LINES 294 TO 317)

```
 294      {   // 8) design a system from scratch using only (simulated) mouse clicks, then
run it
 295          W = 1320; H = 800;
 296          resetView();
 297          A.model = lag::Model();
 298          A.mode = MODE_EDIT; A.selKind = SEL_NONE; A.sel = -1; A.linkStart = -1;
 299          auto click = [](double wx, double wy) { onMouseDown(toScreenX(wx),
toScreenY(wy)); onMouseUp(); };
 300          doAction(ACT_TOOL_GROUND); click(-2, 0);                              // a wall
 301          doAction(ACT_TOOL_MASS);   click(1, 0);                               // a block
 302          doAction(ACT_TOOL_SPRING); click(-2, 0); click(1, 0);                 // spring
wall -> block
 303          doAction(ACT_TOOL_DAMPER); click(-2, 0); click(1, 0);                 // damper
wall -> block
 304          doAction(ACT_TOOL_SELECT); click(1, 0);                               // select
the block
 305          bool built = A.model.nodes.size() == 2 && A.model.links.size() == 2 &&
A.selKind == SEL_NODE;
 306          A.model.nodes[1].rail = lag::RAIL_X;
 307          A.model.nodes[1].vx0 = 2.0;
 308          A.model.links[0].k = 25;
 309          startRun();
 310          for (int i = 0; i < 600; ++i) advanceSim(1.0 / 60);                    // 10 s at
60 frames per second
 311          bool ran = A.mode == MODE_RUN && A.st.t > 9.9 && A.log.size() > 500 &&
std::fabs(A.meter.residual()) < 1e-4;
 312          // a click on empty space with the Spring tool must not crash or connect
anything
 313          stopRun();
```

```
 314          doAction(ACT_TOOL_SPRING); click(3.3, 1.1);
 315          std::snprintf(d, sizeof d, "nodes %zu, links %zu, t = %.2f s, energy error %.1e
J", A.model.nodes.size(), A.model.links.size(), A.st.t, std::fabs(A.meter.residual()));
 316          check("design by clicking, then run", built && ran, d);
 317      }
```

This is the most unusual test: it **simulates the mouse**. It never opens a window. Instead it calls the same functions a real click would call:

- `click(wx, wy)` is a tiny lambda that converts a world position to screen pixels and calls `onMouseDown` followed by `onMouseUp`.
- The test picks the Ground tool and clicks at (−2, 0); picks the Mass tool and clicks at (1, 0); picks the Spring tool and clicks the anchor then the mass; the Damper tool the same way; then the Select tool and clicks the mass.
- It checks that the model now has **2 nodes and 2 links** and that a node is selected, sets a few properties, presses Run, and advances **600 frames of 1/60 s** (10 seconds) through the same `advanceSim` the real program uses. It then checks that the mode is still RUN, the time is above 9.9 s, more than 500 telemetry rows exist, and the energy-law error is below 10^{-4} J (**4.3×10^{-7} J** measured).
- Finally, with the Spring tool, it clicks on empty space at (3.3, 1.1): nothing should happen and nothing should crash.

> **WHY?**
> **Why this test matters.** The earlier eight check the physics and the recorder. This one checks that **a user can actually build a system with the interface and that the system then runs**. That is the main promise of the program. It catches broken click handling, tool logic and mode switching, which physics tests can never see.

14.11 What each test would catch

A mistake like this...	...is caught by
A wrong sign in a spring force	checks 1, 2, 3, 4
A damper that does not remove energy	check 2 (energy law), check 1 (formula)
An error in the gravity term	check 4
A rail that leaks	check 5
A wrong time-step or RK4 slip	checks 1, 2, 4
A file written but not read back	check 7
A video flipped or the encoder refused	check 8 and the orientation test
A click handler that does nothing	check 9
A frequency finder that always answers "0 Hz"	check 6

14.12 The test output

This is the real output of the final build:

```
Vibration Lab self-test
  [PASS] 1-DOF matches theory  worst gap 6.91e-12 m after 10 s
  [PASS] 1-DOF energy law  energy-law error 3.70e-09 J
  [PASS] 2-DOF driven energy law  worst error 1.01e-05 J over 20 s (E up to 1.6
J)
  [PASS] spring pendulum conserves energy  energy wandered by 1.70e-09 J (E0 =
23.467 J)
  [PASS] Y-rail masses never move sideways  sideways drift 0.0e+00 m
  [PASS] frequency finder  found 2.500 Hz (true 2.5)
  [PASS] design file round trip
  [PASS] H.264 video encoder  30 frames written (250857 bytes)
  [PASS] design by clicking, then run  nodes 2, links 2, t = 10.00 s, energy
error 4.3e-07 J

all checks passed
```

14.13 What the tests cannot tell you

Be honest about the limits. The tests prove the **physics is right**, the **recorder produces a valid video**, and the **click logic works**. They do **not** prove that the interface looks good or is comfortable, that every possible click sequence is safe, or that it works on every Windows version and every graphics driver. Those need a human using the program. The tests give a solid floor, not a ceiling.

14.14 Chapter checks

QUIZ

1. What exit code does `--selftest` return on failure, and why does it matter?
2. Why is the frictionless spring pendulum a good test?
3. Why is the sideways drift in check 5 exactly zero and not merely tiny?
4. How does check 9 test the interface without a window?
5. Name one thing the self-test cannot tell you.

ANSWERS

1. A non-zero code (1). Scripts and build systems can then detect failure automatically.
2. Energy must be exactly conserved, and the geometry is fully non-linear, which tests the force directions and the gravity term.
3. The blocked acceleration is set to exactly zero and the blocked velocity starts at exactly zero, so no step ever adds anything to it.

4. It calls the same click handlers (`onMouseDown`, `doAction`) the real mouse would, then runs the simulation through the same `advanceSim`.
5. Whether it looks good or is comfortable to use, or that it works on every Windows version and graphics driver.

Part V — Using It and Growing It

CHAPTER 15

The User Manual, with Experiments

This chapter is for the person at the keyboard. It teaches the program through six experiments. Each one has a prediction you can calculate by hand (using Chapter 3) and a result you can check against the telemetry.

15.1 Running the program inside NOVA

1. Open the folder `examples/vibration-lab` in NOVA (**File, Open Folder**).
2. The run-target box at the top should say `vibration_live`. If it does not, click it and choose it.
3. Press the green Run triangle. NOVA builds with CMake and g++ and then starts the program, which opens its own window.

> **NOTE**
> If the window seems not to appear, look in the Windows taskbar: it can open behind NOVA. NOVA's Run button hides the window of programs that are *building*, but shows the window of the program you *run*.

15.2 The screen at a glance

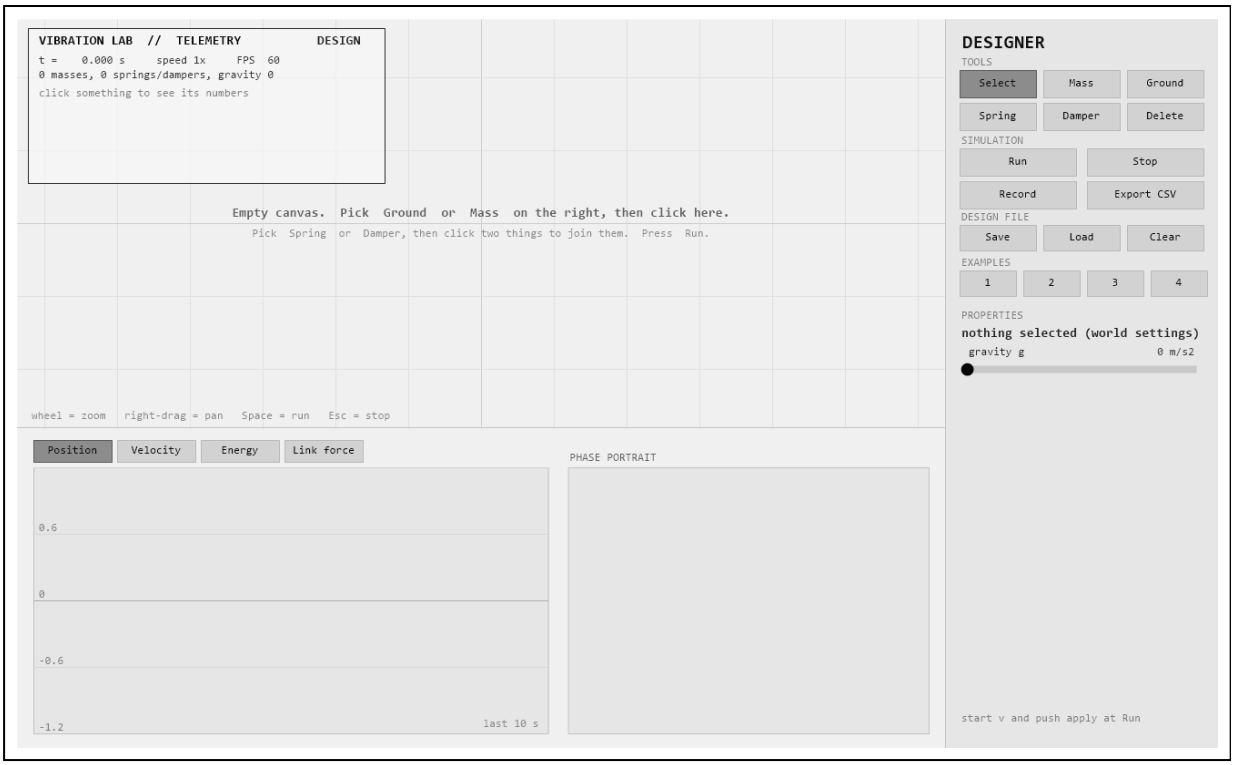


Figure 15.1. *The empty canvas, ready for you to design something.*

Area	What it is for
Canvas (top left)	Draw the system. Grid lines are one metre apart. Mouse wheel zooms, right-button drag pans, Home resets the view.
Telemetry box (top left of the canvas)	Time, energies, the energy-law error, and the numbers of the selected item.
Plot tabs and time plot	Position, Velocity, Energy, Link force: the last 10 seconds.
Phase portrait	Velocity against position of the selected mass.
Panel: TOOLS	Select, Mass, Ground, Spring, Damper, Delete.
Panel: SIMULATION	Run/Pause/Resume, Stop, Record, Export CSV.
Panel: DESIGN FILE	Save, Load, Clear.
Panel: EXAMPLES	Buttons 1 to 4 load the ready-made examples.
Panel: PROPERTIES	Sliders for whatever is selected, plus gravity.

15.3 Experiment 1: build a mass on a spring from scratch

Goal: see that the frequency you measure agrees with $\omega_n = \sqrt{(k/m)}$.

1. Press **Clear**. The canvas is empty.
2. Pick **Ground** and click once at the left of the canvas (a pin appears).

3. Pick **Mass** and click about three metres to the right of it (a blue block).
4. Pick **Spring**. Click the pin, then click the block. A spring appears. Pick **Damper** and do the same: a damper appears beside it.
5. Pick **Select** and click the block. In the panel set **motion** to **X rail**, and **start vx** to 3 m/s.
6. Press **Run** (or the space bar).

> **CHECKPOINT**
> The block slides back and forth, slowing down. The Position tab shows a decaying wave. The telemetry shows the **dominant frequency**. For the default spring (k = 20 N/m), the default mass (m = 1 kg) and the default damper (c = 0.8 N s/m):
>
> - $\omega_n = \sqrt{(20/1)} = 4.47$ rad/s, so $f_n = 4.47 / 2\pi =$ **0.712 Hz**;
> - $\zeta = 0.8 / (2\sqrt{20}) = 0.089$, lightly damped;
> - the damped frequency is almost the same, 0.709 Hz.
>
> The dominant-frequency readout should settle near **0.71 Hz**.

Now change something and predict first: select the block and raise its mass to 4 kg. The frequency should **halve** ($\omega_n \propto 1/\sqrt{m}$). Change the spring's stiffness to 80 N/m with the original mass: it should **double**.

15.4 Experiment 2: resonance

Goal: see a small push build a large motion.

Use the system from Experiment 1 (k = 20, m = 1, c = 0.8). Select the block and set **push force** to 5 N, **push angle** to 0° (to the right) and **push freq** to **4.5 rad/s**, which is the natural frequency.

> **CHECKPOINT**
> After about 10 seconds the motion settles to a steady swing of about **1.4 metres** each side. The theory: at resonance the steady amplitude is $F / (c\,\omega) = 5 / (0.8 \times 4.47) =$ **1.40 m**. A push of only 5 N, applied at the right rhythm, moves the block 1.4 m.

Now change the push frequency to 9 rad/s (twice as fast). The amplitude shrinks to under **0.1 m**: the same push at the wrong rhythm does almost nothing. Open the **Energy** tab to see the story: at resonance the total energy climbs and then flattens when the energy fed in each cycle equals the heat burned in the damper. That is the energy law (3.13) in action.

WHAT IF IT WERE DIFFERENT?

What if the damper were removed (c = 0)? At resonance the amplitude would grow without limit: the push feeds energy every cycle and nothing takes it out. Real bridges and buildings have been damaged by exactly this. The damper is what keeps resonance from being destructive.

15.5 Experiment 3: a car suspension

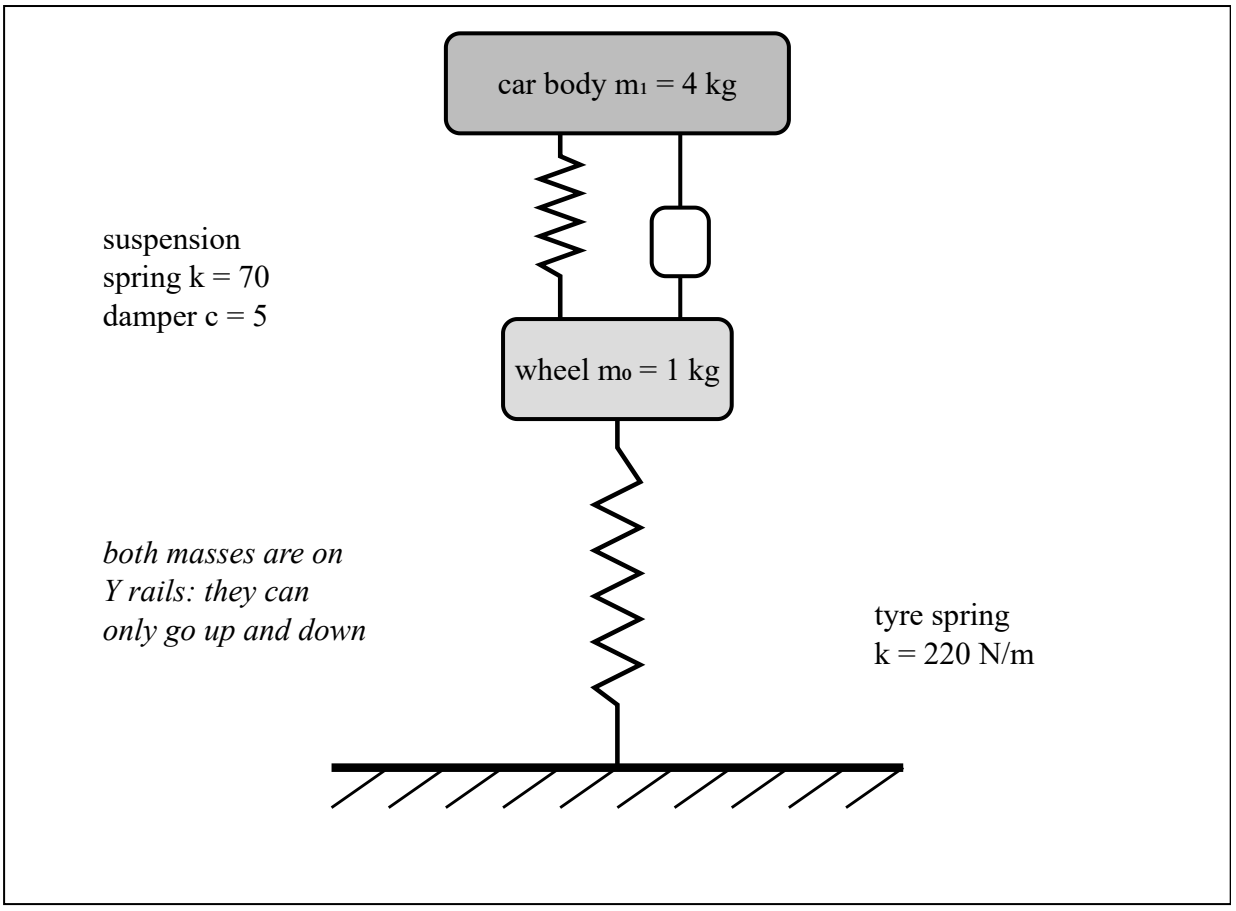


Figure 15.2. *The quarter-car model of example 4: a tyre spring, a wheel, a suspension spring and damper, and the car body. Both masses move only up and down.*

Press **4** (the quarter-car). Press Run and watch the wheel hit a bump. The body rides up and then settles.

- Select the **damper** and move **damping c** while it runs (sliders work *live*): c = 1 makes the car bounce many times; c = 5 (the default) settles in about a second; c = 25 makes it stiff and slow.
- Open the **Link force** tab with the damper selected. The force curve is the number a shock-absorber designer cares about.
- Raise the **body mass** (select the top block): a heavier car sags more. The static sag is m g / k, formula (3.18).

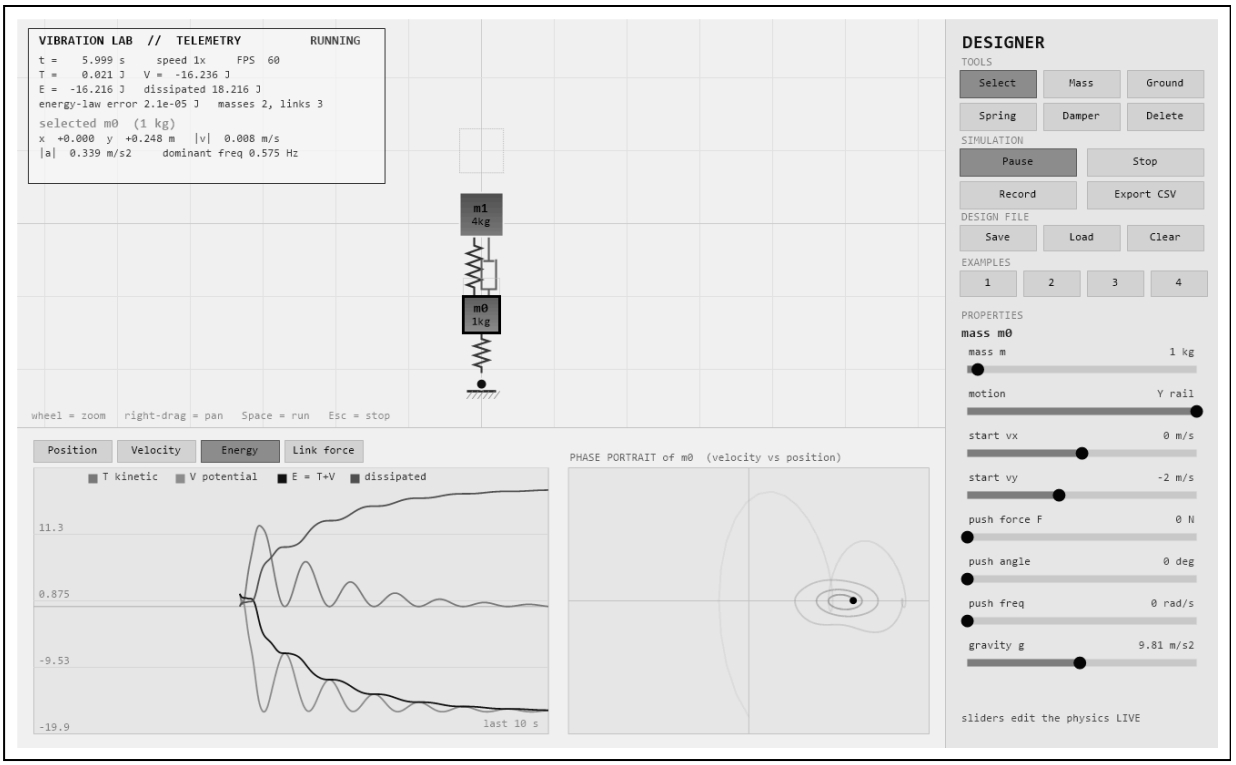


Figure 15.3. *The quarter-car's Energy tab. Kinetic and potential energy trade places while the dampers steadily burn the total away. Gravity has shifted the potential energy to negative values (the car has sagged below where it was drawn).*

15.6 Experiment 4: a hardening spring

Goal: see that for a non-linear spring the frequency depends on how far it swings.

Use Experiment 1's system with the damper's c set to a small number such as 0.05, and set the spring's **cubic k3** to 200. Make the spring start stretched by changing its **rest length** to a little less than the drawn length.

> **CHECKPOINT**
>
> With a small start (stretch 0.2 m) the frequency is close to the linear value, about **0.79 Hz**. With a large start (stretch 1 m) it rises to about **1.9 Hz**, more than double. The stiffer the spring gets as it stretches, the faster it vibrates. This amplitude-dependent frequency is the signature of the **Duffing oscillator**. A linear spring (k3 = 0) always gives 0.71 Hz, whatever the amplitude.

15.7 Experiment 5: a spring pendulum

Press **3**. The mass swings and bounces at once. Select the mass and open the Phase portrait: the curve never repeats exactly.

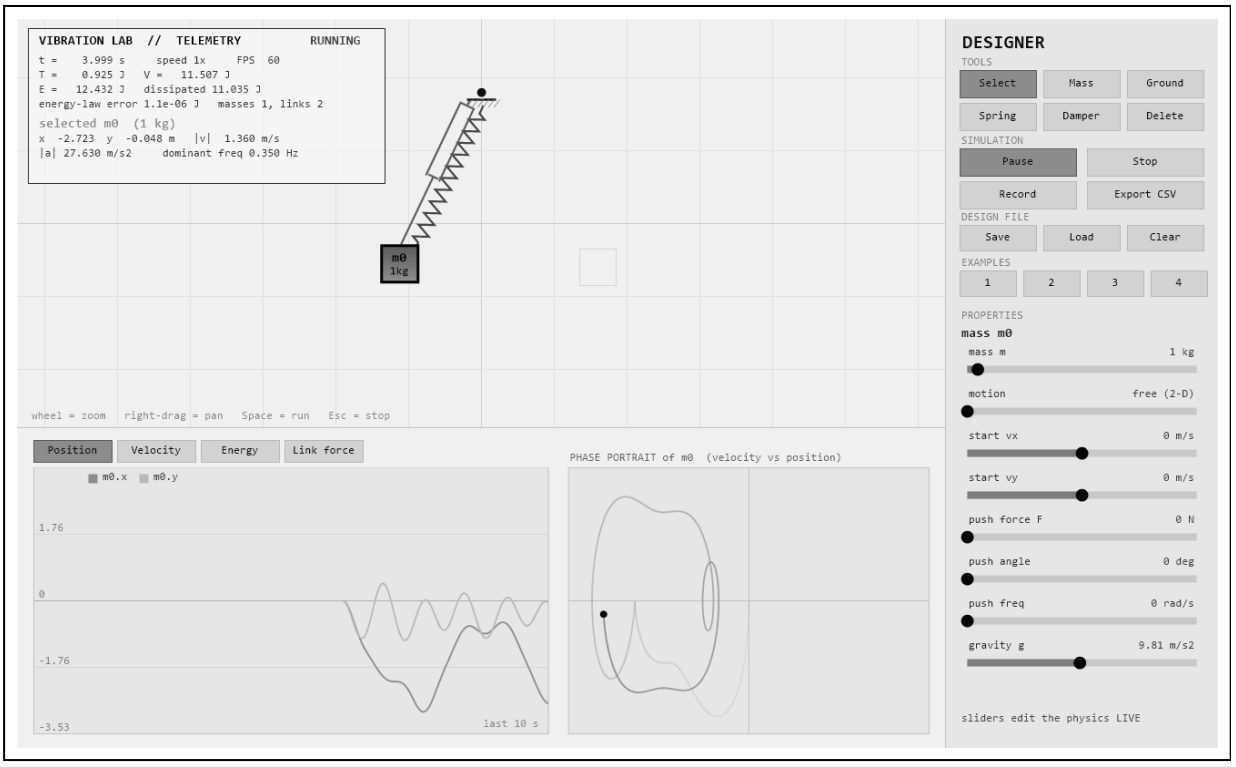


Figure 15.4. *The spring pendulum (example 3) after four seconds. The mass has swung and bounced, the plot shows both x and y, and the phase portrait has not closed on itself.*

Try this: note the picture, press Stop, nudge the mass by one grid step (0.25 m) with the Select tool, and run again. The two runs look the same at first and then separate. That sensitivity to tiny changes is a hallmark of **chaotic** motion, and it is the best reason this engine is two-dimensional.

15.8 Experiment 6: record a video

1. Design or load a system and press **Run**.
2. Press **Record** (or F9). A red dot and "REC 00:03" appear in the corner of the canvas.
3. Do something interesting: drag a mass, change a slider.
4. Press Record again. A message says where the file went.

The video is in the folder `examples/vibration-lab/recordings`, named `vibration_YYYYMMDD_HHMMSS.mp4`. Any player can open it (Windows Media Player, Movies & TV, a browser, PowerPoint). It contains the canvas and the plots, but not the control panel.

> **TIP**
> For a cleaner video, press Run first and then Record, so the video starts when the motion starts. To make a video of a pure replay with nothing but the physics, press Stop, press Record, then press Run.

15.9 What every number in the telemetry means

Number	Meaning
t	Simulated time, in seconds
T	Kinetic energy: motion
V	Potential energy: springs, and height if gravity is on
E = T + V	Total energy. It falls as dampers work and rises when you push
dissipated	Total heat made by dampers so far
energy-law error	How far E is from (start + work in − heat). **Should be tiny** (about 10^{-5} J or less)
x, y	Displacement of the selected mass from where it started
|v|, |a|	Speed and acceleration of the selected mass
dominant freq	The strongest vibration frequency, in hertz
length, stretch	For a spring: how long, and how far from its rest length
force	For a link: the tension. Positive means pulling its ends together
power lost	For a damper: the heat it is making this instant ($c\,\dot{s}^2$)

15.10 Keys and mouse

Input	Action
Space	Run / pause / resume
Esc	Cancel a half-drawn link, or stop the run, or go back to the Select tool
S, M, G, K, D	Tools: Select, Mass, Ground, sprinK, Damper
Delete or Backspace	Delete the selected item
1, 2, 3, 4	Load an example
F9	Start or finish recording
Tab	Next plot tab
, and .	Slower and faster (0.125× to 4×)
Home	Reset the view
Ctrl+S, Ctrl+O	Save / load `design.vlab`
Wheel	Zoom about the cursor
Right-button drag	Pan
Left-drag a mass (running)	Pull it with the mouse spring
Left-drag an item (designing, Select tool)	Move it, snapping to the 0.25 m grid

15.11 When something goes wrong

What you see	What it means and what to do
"Numbers blew up (too stiff or too light)"	A mass is very light with a very stiff spring, or a huge push. The program stopped itself. Raise the mass or lower the stiffness.
"Press Stop to edit the layout"	You tried to add or delete something while running. Press Stop first. Sliders and dragging masses still work.
"Cannot connect those two"	You tried to join a node to itself, two grounds, or repeat the same link.
The energy-law error is large	You changed a slider while running (it rebases) or something is wrong. Press Stop and Run again; if it still grows, report it.
Nothing happens when you press Run	There is no mass on the canvas.
The video looks wrong or will not open	The window may have been resized during recording (the program stops recording if it became smaller than the video). Record again.
The window never appears when started from NOVA	It may be behind NOVA, or you are running an old NOVA. Make sure the build you run is current.

15.12 Chapter checks

QUIZ

1. You double the mass on a spring. What happens to the natural frequency?
2. At resonance, what limits the amplitude?
3. In Experiment 4, why does the frequency rise with the starting stretch?
4. Why is the video width smaller than the window width?
5. What is the energy-law error and what size should it be?

ANSWERS

1. It falls by a factor $\sqrt{2}$: $f_n \propto 1/\sqrt{m}$, so about 0.707 of before.
2. The damper: at steady state the energy fed in per cycle equals the heat burned per cycle.
3. The cubic term $k_3 s^3$ makes the spring stiffer as it stretches, and a stiffer spring vibrates faster.
4. The control panel (300 pixels) is left out of the video; only the canvas and plots are recorded.
5. The difference between the measured energy and (start + work in − heat); it should be tiny, around 10^{-5} J or less.

CHAPTER 16

Where to Go Next

The engine was built so that each new feature is a **small addition**, not a rewrite. This chapter lists projects in increasing difficulty. For each, the basic steps are given so you can start.

16.1 Level 1: small and safe

1. Undo and redo. *Idea:* store a copy of the `Model` before every edit. *Steps:* (a) add `std::vector<lag::Model> undoStack` to `App`; (b) at the start of each function that changes the layout (`addMass` calls, `deleteSelection`, drags), push `A.model`; (c) Ctrl+Z pops the last model back; (d) Ctrl+Y for redo with a second stack. The design is only a few hundred bytes, so copying it is cheap: this is why `Model` is plain data with no pointers.

2. Type numbers into the sliders. *Steps:* add a small text box state (the property being edited and a string); on double-click, capture keyboard characters in `WM_CHAR`; on Enter parse the number with `std::stod` and write it through the same `Prop::p` pointer.

3. A parameter-sweep plot. *Idea:* the *frequency response*: amplitude against drive frequency. *Steps:* a loop that runs the engine headless (no window) for each frequency from 0.5 to 15 rad/s, records the steady amplitude, and plots it. The peak lands at ω_n. This uses only `lag::advance`, which is why the engine knows nothing about windows.

16.2 Level 2: new physics

4. More elements. Each new part is a new term in T, V, D or Q:

Element	Term	New force
Inerter (mass-like link)	$T = \tfrac{1}{2}\, b\, \dot{s}^2$	$b\, \ddot{s}$ along the link
Dry friction (Coulomb)	D-like, force $c \cdot \mathrm{sign}(\dot{s})$	constant magnitude, opposite to motion
Base motion	position of a ground anchor moves: $r(t)$	spring and damper see a moving end
Bump stop	$V = \tfrac{1}{2}\, k\, (s - s_0)^2$ only when $s > s_0$	one-sided spring

5. Rigid rods (pendulums, linkages). A rod fixes the distance between two masses: the constraint $|r_b - r_a| = \ell$. The proper Lagrangian way uses **Lagrange multipliers**. A simple first version replaces the rod by a *very stiff spring*, which works if you accept the stiffness limits of

Chapter 4 (a smaller step).

6. Angular coordinates and rigid bodies. Give a body an angle θ and a moment of inertia I: add $T = \tfrac{1}{2} I \dot{\theta}^2$. The generalized coordinates grow from (x, y) to (x, y, θ), and a link attached off-centre needs its attachment point rotated, as in the rotation trick of `drawLink`.

16.3 Level 3: better numerics

7. Energy-friendly integrators. RK4 slowly loses or gains energy over very long runs. A **velocity-Verlet** (symplectic) integrator keeps the energy bounded for conservative systems; swap it into `rk4()`'s place and compare with the Meter.

8. Adaptive steps (RK45). Choose the step from an error estimate: small steps when the motion is violent, large steps when it is calm. The accumulator in `advanceSim` already separates the step size from the frame rate.

9. Modal analysis. Linearise the system about rest, build the mass matrix M and the stiffness matrix K, and compute the eigenvalues of $M^{-1}K$. The square roots are the natural frequencies. This turns the "dominant frequency" guess into the full list for any design.

16.4 Level 4: the interface and the recorder

10. A live FFT plot. The frequency finder already computes the whole spectrum; add a fifth tab that plots it.

11. Audio. Turn the motion of a mass into a tone (the displacement is a waveform). Windows offers `waveOut` or WASAPI.

12. Audio inside the video. Add a second stream to the Sink Writer: `AddStream` with an AAC output type and a PCM input type. Time-stamp the audio samples with the same fixed clock.

13. A cross-platform version. Replace Win32 + GDI+ with SDL2 for the window and Cairo for the drawing, and ffmpeg's libraries for the video. The engine, `model_io.hpp` and `app.hpp` do not change: that is what separation of concerns buys.

14. A web version. Port `engine.hpp` to JavaScript (it is a few hundred lines of plain arithmetic), draw on a `<canvas>`, and record with the browser's `MediaRecorder`.

16.5 Level 5: larger projects

15. Finite elements. A beam is a long chain of masses and springs. Add a "chain" tool that places *n* masses and links between two points, then watch waves travel along it.

16. Optimisation. Write a program that varies a suspension's spring and damper and searches for the setting that gives the smoothest ride (smallest peak body acceleration over a bump). It would call `lag::advance` thousands of times, headless.

16.6 A habit for every project

Whatever you build, add a check to `--selftest` *first*. A new element is only finished when it passes a test against a formula or a conservation law.

16.7 Chapter checks

QUIZ

1. Why is undo easy in this program?
2. What new physics term would you add for a one-sided bump stop?
3. Why might a velocity-Verlet integrator be preferred for very long runs?
4. Which files would not change in a cross-platform port?

ANSWERS

1. `Model` is plain data with no pointers, so copying it before each edit is cheap and safe.
2. A spring-energy term that only exists when the stretch exceeds the gap: $V = \tfrac{1}{2} k (s - s_0)^2$ for $s > s_0$, otherwise 0.
3. It keeps the energy bounded instead of drifting slowly.
4. `engine.hpp`, `model_io.hpp` and `app.hpp` (the drawing, window and recorder files are the ones that change).

APPENDIX A

Glossary

Word	Meaning
Action (S)	The integral of the Lagrangian over time. The real path makes it stationary.
Accumulator	A bank of leftover real time, spent in fixed 1 ms physics steps (`simAcc`).
Anti-aliasing	Smoothing the edges of drawn shapes so they do not look jagged.
Bitrate	Bits of compressed video per second.
BitBlt	"Bit-block transfer": copy a rectangle of pixels from one place to another.
Canvas	In the program: the hidden bitmap everything is drawn into. In the window: the drawing area.
Codec	A compression method (H.264).
COM	Component Object Model: Microsoft's system of objects with reference counts (Media Foundation uses it).
Container	The file format that holds compressed video plus an index (MP4).
Critical damping	$\zeta = 1$: the fastest return to rest without overshoot.
CSV	Comma-separated values: a plain text table that spreadsheets open.
Damper (dashpot)	A part whose force opposes speed: $F = c\,\dot{s}$.
Damping ratio (ζ)	How strong the damping is compared with critical.
Degrees of freedom	The number of independent numbers describing the system's position.
Dissipation (D)	Rayleigh function $\frac{1}{2} c\,\dot{s}^2$; heat power is 2D.
DIB section	A bitmap whose pixel memory we can access directly.
Double buffering	Drawing to a hidden picture then copying it to the window in one move.
Duffing oscillator	A mass on a spring with a cubic term; its frequency depends on amplitude.
Euler–Lagrange equation	$d/dt(\partial L/\partial \dot{q}) - \partial L/\partial q = 0$ (plus $\partial D/\partial \dot{q} = Q$ with dissipation and forces).
Frame	One still picture of a video, or one row of telemetry.
Generalized coordinates	The list of numbers q describing where everything is.
Generalized force (Q)	Outside forces written for the chosen coordinates.
GDI+	Windows' built-in 2-D drawing library.
H.264	The most widely used video codec.
Hann window	A fade-in/fade-out weighting used before a Fourier test.

Word	Meaning
HRESULT	A COM result code: negative means failure.
Kinetic energy (T)	Energy of motion, ½ m v².
Lagrangian (L)	T − V.
Media Foundation	Windows' multimedia framework, including the H.264 encoder we use.
Message	A note from Windows about an event (mouse, key, resize).
Namespace	A labelled box of names, used to prevent clashes.
Natural frequency (ωn)	√(k/m): how fast it would vibrate without damping.
Nyquist limit	The highest frequency you can see in samples: half the sample rate.
Phase portrait	A plot of velocity against position.
Potential energy (V)	Energy stored by position (springs, height).
Rail	A constraint letting a mass move along only x or only y.
RAII	Resource Acquisition Is Initialisation: objects clean up after themselves.
Resonance	Large motion when a push matches the natural frequency.
RK4	Fourth-order Runge–Kutta, a method for stepping differential equations.
Sink Writer	The Media Foundation object that encodes and writes a media file.
State	The positions and velocities at one instant.
Stretch (s)	Length minus rest length of a spring.
Telemetry	The measured numbers (and plots) shown while running.
Toast	A short message that appears and fades.
Trapezoid rule	Integrating by averaging the two ends of each step.
Unit vector	A direction arrow of length exactly 1.
Window procedure	The function that receives a window's messages.

APPENDIX B

Maths to Code

Every symbol of the derivation, and the name it has in the program.

Symbol	Meaning	Code
q, v	positions, velocities	`State::q`, `State::v` (flat arrays, 2 per node)
m_i	mass	`Node::m`
r_a, r_b	positions of the two link ends	`q[2L.a]`, `q[2L.b]` (and +1 for y)
d	vector from a to b	`dx, dy` in `linkInfo`
ℓ	link length	`LinkInfo::len`
u	unit vector	`LinkInfo::ux, uy`
L_0	rest length	`Link::L0`
s	stretch	`LinkInfo::stretch`
ṡ	rate of change of length	`LinkInfo::rate`
k, k_3	linear and cubic stiffness	`Link::k`, `Link::k3`
c	damping coefficient	`Link::c`
f	tension in a link	`LinkInfo::force`
g	gravity	`Model::gravity`
F_0, θ, ω	drive amplitude, angle, frequency	`driveF`, `driveAngle`, `driveW`
Q	outside forces	`driveForce(...)` + `ext`
T	kinetic energy	`kinetic()`
V	potential energy	`potential()`
D	dissipation	`rayleigh()`
a	accelerations	`accel()` output
h	time step	`SIM_STEP` (0.001 s)
k_1…k_4	RK4 slopes	`a1..a4`, `v, v2, v3, v4`
E, E_0	total energy, start energy	`Meter::E`, `Meter::E0`
P_{in}	input power Σ Q·v	`inputPower()`, `Meter::workIn`
2D	heat power	`2.0 * rayleigh()`, `Meter::dissipated`
ω_n, ζ, ω_d	natural, damping ratio, damped	computed in `selfTest` and the telemetry

APPENDIX C

Files and Folders

```
examples/vibration-lab/
  CMakeLists.txt     one target: vibration_live (live.cpp)
  engine.hpp         the Lagrangian physics
  model_io.hpp       save / load / the four examples
  app.hpp            state, clicks, keys, sliders
  render.hpp         all drawing
  encoder.hpp        H.264 video encoder (Media Foundation)
  live.cpp           window, main loop, recording, CSV, self-test
  design.vlab        your last design (written when you close the window)
  recordings/        videos (.mp4) and telemetry (.csv)
  cmake-build-debug/ created by NOVA when it builds
```

Command	What it does
`vibration_live`	Opens the window
`vibration_live --selftest`	Runs the nine checks; exit code 0 means all passed
`vibration_live --shot out.png --preset 2 --run 5 --select node:1 --tab 0`	Draws one picture without a window
`vibration_live --demo out.mp4 --preset 3 --run 6`	Records a video without a window